

Sistemi Operativi

[Appunti di Piai Luca]

Academic Year 2022-2023

Indice

1	Introduzione	6
1.1	Ruolo del sistema operativo	6
1.2	Organizzazione del sistema di calcolo	6
1.2.1	Dispositivi I/O	7
1.2.2	Multiprocessori e multicore	7
1.2.3	Cluster	7
1.3	Struttura del sistema operativo	7
1.4	Operating-System Operations	8
1.4.1	Dual-mode e multimode operation	8
1.5	Gestione processi	8
1.6	Gestione memoria centrale	9
2	Struttura dei sistemi operativi	11
2.1	I servizi	11
2.2	Interfaccia utente e CLI	12
2.3	System Calls	12
2.3.1	Tipi di system call	13
2.4	Programmi di sistema	15
2.4.1	Linker e loader	15
2.5	Struttura sistema operativo	16
2.5.1	Approccio stratificato	16
2.5.2	Microkernel	17
2.5.3	Kernel modulari	17
2.5.4	Kernel ibridi	18
2.6	Debugging di un SO	18
3	Thread e concorrenza	19
3.1	Thread	19
3.1.1	Gestione thread	19
3.2	Programmazione multicore	20
3.3	Risorse	20
3.3.1	Deadlock, livelock e grafo delle risorse	21
3.3.2	Individuare un deadlock	23
3.3.3	Algoritmo del banchiere	24
4	Sincronizzazione processi	26
4.1	Il problema delle sezioni critiche	27
4.2	Soluzioni software	27
4.2.1	Semafori	28
4.2.2	Monitor	29

5	CPU scheduling	32
5.1	Processi e process scheduling	32
5.1.1	Stato di un processo	32
5.1.2	Il dispatcher	33
5.1.3	Scheduling CPU	33
5.1.4	Comunicazione tra processi	34
5.2	Algoritmi di scheduling	35
5.2.1	First Come First Served (FCFS)	35
5.2.2	Shortest Job First (SJF)	35
5.2.3	Round Robin (RR)	36
5.2.4	Shortest Process Next (SPN) e Shortest Remaning Time (SRT)	36
5.2.5	Highest Response Ratio Next (HRRN)	36
5.2.6	Code multilivello	37
5.3	Scheduling sistemi Real-Time	37
5.3.1	Scheduling con priorità proporzionale alla frequenza	37
5.3.2	Scheduling Earliest-Deadline-First (EDF)	38
5.4	Scheduling multiprocessore	38
6	Gestione memoria principale	40
6.1	Background	40
6.2	Allocazione contigua	41
6.3	Paginazione	42
6.3.1	Tabella delle pagine invertita	43
6.3.2	Paginazione gerarchica	44
6.3.3	Tabella delle pagine hash	45
6.4	Segmentazione	45
6.5	Memoria virtuale	46
6.5.1	Sostituzione delle pagine	48
6.5.2	Allocazione dei frame	51
6.5.3	Allocazione di memoria al Kernel	51
7	Gestione file	54
7.1	Introduzione	54
7.2	Operazioni sui file	54
7.2.1	Accesso ai file	55
7.3	Struttura directory	55
7.4	Operazione di montaggio	58
7.5	Struttura file system	59
7.6	Metodi di allocazione dei file	60
8	Gestione memoria secondaria	65
8.1	Generalità sui dischi	65
8.2	Lo scheduling del disco	66
8.2.1	FCFS	67
8.2.2	Shortest seek time first (SSTF)	67
8.2.3	Scan	68
8.2.4	C-Scan	68
8.2.5	C-Look	69
8.3	La formattazione	69
8.3.1	Gestione degli errori	69
8.4	Il bootstrap	70

8.5	RAID	70
8.6	Gestione dei blocchi liberi	72
9	Affidabilità dei dispositivi	74
9.1	Definizioni e formule	74
10	Introduzione a Go	75
10.1	Variabili	75
10.1.1	Puntatori	75
10.2	Slices	75
10.3	Funzioni	76
10.3.1	Metodi	77
10.4	Strutture	77
10.5	Defer e Panic	78
10.6	Goroutine	79
10.7	Channels	79
10.8	Select	80
10.9	Data races	81
10.10	WaitGroup	81

Capitolo 1

Introduzione

1.1 Ruolo del sistema operativo

Un sistema operativo è un software che funge da intermediario tra utente e hardware del computer. Deve gestire le richieste di uno o più utenti e gestire uno o più processi.

Il suo scopo è: eseguire programmi a supportando le richieste dell'utente; rendere agevole l'interfaccia per l'utente; gestire in maniera efficace le risorse software/hardware del sistema di calcolo; prevenire interferenze tra/con le attività degli utenti.

La struttura di un sistema operativo viene schematizzata attraverso un modello a cipolla, gli strati o gusci concentrici rappresentano programmi che operano a diversi livelli di interazione. Lo strato più esterno è la *shell*, è lo strato a contatto con l'utente. Nello strato più interno troviamo i dispositivi hardware, questi comunicano con il **kernel**, l'unico programma che è sempre in esecuzione quando il computer è acceso, tutti gli altri programmi sono software di sistema o applicativi. In generale gli strati possono variare a seconda del dispositivo che stiamo considerando.

All'accensione del sistema di calcolo (o in fase di reboot) deve essere caricato il programma di *bootstrap* (**BIOS**). Il bootstrap indica l'insieme dei processi che vengono eseguiti in fase di avvio, fino al caricamento in memoria primaria del kernel del sistema operativo a partire dalla memoria secondaria. Il BIOS viene tipicamente memorizzato nella ROM e viene identificato con il termine **firmware**. Un processo particolare che viene lanciato dal BIOS è il **bootloader**, questo è memorizzato su disco fisso (nel **Master Boot Record, MBR**) e ha il compito di caricare il codice del primo settore del sistema operativo scelto.

Quando il kernel è stato caricato ed è in esecuzione, può iniziare elaborare i servizi richiesti dell'utente. Il kernel non svolge qualsiasi servizio, alcuni vengono svolti da programmi di sistema esterni che vengono caricati sempre in fase di boot in memoria, questi prendono il nome di **system daemons**, rimangono attivi fintantoché il kernel è attivo.

Nei computer più moderni il BIOS è stato sostituito dal firmware **UEFI**. La tecnologia Uefi ha cambiato il ruolo del bootloader, anche la locazione in memoria è cambiata, infatti il firmware UEFI si trova fisicamente nella cartella efi contenuta nella relativa partizione di sistema.

1.2 Organizzazione del sistema di calcolo

Un sistema di calcolo è composto da una o più CPU e diversi controllori di device, questi sono connessi ad un bus comune che permette l'accesso ad una memoria condivisa, in questo modo riescono a comunicare. I controllori di device competono per far sì che i loro processi vengano eseguiti, questo è necessario in quanto le CPU costituiscono un sistema concorrente, cioè un sistema in cui solo un certo numero di processi può essere eseguito in parallelo.

1.2.1 Dispositivi I/O

Quando un controllore di device porta a termine un'operazione di I/O causa un segnale di **interrupt** che viene inviato alla CPU. In questo modo, il dispositivo, comunica alla CPU che ha terminato le sue operazioni. Al verificarsi di un interrupt il controllo viene trasferito all'opportuna routine di gestione utilizzando il vettore degli interrupt (contenuto nella memoria) che contiene tutti gli indirizzi delle routine.

Esistono diverse tipologie di dispositivi I/O, in particolare: I/O programmato; I/O ad interrupt; DMA. I più sofisticati sono i DMA, si tratta di dispositivi I/O ad alte prestazioni. In questi dispositivi il controllore di device trasferisce blocchi di dati direttamente nella memoria principale. Una volta terminato questo processo invia un segnale di interrupt alla CPU. La differenza dei DMA, gli I/O ad interrupt, generano un interrupt per ogni byte trasferito in memoria principale. Esempi di dispositivi hardware gestiti tramite DMA sono: controller di unità a disco; schede grafiche, di rete, audio.

1.2.2 Multiprocessori e multicore

Abbiamo detto che i moderni computer costituiscono un sistema multiprocessore, questo tipo di sistema presenta dei vantaggi rispetto ai sistemi a singolo processore:

- permettono la multielaborazione asimmetrica: ogni processore svolge un compito specifico, un processore principale sovrintende all'intero sistema;
- permettono la multielaborazione simmetrica: ogni processore è abilitato all'esecuzione di tutte le operazioni del sistema.

Un trend del design delle CPU moderne è quello di includere molteplici **core** all'interno di un singolo chip. Questi sistemi sono noti con il nome di **multicore**. Si fruttava questa tecnica per migliorare l'efficienza della CPU, infatti, la comunicazione on-chip è più veloce della comunicazione between-chip, inoltre richiede molta meno potenza. Nota che in generale, ogni sistema multicore è un sistema multiprocessore, mentre non è detto che un sistema multiprocessore sia un sistema multicore.

1.2.3 Cluster

Oltre ai sistemi multiprocessori esistono i sistemi **cluster**. Questo tipo di sistemi sono simili ai sistemi multiprocessori, si differenziano dal fatto che sono composti da due o più calcolatori completi collegati tra loro (detti nodi). I nodi, normalmente, vengono collegati ad una memoria secondaria condivisa tramite network.

Esistono due tipologie di clustering: **asimmetrico**, un calcolatore si trova in modalità stand-by e si attiva solo nel caso di malfunzionamenti, gli altri calcolatori eseguono le applicazioni; **simmetrico**, in questo caso tutti i calcolatori eseguono le applicazioni e si controllano reciprocamente.

Questo tipo di sistemi hanno prestazioni migliori rispetto ad un sistema composto da un singolo processore in quanto le applicazioni possono girare in maniera concorrente in tutti i calcolatori del cluster. Le applicazioni devono tuttavia essere sviluppate in maniera tale da sfruttare al massimo questa architettura.

1.3 Struttura del sistema operativo

In un sistema operativo girano contemporaneamente molteplici programmi. Per sfruttare al massimo la CPU, bisogna far sì che non ci siano tempi morti, cioè bisogna far sì che abbia

sempre un **job** (codice e dati) da eseguire. Questo fondamentale comportamento si ottiene con la **multiprogrammazione**.

Con la multiprogrammazione più job sono memorizzati contemporaneamente nella memoria principale, in questo modo la CPU è sempre occupata. La scelta del job che verrà mandato in esecuzione (utilizzo esclusivo CPU), viene affidata ad un algoritmo di CPU scheduling. Quando il job entra in una fase di attesa, il sistema operativo seleziona un nuovo job da eseguire. La CPU riesce a gestire molteplici job, la multiprogrammazione avviene in maniera talmente rapida che l'utente può interagire con vari programmi mentre questi stanno girando.

1.4 Operating-System Operations

Come abbiamo anticipato, i moderni sistemi operativi sono interrupt driven. Se non ci sono processi da eseguire, non ci sono dispositivi I/O da gestire, e nessun utente da gestire, il sistema operativo rimarrà in attesa di un evento da gestire. Gli eventi vengono segnalati tramite interrupt o trap (errori o eccezioni), questi vengono gestiti tramite apposite routine.

Siccome sistema operativo e utente condividono le risorse hardware e software, bisogna fare in modo che un errore in un programma dell'utente venga isolato, non deve poter intaccare altri programmi e processi che girano sul sistema. Un sistema operativo progettato correttamente deve far in modo che un programma che contiene errori non causi la scorretta esecuzione di altri programmi.

1.4.1 Dual-mode e multimode operation

Per garantire la stabilità del sistema operativo, distinguiamo due modalità di esecuzione: **user mode**; **kernel mode**.

La user mode è caratterizzata da un basso livello di privilegi verso la memoria e hardware. La kernel mode è lo stato di privilegio massimo riservato all'esecuzione del kernel, in questa modalità si ottiene un accesso illimitato verso memoria e hardware.

A livello hardware è presente un **mode bit** che indica la modalità corrente: cambiando il valore del bit si passa da una modalità all'altra. Dopo ogni **system call** si passa alla kernel mode, il ritorno alla chiamata riporta il sistema alla user mode.

1.5 Gestione processi

Un processo è un programma in esecuzione. Un programma è un'entità passiva (es: contenuto di un file salvato su disco) a differenza di un processo che è un'entità attiva. Un programma diventa un processo quando viene caricato nella memoria principale. Il processo rappresenta l'unità di lavoro del sistema di calcolo, necessita di alcune risorse (tempo CPU, memoria, etc) per poter portare a termine il proprio lavoro. Una volta che il processo termina, vengono recuperate tutte le risorse allocate precedentemente.

Introduciamo i **thread**. Il thread è un sottoprocesso, diversi thread di uno stesso processo condividono alcune risorse (spazio di indirizzamento del processo). Per chiarire le idee: per definizione, un processo, possiede delle risorse e deve essere eseguito. Questi due aspetti vengono gestiti in maniera indipendente dal sistema operativo, in particolare: il thread è l'elemento che viene eseguito; il processo è l'elemento che possiede le risorse. Distinguiamo i processi in base al numero di thread:

- single thread: il processo ha un'unico program counter;

- multi thread: il processo ha più program counter, uno per ogni thread, ogni thread condivide gli stessi dati.

Il sistema operativo è responsabile delle seguenti attività relative alla gestione dei processi:

- creazione di processi utente e di sistema;
- sospensione e riattivazione processi;
- fornire meccanismi per la sincronizzazione di processi;
- fornire meccanismi per la comunicazione fra processi e la gestione dei **deadlock** (evitare che processi si blocchino in attesa di risorse).

Mentre un processo viene eseguito è soggetto a transazioni di stato, questi stati rappresentano il suo "ciclo di vita". Gli stati sono:

1. new: il processo viene creato;
2. running: il processo è in esecuzione;
3. waiting: il processo è in attesa di un evento. Un processo finisce in questa fase quando è in attesa di ricevere un dato. In questo caso la CPU si dedica all'esecuzione di un altro processo;
4. ready: il processo è in attesa di essere riassegnato ad un processore. Un processo finisce in questa fase in due casi: arriva da waiting; arriva da running, ciò avviene se un processo con priorità maggiore si trova in waiting (*prelazione*);
5. terminated: il processo ha terminato la propria esecuzione. Questo stato viene raggiunto anche a causa di errori nel codice: eccezioni; uso scorretto risorse (accesso non consentito in memoria, superamento tempo massimo utilizzo CPU).

Tutte le informazioni che riguardano un processo vengono salvate in una struttura dati chiamata *Process Control Block (PCB)*, questa viene aggiornata di volta in volta con le informazioni del processo attualmente in corso.

1.6 Gestione memoria centrale

La memoria centrale è costituita da un enorme array di byte ognuno dei quali ha il proprio indirizzo. Si tratta sostanzialmente di un deposito di dati ad accesso rapido condiviso tra CPU e dispositivi I/O. Il sistema operativo è responsabile delle seguenti attività relative alla gestione della memoria:

- tener traccia di quali parti della memoria sono attualmente usate e da chi;
- decidere quali (parti di) processi caricare in memoria quando c'è spazio disponibile;
- allocare e deallocare lo spazio di memoria secondo necessità del sistema stesso.

Un buon sistema di calcolo deve avere anche una buona memoria secondaria. Il sistema deve consentire l'archiviazione secondaria per salvare i contenuti della memoria centrale. Ciò è necessario in quanto la memoria centrale è volatile e troppo piccola per contenere permanentemente tutti i dati. La gestione efficiente della memoria secondaria è fondamentale per le prestazioni del sistema. Il sistema operativo è responsabile delle seguenti attività relative alla gestione della memoria secondaria:

- gestione spazio libero;
- allocazione spazio;
- determina chi accede al disco (shceduling disco).

Capitolo 2

Struttura dei sistemi operativi

2.1 I servizi

Un sistema operativo fornisce un ambiente in cui si possono eseguire programmi. I servizi che vengono forniti dal sistema operativo servono per semplificare la programmazione degli applicativi che si devono interfacciare con esso. Una parte di questi servizi fornisce funzionalità che sono utili all'utente:

- *interfaccia utente*: *command line interface (CLI)* e/o *graphic user interface (GUI)*;
- *esecuzione programmi*: caricare programmi in memoria ed eseguirli;
- *operazioni I/O*: il SO fornisce ai programmi utente i mezzi per effettuare l'I/O su file o periferica;
- *gestione file system*: capacità dei programmi di creare, leggere, scrivere e cancellare file e muoversi nella struttura delle directory;
- *comunicazioni*: scambio di informazioni fra processi in esecuzione sullo stesso elaboratore o su sistemi indipendenti connessi via rete. Le comunicazioni possono avvenire utilizzando memoria condivisa o con scambio di messaggi;
- *rilevamento errori*: il SO rileva eventuali errori che si possono verificare nella CPU e nella memoria, nei dispositivi I/O o durante l'esecuzione di programmi utente. In caso di errore il SO deve garantire una modalità operativa corretta e consistente.

Esiste un altro set di servizi utili a garantire l'efficienza del sistema. Questi servizi non sono esplicitamente orientati all'utente. Troviamo:

- *allocazione risorse*: quando più utenti o più processi vengono serviti in concorrenza, le risorse devono essere allocate equamente ad ognuno di essi;
- *accounting e contabilizzazione uso risorse*: per ottimizzare le prestazioni del sistema di calcolo, il SO, tiene traccia di quali utenti usano quante e quali risorse;
- *protezione e sicurezza*: tutti gli accessi alle risorse di sistema sono controllati. In questo modo, nei sistemi multiutente o distribuiti, si proteggono i possessori di informazioni da accessi indesiderati ai propri dati. La sicurezza si basa sull'obbligo di identificazione tramite *password* e si estende alla difesa dei dispositivi di I/O esterni (es: modem, adattatori di rete, etc).

2.2 Interfaccia utente e CLI

Per interfacciarsi con il SO esistono due modalità: attraverso **CLI**; attraverso la **GUI**.

L'interfaccia utente a riga di comando permette di impartire direttamente comandi al SO (detti istruzioni di controllo). Solitamente viene implementata attraverso programmi di sistema e può essere personalizzabile, cioè il SO può offrire più *shell* (diversi ambienti da cui l'utente può impartire le proprie istruzioni).

La sua funzione principale è quella di interpretare ed eseguire le istruzioni di comando (sia built-in del SO che nomi di eseguibili utente), dunque è presente un **interprete dei comandi**. I comandi ricevuti da quest'ultimo possono essere eseguiti secondo due modalità:

1. se il codice relativo al comando è parte del codice dell'interprete, viene effettuato un salto all'opportuna sezione di codice. In questo caso, il numero di comandi implementati determina le dimensioni dell'interprete;
2. i comandi vengono implementati per mezzo di programmi di sistema. Questo approccio (usato dai sistemi UNIX) permette ai programmatori di aggiungere nuovi comandi al sistema creando nuovi file (memorizzati in directory opportune), inoltre l'interprete non viene modificato, può avere dimensioni ridotte. Un esempio di comando può essere `rm file.txt`. In questo caso l'interprete non comprende il comando, invece va a cercare il file nominato "`rm`" (contenente il codice), lo carica in memoria e lo esegue passandogli come parametro "`file.txt`".

La più comune *GUI* offre un'esperienza più user-friendly. Permette di interagire con il SO attraverso: un'interazione semplice tramite mouse, tastiera, monitor; icone che rappresentano file, directory, programmi, etc. Cliccare con il mouse su oggetti differenti permette l'esecuzione di diversi tipi di azione, che a loro volta corrispondono a dei comandi che vengono impartiti all'interprete dei comandi.

2.3 System Calls

Le **system call** forniscono l'interfaccia fra processi e i servizi offerti dal SO. Generalmente sono realizzate utilizzando linguaggi di alto livello (C o C++) e vengono richiamate dagli applicativi attraverso *Application Programming Interface (API)* piuttosto che per invocazione diretta. Le API in sostanza forniscono le funzioni e la loro documentazione, ciò facilita il lavoro dei programmatori. Alcuni esempi di API sono: Win64 API per Windows; POSIX API per i sistemi POSIX-based (tutte le versioni di UNIX, LINUX e Mac OS); Java API per la JVM.

Normalmente a ciascuna *system call* è associato un numero. La chiamata di una system call viene gestita attraverso un'*interfaccia alle chiamate di sistema*. Questa mantiene una tabella indicizzata dal numero di system call, effettua le chiamate e ritorna lo stato del sistema dopo l'esecuzione (eventualmente anche valori restituiti).

L'utente non deve conoscere i dettagli implementativi delle system call, deve conoscere la modalità di utilizzo delle API. Per esempio per Linux, la libreria standard di C fornisce una parte della API. Nella figura 2.3: il programma C invoca la funzione `printf()`; la libreria C intercetta la funzione e invoca la system call `write()`; infine la libreria riceve il valore restituito dalla chiamata al sistema e lo passa al programma scritto dall'utente.

Spesso capita che la system call necessiti ulteriori informazioni, quindi ulteriori parametri. Il **passaggio di parametri** può essere effettuato attraverso tre modalità:

1. passaggio di parametri nei registri;

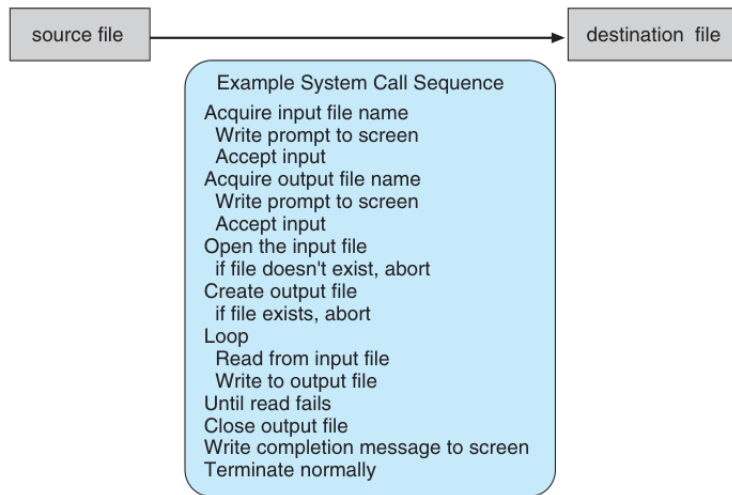


Figura 2.1: Sequenza di System call per realizzare la copia di un file in un altro

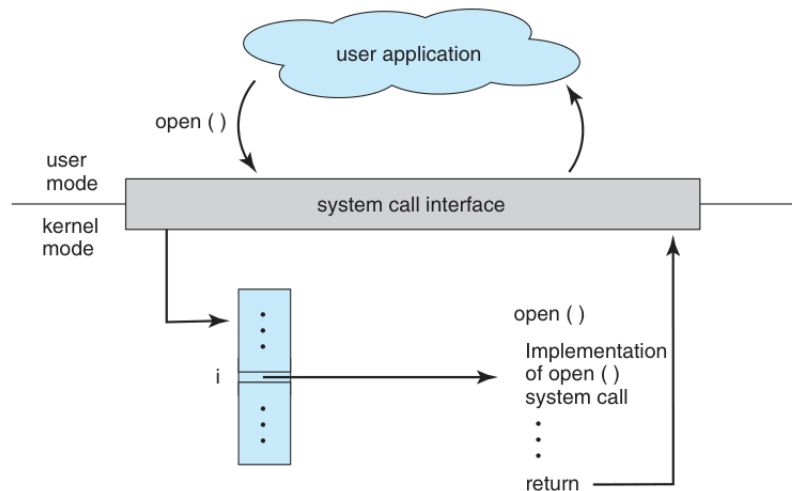


Figura 2.2: Gestione della system call `open()` invocata da un'applicazione

- memorizzazione dei parametri in un blocco di memoria e passaggio dell'indirizzo del blocco come parametro in un registro: necessario se i parametri sono in numero maggiore rispetto ai registri (approccio seguito da Linux per > 5 parametri).
- i parametri vengono memorizzati nello *stack* tramite *push* del programma e recuperati dal SO tramite *pop*.

Gli ultimi due metodi non hanno vincoli sul numero e lunghezza dei parametri da passare.

2.3.1 Tipi di system call

Possono essere raggruppate in sei principali categorie:

- creazione e terminazione di un processo (`fork`, `exit`);
- caricamento ed esecuzione (`exec`/`execve`);

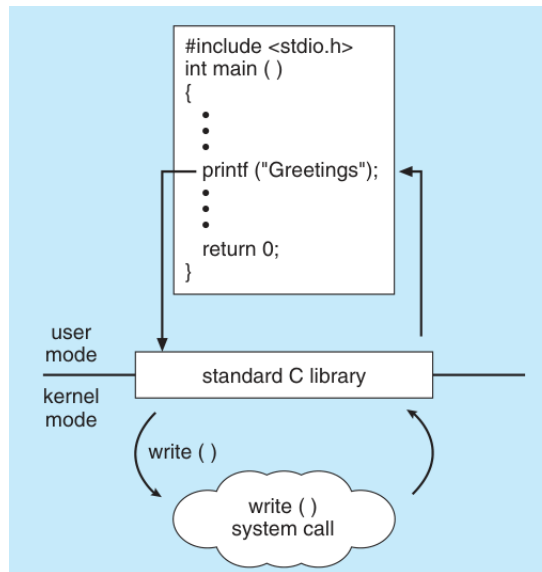


Figura 2.3: Esempio di system call invocato dalla libreria standard di C

3. lettura e modifica degli attributi di un processo (priorità, tempo massimo di esecuzione) (get/set process attributes);
4. attesa per il tempo indicato o fino alla segnalazione di un evento (wait/waitpid);
5. assegnazione e rilascio memoria (alloc, free)
6. invio di segnali (signal, kill).

EXAMPLES OF WINDOWS AND UNIX SYSTEM CALLS		
	Windows	Unix
Process Control	CreateProcess() ExitProcess() WaitForSingleObject()	fork() exit() wait()
File Manipulation	CreateFile() ReadFile() WriteFile() CloseHandle()	open() read() write() close()
Device Manipulation	SetConsoleMode() ReadConsole() WriteConsole()	ioctl() read() write()
Information Maintenance	GetCurrentProcessID() SetTimer() Sleep()	getpid() alarm() sleep()
Communication	CreatePipe() CreateFileMapping() MapViewOfFile()	pipe() shm_open() mmap()
Protection	SetFileSecurity() InitializeSecurityDescriptor() SetSecurityDescriptorGroup()	chmod() umask() chown()

Figura 2.4: Esempi di system call

2.4 Programmi di sistema

I programmi di sistema forniscono un ambiente convenzionale per lo sviluppo e l'esecuzione di programmi utente (semplici interfacce alla system call o programmi complessi). Esistono diversi programmi di sistema, ognuno svolge diversi compiti specifici. Per esempio: gestione file (es: `cp`); informazioni di stato (es: `du`, `who`); editing file (es: `nano`, `vi`); supporto ai linguaggi di programmazione (es: `gcc`); caricamento ed esecuzione di programmi; comunicazioni; supporto alla realizzazione di applicativi. L'aspetto del sistema operativo è definito dai programmi di sistema e non dalle system call vere e proprie.

Due importanti categorie di programmi di sistema sono:

- *servizi background*: vengono lanciati durante la fase di boot, alcuni sono utili durante la fase di boot, altri rimangono attivi dal boot fino allo shutdown;
- *programmi applicativi*: questi non fanno parte del sistema operativo, vengono eseguiti dagli utenti tramite linea di comando, mouse o pressione del touch screen.

2.4.1 Linker e loader

I moderni sistemi general purpose non collegano direttamente le librerie ai file eseguibili, utilizzano piuttosto le librerie dinamiche. In questo modo, le librerie, vengono caricate in memoria dinamicamente secondo le necessità dei vari programmi, ciò evita sprechi di spazio.

I file oggetto ed eseguibili hanno formati standard per far sì che il sistema operativo sappia come caricarli e avviarli. Nei sistemi UNIX-like il formato standard è l'*ELF (Executable and Linkable Format)*, mentre nei sistemi Windows il formato è *PE (Portable Executable)*.

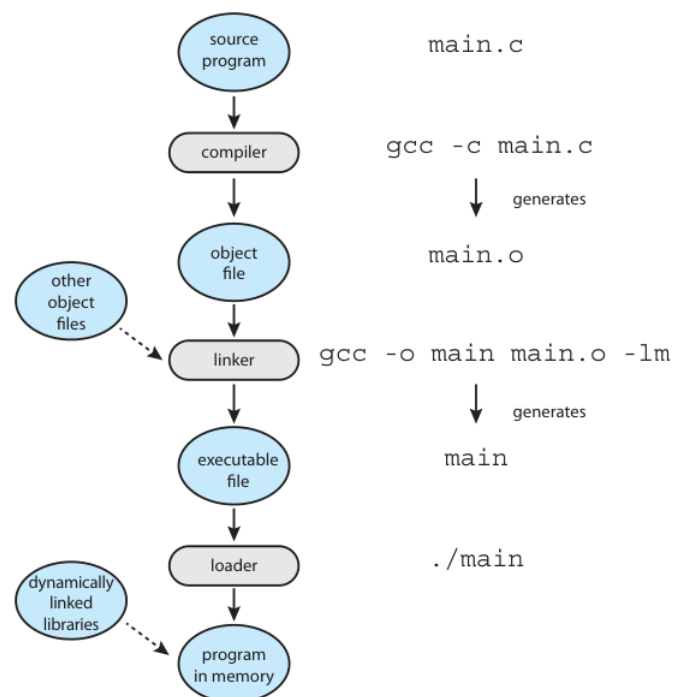


Figura 2.5: Ruolo del linker e del loader

2.5 Struttura sistema operativo

Storicamente i sistemi operativi erano per lo più **monolitici**, ciò significa che le funzioni di gestione delle risorse sono tutte realizzate nel kernel, l'intero SO tende ad identificarsi con il kernel. In questo modo, poiché tutti i moduli (file oggetto) operano nello stesso spazio di indirizzi, un bug in uno di essi può bloccare l'intero sistema. Il vantaggio d'altro canto è che, quando l'implementazione è sicura, la stretta integrazione interna dei componenti, rende un **kernel monolitico** estremamente efficiente. Attualmente nei sistemi è più comune la suddivisione in piccole componenti, quindi più moduli con più interfacce e funzioni definite in fase di progettazione.

Ci sono principalmente quattro maniere differenti in cui il SO potrebbe essere organizzato:

- approccio stratificato;
- microkernel;
- kernel modulare;
- kernel ibrido.

Analizziamoli uno per uno.

2.5.1 Approccio stratificato

Il sistema operativo viene suddiviso in un certo numero di strati (o livelli), ciascuno costruito sopra gli strati inferiori. Al livello più basso (livello 0) si trova l'hardware, questo fornisce operazioni di base piuttosto semplici. Man mano che si sale con le stratificazioni, vengono offerte sempre più funzionalità che però dipendono dalle funzionalità offerte dagli strati inferiori. All'ultimo livello (livello n) si trova l'interfaccia utente.

Dato che ogni strato può utilizzare funzioni e servizi di strati di livello inferiore, il codice risulta semplice da debuggare, infatti: se so che gli strati inferiori sono corretti, basta cercare in quelli superiori. Mentre lo svantaggio di questo approccio sta nel fatto che per passare da uno strato all'altro, vengono invocate molte funzioni, ciò comporta a molti record di attivazione attivi, di conseguenza le prestazioni peggiorano.

Questo metodo viene utilizzato per esempio per le macchine virtuali.

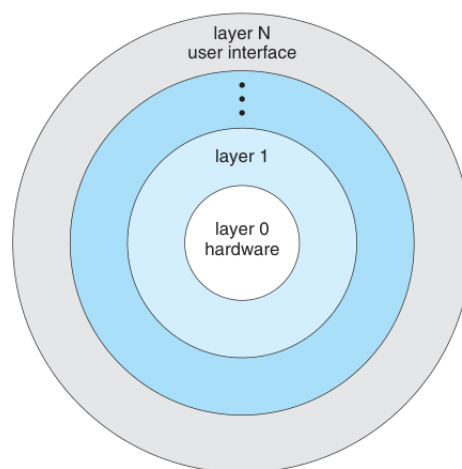


Figura 2.6: Layered operating system

2.5.2 Microkernel

Le funzionalità del kernel sono tutte spostate nello spazio utente. Il microkernel offre soltanto servizi minimi di gestione di processi, memoria e di comunicazione. Il suo scopo principale è quello di fornire funzioni di comunicazione tra programmi client e servizi (implementati esternamente), tali comunicazioni hanno luogo tra moduli utente mediante scambio di messaggi (mediati dal kernel).

Il vantaggio sta nel fatto che, essendo modulare, le funzionalità del sistema sono più semplici da estendere: si possono aggiungere altri moduli senza dover ricompilare il kernel ogni volta. Lo svantaggio è che l'overhead di comunicazione tra spazio utente e spazio kernel può causare un decadimento delle prestazioni.

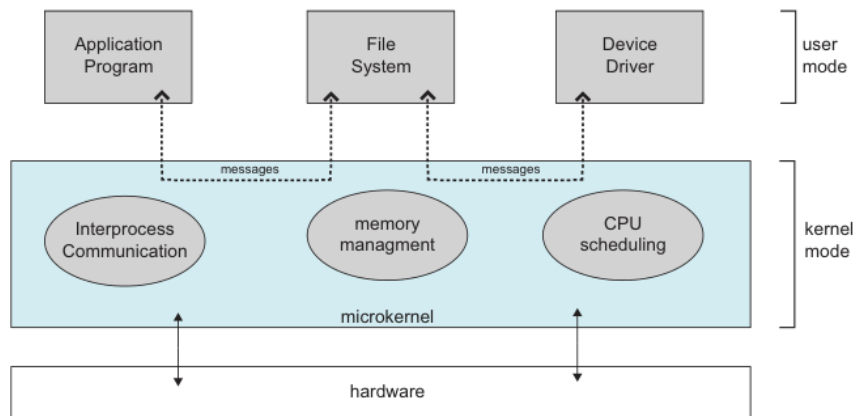


Figura 2.7: Architettura microkernel

2.5.3 Kernel modulari

L'architettura a moduli è molto simile all'architettura a strati, la differenza sta nel fatto che i moduli hanno la possibilità di comunicare direttamente tra di loro. Ciascun modulo può essere caricato o meno in memoria come parte del kernel; i moduli possono essere caricati dinamicamente all'avvio o a run-time.

```

1 // ### Esempio di modulo che possiamo caricare dinamicamente ###
2 #include <linux/init.h>
3 #include <linux/kernel.h>
4 #include <linux/module.h>
5
6 /* This function is called when the module is loaded. */
7 int simple_init(void) {
8     printk(KERN_INFO "Loading Kernel Module\n");
9     return 0;
10 }
11
12 /* This function is called when the module is removed. */
13 void simple_exit(void) {
14     printk(KERN_INFO "Removing Kernel Module\n");
15 }
16
17 /* Macros for registering module entry/exit points. */
18 module_init(simple_init);

```

```

19  module_exit(simple_exit);
20
21  MODULE_LICENSE("GPL");
22  MODULE_DESCRIPTION("Simple Module");
23  MODULE_AUTHOR("SGG");

```

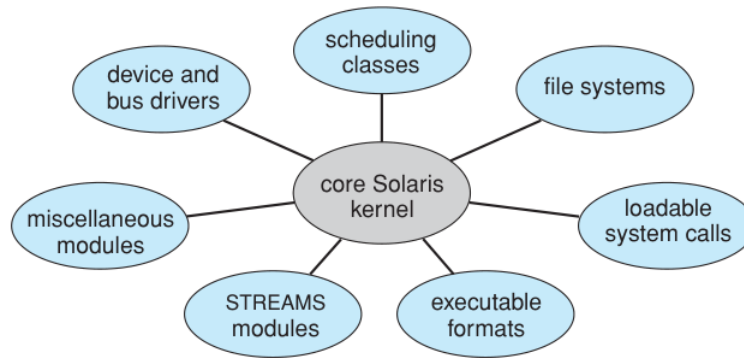


Figura 2.8: Moduli dinamici Solaris

2.5.4 Kernel ibridi

La maggior parte dei SO attuali non adotta un modello "puro", si sfruttano modelli ibridi. I modelli ibridi combinano diversi approcci implementativi allo scopo di migliorare le performance.

Un esempio è il kernel Linux e Solaris: sono fondamentalmente monolitici in modo da avere le prestazioni migliori; sono anche modulari, c'è quindi la possibilità di aggiungere dinamicamente nuove funzionalità al kernel. Windows invece è sempre modulare ma conserva alcuni comportamenti tipici dei sistemi microkernel.

2.6 Debugging di un SO

Il **debugging** è l'attività di individuazione e risoluzione di errori nel sistema, comprende anche il **performance tuning**. Gli errori di sistema rilevati dal sistema operativo possono essere generati da bug nei processi o bug nel kernel:

- **core dump:** istantanea post-mortem dello stato del processo al momento della terminazione anomala;
- **crash dump:** un guasto nel kernel viene chiamato crash. L'informazione riguardante l'errore viene salvata in un file di log, lo stato della memoria viene salvato in un'immagine su memoria di massa dedicata.

Con *performance tuning* si intende l'insieme delle tecniche atte ad ottimizzare le prestazioni del sistema, eliminando i colli di bottiglia. Anche questo tipo di problemi che condizionano le prestazioni vengono considerati bachi. Per effettuare un debugging possiamo sfruttare software dedicati che effettuano misurazioni sul sistema oppure strumenti interattivi (es: `top` in Linux).

Capitolo 3

Thread e concorrenza

3.1 Thread

Nella sezione 1.5 abbiamo introdotto i thread. I thread costituiscono l'unità base dell'uso della CPU, sono composti da: un program counter; un insieme di registri; uno stack. Abbiamo detto che i thread di uno stesso processo condividono la sezione del codice, la sezione dei dati e le altre risorse allocate dal processo originale (es: file aperti).

Il vantaggio dei thread sta nel fatto che: se un processo è composto da più thread (**multithread**) allora è in grado di eseguire più compiti in maniera concorrente. In questo modo si incrementa l'efficienza del programma in esecuzione.

La programmazione multithread può produrre codice più semplice e più efficiente, per questo motivo i kernel attuali sono normalmente multithread. Inoltre il multithreading permette di sfruttare al meglio le architetture **multicore**.

I programmi vengono divisi in più thread e non in più processi perché i processi richiedono più spazio, mentre i thread sono più leggeri dato che condividono diverse risorse.

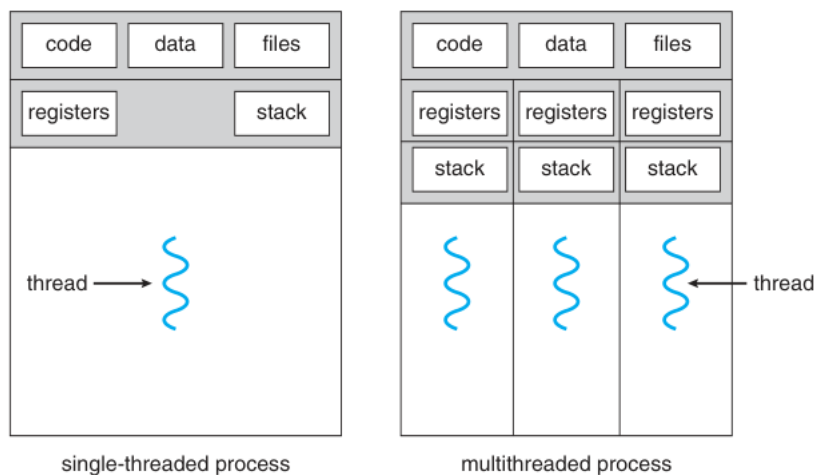


Figura 3.1: Processi single-threaded e multithreaded

3.1.1 Gestione thread

La prima implementazione di thread prevedeva che essi fossero visibili solamente a livello utente, il sistema operativo era in grado di trattare solo processi. Questa soluzione presentava un problema: un singolo thread in waiting blocca l'intero processo. Per risolvere il problema sono stati inventati i **thread kernel**.

Ci sono tre modi principali in cui i thread possono rapportarsi con il sistema operativo:

- *one-to-one*: ogni thread utente viene mappato ad un kernel thread. Si tratta della soluzione adottata da Linux e Windows;
- *many-to-many*: più thread utente mappati a più kernel thread (in numero inferiore);
- *two-level mode*: l'unione delle due precedenti.

3.2 Programmazione multicore

Esiste una distinzione netta tra **concorrenza** e **parallelismo**.

- un sistema è parallelo se può eseguire simultaneamente più task;
- un sistema concorrente supporta più task consentendo a tutti di progredire nell'esecuzione.

In generale la concorrenza non necessita del parallelismo. I vecchi computer erano composti da CPU single core, attraverso la concorrenza si aveva l'illusione che i programmi in esecuzione fossero eseguiti in parallelo. Questo effetto è dovuto alla velocità con cui lo scheduler della CPU riesce a portare in esecuzione diversi processi, consentendo ad ognuno di progredire.

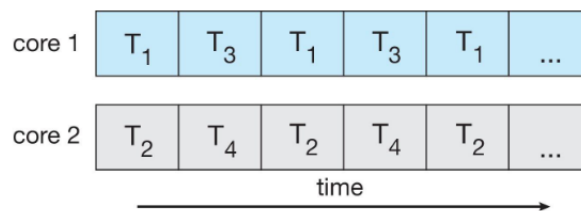


Figura 3.2: Differenza tra concorrenza e parallelismo

Esistono due tipi di parallelismo:

- **data parallelism**: la stessa operazione viene eseguita contemporaneamente (in parallelo) sull'insieme di dati. L'insieme originale dei dati viene suddiviso in partizioni in modo che più thread possano agire simultaneamente;
- **task parallelism**: si tratta di una forma di parallelizzazione del codice tra più processori in ambienti di calcolo parallelo. Il programma viene diviso in più thread, ognuno dei quali svolge operazioni differenti.

Progetti hardware recenti (**hyperthreading**) implementano unità di calcolo multithread in cui due o più thread hardware sono assegnati ad una singola CPU.

3.3 Risorse

La risorsa è un'astrazione per rappresentare un'entità necessaria ad un processo per svolgere il proprio lavoro. Se un processo necessita di una risorsa non disponibile, allora dovrà attendere per utilizzarla. Il termine risorsa è generico, può essere sia output di un altro processo che spazio di memoria. In generale, possiamo dividere le risorse in due tipi:

- *condivisibili*: possono essere usate da più processi in parallelo (es: più processi accedono alla RAM);

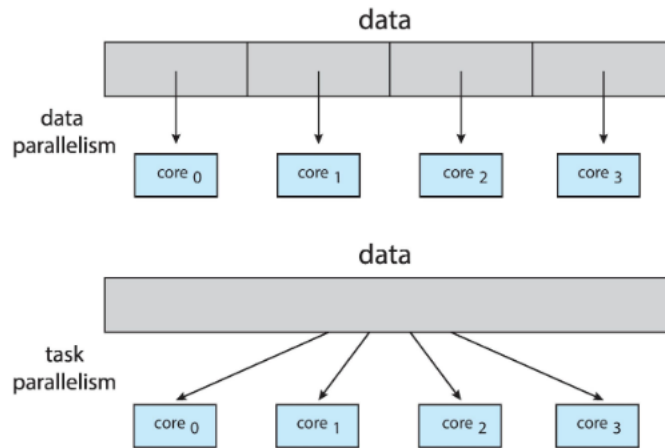


Figura 3.3: Differenza tra data e task parallelism

- *non condivisibili*: non possono essere utilizzate da più processi contemporaneamente (es: la stampante). Se più processi necessitano contemporaneamente di una risorsa non condivisibile, allora questi si bloccano a vicenda (**stallo**). In questi casi la risorsa viene assegnata a uno dei processi, gli altri rimangono in attesa.

Vengono ulteriormente suddivise in:

- *consumabili*: non riutilizzabili, si tratta di risorse esterne al sistema (es: corrente);
- *non consumabili*: possono essere riutilizzate (es: core di una CPU).

Alcune risorse sono definite *sottraibili*: risorsa che può essere rimossa forzatamente da un processo. Un esempio: la CPU viene sottratta da un processo con priorità bassa se un processo ad alta priorità necessita di potenza di calcolo.

3.3.1 Deadlock, livelock e grafo delle risorse

Un **deadlock** o stallo si verifica in un ambiente multiprogrammato quando più thread entrano in competizione per ottenere un numero finito di risorse. Quando si verifica una tal situazione, è necessario eliminare i processi bloccati e a loro volta i processi che dipendono dai processi bloccati, e così via.

Precisiamo che i SO come Windows o Linux non prevedono la gestione di queste situazioni in quanto richiederebbero operazioni onerose, intervengono direttamente con un riavvio forzato.

Il protocollo di accesso alle risorse da parte di un thread prevede tre fasi: *richiesta*, se la risorsa è occupata il thread viene messo in attesa fino al rilascio; *utilizzo*; *rilascio*.

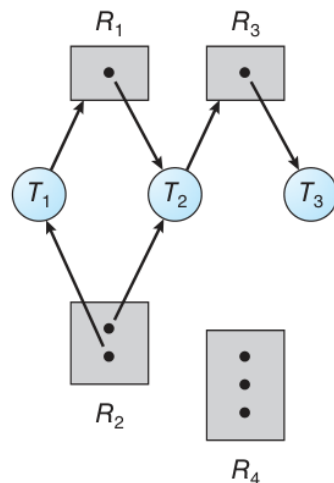
Elenchiamo le condizioni necessarie (ma non sufficienti) affinché si verifichi uno stallo:

1. *mutua esclusione*: presenza di risorse che possono essere utilizzate da un solo processo alla volta;
2. *allocazione parziale*: un processo richiede diverse risorse durante la sua evoluzione (non tutte in una singola operazione);
3. *presenza di risorse non sottraibili*: il processo detiene il possesso della risorsa finché non conclude (in Linux ogni processo ha un grado di *gentilezza*);

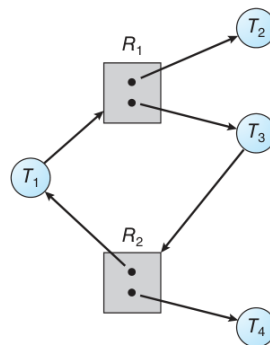
4. *attesa circolare*: ci sono n processi dove P_1 attende P_2 che a sua volta attende P_3 , ..., P_{n-1} attende P_n . Tutti aspettano una risorsa che non verrà mai rilasciata, nessuno lavora.

Il **livelock** o stallo attivo è una situazione molto simile al deadlock, la differenza sostanziale è che i thread non sono effettivamente bloccati, ma di fatto non progrediscono. In altre parole, in tale situazione si ha che un thread tenta continuamente un'azione (*polling*, es: tentare di acquisire una risorsa) che non ha successo, non sono quindi in deadlock in quanto continuano a tentare di portare a termine il loro compito.

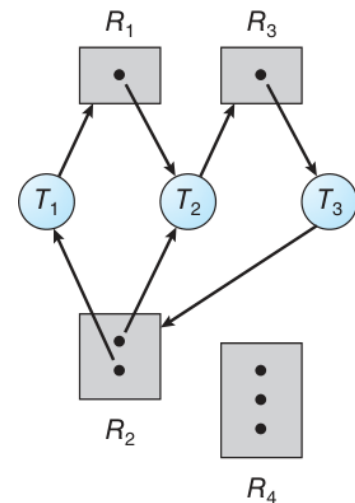
Il **grafo delle risorse** è utile per rappresentare le richieste e le assegnazioni di risorse ai vari processi. Definiamo il grafo $G(V, E)$ come segue dove V è l'insieme dei vertici ed E è l'insieme degli archi. V è partizionato in due sottoinsiemi: $T = \{T_1, \dots, T_n\}$ che comprende tutti i thread attivi nel sistema; $R = \{R_1, \dots, R_m\}$ che comprende tutti i tipi di risorse presenti nel sistema. Distinguiamo inoltre due tipi di archi: l'*arco di richiesta* $T_i \rightarrow R_j$; l'*arco di assegnazione* $R_j \rightarrow T_i$; l'*arco di reclamo* $T_i \dashrightarrow R_j$, indica che il thread può richiedere tale risorsa in futuro. Un arco di reclamo viene convertito in arco di richiesta quando il thread richiede effettivamente la risorsa, mentre un arco di richiesta viene convertito ad arco di assegnazione quando la risorsa viene allocata al thread.



(a) Grafo di assegnazione senza deadlock



(b) Grafo ciclico senza deadlock



(c) Grafo ciclico con deadlock

Nella figura 3.4a: $E = \{T_1 \rightarrow R_1, T_2 \rightarrow R_3, R_1 \rightarrow T_2, R_2 \rightarrow T_2, R_2 \rightarrow T_1, R_3 \rightarrow T_3\}$. Si nota che non ci sono deadlock, quando il processo T_3 termina, la risorsa R_3 viene assegnata a T_2 e così via. Nessun thread rimane bloccato.

Nella figura 3.4c, è presente una situazione di attesa circolare: T_3 necessita R_2 che però è già assegnata a T_2 , il quale necessita R_3 che è assegnata a T_3 . Sono tutti in attesa ma nessuno progredisce. La domanda è: guardando il grafo, come si deduce la presenza di deadlock? Si nota che:

- se il grafo è aciclico $\Rightarrow$ non ci sono deadlock;
- se il grafo contiene almeno un ciclo:
 - se vi è una sola istanza per ogni tipo di risorsa $\Rightarrow$ si ha un deadlock;
 - se si hanno più istanze per tipo di risorsa $\Rightarrow$ il deadlock è possibile ma non certo.

3.3.2 Individuare un deadlock

Nel paragrafo precedente abbiamo definito il grafo delle risorse (anche detto grafo di Hall). Un sistema non è in stallo se e solo se tale grafo è completamente riducibile, cioè se esiste una sequenza di passi di riduzione che elimina tutti gli archi del grafo. Una riduzione è possibile se un processo può vedersi assegnate tutte le risorse di cui necessita in modo tale da poter soddisfare la sua richiesta.

Analizziamo direttamente un esempio: abbiamo 3 tipi di risorse R_1, R_2, R_3 e 4 processi P_1, P_2, P_3, P_4 .

	R_1	R_2	R_3
D	3	3	1
P_1	2	1	0
P_2	2	1	1
P_3	0	2	1
P_4	1	1	1

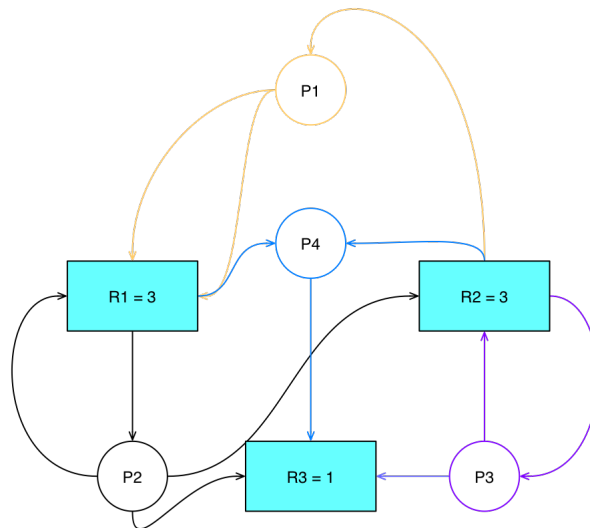
(a) Dichiarate

	R_1	R_2	R_3
D	2	3	0
P_1	0	1	0
P_2	1	0	0
P_3	0	1	0
P_4	1	1	0

(b) Allocate

	R_1	R_2	R_3
D	1	0	1
P_1	2	0	0
P_2	1	1	1
P_3	0	1	1
P_4	0	0	1

(c) Richieste



Nella prima tabella vengono indicate tutte le risorse che i processi dichiarano di utilizzare, la riga D indica le risorse totali disponibili inizialmente. Nella seconda tabella vengono indicate tutte le risorse che sono state assegnate ad un processo, la riga D indica il numero di archi di assegnazione di ciascuna risorsa. Nell'ultima tabella vengono indicate le risorse richieste dai processi, la riga D abbiamo sottratto le risorse totali disponibili con quelle totali allocate.

Una riga P_i della tabella C può essere ridotta se vettorialmente è $\leq$ della riga D della tabella C, in quanto le richieste del processo possono essere soddisfatte. Se tale riduzione è possibile, possiamo eliminare il processo dalla tabella e incrementare D con le risorse che aveva allocato il processo nella tabella B.

La riduzione è completa quando tutte le righe della tabella C sono eliminate, se ciò avviene allora il sistema non è in stallo.

Quindi facendo riferimento alle tabelle:

- P_4 (tabella C) è vettorialmente inferiore a D, quindi può terminare, di conseguenza incremento D che diventa $\begin{pmatrix} 1 & 0 & 1 \end{pmatrix} + \begin{pmatrix} 1 & 1 & 0 \end{pmatrix} = \begin{pmatrix} 2 & 1 & 1 \end{pmatrix}$;

- ora P_1 può terminare, quindi $D = \begin{pmatrix} 2 & 2 & 1 \end{pmatrix}$;
- ora P_2 può terminare, quindi $D = \begin{pmatrix} 3 & 2 & 1 \end{pmatrix}$;
- ora P_3 può terminare, quindi $D = \begin{pmatrix} 3 & 3 & 1 \end{pmatrix}$.

3.3.3 Algoritmo del banchiere

L'algoritmo del banchiere è un algoritmo utilizzato per evitare deadlock nell'allocazione delle risorse. In particolare questo algoritmo può indicare se un sistema si ritroverebbe in uno stato sicuro o meno nel caso assegnasse una risorsa ad uno dei processi richiedenti. Con **stato sicuro** intendiamo che l'assegnazione di risorse deve ammettere almeno una sequenza sicura. Con **sequenza sicura** intendiamo un ordinamento di n processi tale per cui le richieste di P_i siano soddisfacibili dalle risorse disponibili più quelle possedute da tutti i processi P_j con $j < i$.

Esempio nel caso di risorse aventi un'unica istanza:

definiamo: M_i il numero massimo di risorse che P_i necessita; D come il totale di risorse disponibili; A_{ij} come il numero di risorse assegnate al processo P_i nella fase j ; R_{ij} come il numero di risorse richieste di P_i nella fase j ; $D_j = D - \sum_i A_{ij}$ come il numero di risorse disponibili durante la fase j .

Solo i processi che possono portare a termine la propria esecuzione avranno concesse le risorse richieste, quindi la condizione di terminazione del processo P_i è che $M_i - A_{ij} \leq D_j$.

```

1 // Algoritmo del banchiere
2 #include <stdio.h>
3
4 #define N 3 /* number of processes */
5 #define M 3 /* number of resources */
6
7 int main() {
8     /* Allocation matrix */
9     int alloc[N][M] = {{0, 1, 0}, /* P0 */
10                        {1, 1, 0}, /* P1 */
11                        {0, 0, 1}}; /* P2 */
12
13     /* Max matrix */
14     int max[N][M] = {{1, 1, 0}, /* P0 */
15                     {1, 1, 1}, /* P1 */
16                     {0, 1, 1}}; /* P2 */
17
18     /* Available resources */
19     int avail[M] = {0, 0, 0};
20
21     /* Finish vector */
22     int finish[N];
23     for (int i = 0; i < N; i++)
24         finish[i] = 0;
25
26     /* Needs matrix */
27     int need[N][M];
28     for (int i = 0; i < N; i++) {
29         for (int j = 0; j < M; j++)
30             need[i][j] = max[i][j] - alloc[i][j];
31     }

```



```

32
33  /* Auxiliary variables to keep track of finishing progress */
34  int ans[N], ind = 0;
35
36  int prev_ind;
37  do {
38      prev_ind = ind;
39      /* Scan all processes and remaining needs */
40      for (int i = 0; i < N; i++) {
41          if (finish[i] == 0) { /* Check only unfinished processes */
42              int waiting = 0;
43              for (int j = 0; j < M; j++) {
44                  if (need[i][j] > avail[j]) { /* Process 'i' is waiting for
45                      more resources */
46                      waiting = 1;
47                      break;
48                  }
49              }
50              if (waiting == 0) { /* Process 'i' has enough resources to finish
51                  */
52                  for (int j = 0; j < M; j++)
53                      avail[j] += alloc[i][j]; /* release allocated resources */
54                  finish[i] = 1;
55                  ans[ind++] = i; /* keep track of the finishing order */
56              }
57          }
58      }
59      while (ind > prev_ind); /* continue as long as more processes finish */
60
61      for (int i = 0; i < N; i++) {
62          if (finish[i] == 0) { /* At least one process cannot finish */
63              printf("UNSAFE system\n");
64              return 0; /* exit program */
65          }
66      }
67
68      printf("SAFE sequence\n");
69      for (int i = 0; i < N-1; i++)
70          printf(" P%d ->", ans[i]);
71      printf(" P%d\n", ans[N-1]);
72
73      return 0; /* exit program */

```

Capitolo 4

Sincronizzazione processi

Una **race condition** si verifica quando più processi accedono in maniera concorrente e modificano dati condivisi. In tal caso, l'esito dell'esecuzione, dipende dall'ordine nel quale sono avvenuti gli accessi. Queste situazioni si verificano spesso nei SO in cui diverse componenti compiono operazioni su risorse condivise, per esempio nei sistemi multicore, in cui i thread vengono eseguiti in parallelo su unità distinte.

Per evitare questo tipo di problematica, occorre che i processi vengano sincronizzati. Analizziamo un esempio:

```
1 // Processo produttore
2 elemento appena_prodotto;
3 while (1) {
4     while (contatore ==
5         BUFFER_SIZE) ; // do nothing
6     buffer[in] = appena_prodotto;
7     // index succ pos libera buffer
8     in = (in + 1) % BUFFER_SIZE;
9     contatore++;
10 }
```

```
1 // Processo consumatore
2 elemento da_consumare;
3 while (1) {
4     while (contatore == 0) ; // do
5         nothing
6     da_consumare = buffer[out];
7     // index prima pos piena buffer
8     out = (out - 1) % BUFFER_SIZE;
9     contatore--;
10 }
```

Se i due frammenti di codice vengono eseguiti in concorrenza, le procedure di consumatore e produttore possono non funzionare correttamente. Ciò è dovuto alle istruzioni `contatore++` e `contatore--`, bisogna far sì che tali due istruzioni vengano eseguite **atomicamente**. Un'operazione viene definita **atomica** quando viene completata senza subire interruzioni. A livello di codice macchina il contatore viene incrementato e decrementato in questo modo:

```
1 registro_1 = contatore
2 registro_1 = registro_1 + 1
3 contatore = registro_1
```

```
1 registro_2 = contatore
2 registro_2 = registro_2 - 1
3 contatore = registro_2
```

Se produttore e consumatore tentano di accedere contemporaneamente alla memoria, le istruzioni in linguaggio macchina possono incrociarsi, la sequenza effettiva dipende dallo scheduling della CPU. Per esempio un intreccio potrebbe essere il seguente:

```
1 // Inizialmente contatore = 5
2 t0 -> produttore -> registro_1 = contatore => registro_1 = 5
3 t1 -> produttore -> registro_1 = registro_1+1 => registro_1 = 6
4 t2 -> consumatore -> registro_2 = contatore => registro_2 = 5
5 t3 -> consumatore -> registro_2 = registro_2-1 => registro_2 = 4
6 t4 -> produttore -> contatore = registro_1 => contatore = 6
```

```

7 t5 -> consumatore -> contatore = registro_2 => contatore = 4
8 // Il contatore potrebbe valere 4, 5 oppure 6, mentre dovrebbe valere sempre 5
9 // in quando viene prodotto un elemento e un elemento viene consumato

```

4.1 Il problema delle sezioni critiche

Consideriamo n processi $P_0, \dots, P_n$ che competono per poter utilizzare dati condivisi. Dobbiamo ciascun processo di un segmento di codice (chiamato sezione critica) in cui accede ai dati condivisi. Bisogna dunque definire un protocollo di cooperazione tra i processi cooperanti: ogni processo deve chiedere il permesso di accesso alla sezione critica tramite una *entry section*; la sezione critica termina con un *exit section*.

Una soluzione al problema delle sezioni critiche deve tenere conto di tre aspetti:

1. **mutua esclusione:** se il processo P_i è in esecuzione nella propria sezione critica, nessun altro processo può eseguire la propria sezione critica;
2. **progresso:** se nessun processo è in esecuzione nella propria sezione critica ed esiste qualche processo che desidera accedervi, un processo tra quelli disponibili deve eseguire il codice critico;
3. **attesa limitata:** se un processo ha effettuato la richiesta di ingresso nella sezione critica, è necessario porre un limite al numero di volte che si consente ad altri processi di entrare nelle proprie sezioni critiche, prima che la richiesta del primo processo sia stata accordata (cioè si vuole *starvation*).

Una soluzione a tale problema:

```

1 do {
2     {entry section}
3     codice critico
4     {exit section}
5     codice non critico
6 } while(1);

```

In generale, qualsiasi **soluzione software** al problema della sezione critica richiede l'uso di un semplice strumento: il **lock**. Il processo che deve accedere alla sezione critica deve: acquisire il possesso di un lock in entrata; rilasciare il lock in uscita.

```

1 // Globale:
2 bool lock = false;
3 int x = 0;

```

```

1 // Thread 1:
2 while (!lock) ; // wait
3 print x; // stampa 100

```

```

1 // Thread 2:
2 x = 100;
3 lock = true;

```

4.2 Soluzioni software

In generale le soluzioni hardware (es: *TestAndSet*, *CompareAndSwap*) al problema della sezione critica sono complicate e inaccessibili ai programmatori di applicazioni. Di conseguenza, i progettisti di SO, hanno implementato degli strumenti software per risolvere il problema.

La più semplice delle soluzioni è il **lock mutex** (*mutual exclusion*: si accede ad una sezione critica con un `acquire()` e la si rilascia con un `release()`). Ovviamente queste due funzioni devono essere atomiche, solitamente vengono implementate tramite istruzioni (atomiche) hardware.

```

1 acquire() {
2     while (!available)
3         /*busy waiting*/;
4     available = false;
5 }

```

```

1 release() {
2     available = true
3 }

```

```

1 do {
2     acquire()
3     codice critico
4     release()
5     codice non critico
6 } while(1);

```

Il problema di questo tipo di soluzione è che impone l'**attesa attiva**: mentre un processo si trova nella propria sezione critica, ogni altro processo che ne tenti l'accesso deve ciclare effettuando la chiamata ad `acquire()`. In questo modo il processo mantiene occupata la CPU inutilmente.

4.2.1 Semafori

Un'altra tecnica è quella dei **semafori** (sviluppata da Dijkstra). Un semaforo è semplicemente una variabile s intera non negativa sulla quale possono essere effettuate due operazioni:

1. `wait(s)`: decrementa il valore di s se $s > 0$, altrimenti sospende il processo che ha chiamato l'operazione;
2. `signal(s)`: risveglia un processo sospeso sul dato semaforo dalla coda dei processi in attesa. Nel caso in cui nessun processo sia sospeso sul semaforo, allora s viene incrementato di 1.

Possiamo distinguere due tipi di semafori:

- **semafori binari**: possono assumere soltanto valori 0 e 1. Molto simili al lock mutex.
- **semafori contatori**: possono assumere valori in un dominio non limitato. Solitamente vengono inizializzati con il numero di risorse disponibili.

```

1 wait(S) {
2     while (S <= 0) ; // no-op
3     S--;
4 }

```

```

1 signal(S) {
2     S++;
3 }

```

```

1 // ### Esempio semaforo binario ###
2 semaphore mutex = 1; // variabile condivisa
3 processo() {
4     do {
5         wait(mutex);
6         codice critico
7         signal(mutex);
8         codice non critico
9     } while(1);
10 }

```

Dall'esempio riportato, si nota che sussiste ancora il problema dell'attesa attiva. Si ricorre dunque ad una definizione alternativa di semaforo:

```

1 typedef struct {
2     int valore; // numero processi in attesa
3     struct processo *lista; // puntatore head lista processi in attesa
4 } semaphore;

```

Si assume inoltre che siano disponibili due operazioni (fornite dal SO come system call): **block**, posiziona il processo che richiede di essere bloccato nell'opportuna coda di attesa (cioè sospende il processo e dunque libera la CPU/core); **wakeup**, rimuove un processo dalla coda di attesa e lo sposta nella ready queue.

```

1 wait(semaphore *S) {
2     S->value--;
3     if (S->value < 0) {
4         // aggiungi this proc a
5         S->list
6         block(); // metti in attesa
7     }
8 }

```

```

1 signal(semaphore *S) {
2     S->value++;
3     if (S->value <= 0) {
4         // rimuovi proc P da S->list
5         wakeup(P); // sveglia P
6     }
7 }

```

Dalla nuova definizione di semaforo, si nota che: se il valore è negativo, allora `abs(valore)` indica il numero di processi in coda.

Tramite la realizzazione di semaforo con lista di attesa abbiamo risolto il problema dell'attesa attiva, tuttavia questa soluzione non è esente dal problema dei *deadlock*. Tale situazione si verifica quando due o più processi attendono indefinitivamente un evento che può essere causato soltanto da uno dei due processi in attesa. Per esempio:

```

1 // proc 1
2 wait(S);
3 wait(Q);
4 ...
5 signal(S);
6 signal(Q);

```

```

1 // proc 2
2 wait(Q);
3 wait(S);
4 ...
5 signal(S);
6 signal(Q);

```

Per quanto riguarda la *starvation*: se la coda associata al semaforo viene implementata in modalità FIFO, allora si assicura un'attesa *limitata*, cioè non si verifica la starvation.

4.2.2 Monitor

I semafori costituiscono un meccanismo pratico ed efficiente per la gestione della sincronizzazione dei processi, tuttavia, il loro uso scorretto, può generare errori difficili da individuare, in quanto si manifestano solo in presenza di particolari sequenze di esecuzione. Introduciamo i **monitor**: semplificano molto la gestione della mutua esclusione. In generale si preferisce utilizzare i semafori, i monitor rendono il codice meno parallelizzabile e più sequenziale, per questo motivo la loro gestione è più semplice.

La struttura gerarchica di un monitor è la seguente:

```

1 // ### Pseudocodice sintassi monitor ###
2 monitor monitor-name {
3     ... // dichiarazione variabili condivise
4     function P1(...) { }
5     function P2(...) { }
6     ...
7     function Pn(...) { }
8     initialization_code(...) { }
9 }

```

le variabili condivise possono essere modificate soltanto dalle funzioni specificate all'interno del monitor. Alcune di queste variabili sono di tipo `condition`, queste permettono di sincronizzare i processi. Una variabile `condition` `x` possiede due funzioni per essere gestita:

- `wait` che sospende il processo che la esegue (lo inserisce nella coda associata);
- `signal` che causa il risveglio di un processo sospeso (se sono presenti), oppure è ininfluente. Esistono due politiche differenti in questo caso. Poniamo che il processo *P* abbia invocato `signal` e che *Q* sia in attesa:
 - *segnalare ed attendere*: nel monitor entra *Q* e *P* attende (fuori dal monitor) che *Q* abbia terminato o si metta in attesa su una `condition`;
 - *segnalare e proseguire*: *Q* attende che *P* lasci il monitor o si metta in attesa su una `condition`;

Queste due politiche sono fondamentali, se non si adotta una delle due si ottiene che dopo un `signal` due processi vengono eseguiti simultaneamente all'interno del monitor.

In questo modo si ha che un solo processo per volta può chiamare le funzioni definite all'interno del monitor, quindi un solo processo per volta può modificare i valori delle variabili condivise tra i vari processi.

Esempio: il problema dei 5 filosofi Ci sono 5 filosofi sono seduti attorno a un tavolo circolare; ogni filosofo ha un piatto di spaghetti tanto scivolosi che necessitano di 2 forchette per poter essere mangiati; sul tavolo vi sono in totale 5 forchette. Ogni filosofo ha un comportamento ripetitivo, che alterna due fasi: una fase in cui pensa; una fase in cui mangia. I filosofi non possono mangiare tutti insieme: ci sono solo 5 forchette, mentre ne servirebbero 10; 2 filosofi vicini non possono mangiare contemporaneamente perché condividono una forchetta e pertanto quando uno mangia, l'altro è costretto ad attendere.

```

1  monitor filosofo {
2      enum { PENSA, AFFAMATO, MANGIA } stato[5];
3      condition self[5];
4
5      void prende(int i) {
6          stato[i] = AFFAMATO;
7          test(i);
8          if (stato[i] != MANGIA) self[i].wait;
9      }
10
11     void posa(int i) {
12         stato[i] = PENSA;
13         // controllo vicini a sx e a dx (coda circolare)
14         test((i+4) % 5);
15         test((i+1) % 5);
16     }
17
18     void test(int i) {
19         if ((stato[(i+4) % 5] != MANGIA)
20             && (stato[i] == AFFAMATO)
21             && (stato[(i+1) % 5] != MANGIA)) {
22             stato[i] = MANGIA;
23             self[i].signal();
24         }
25     }

```

```
26
27     codice_iniz() {
28         for (int i = 0; i < 5; i++)
29             stato[i] = PENSA;
30     }
31 }
```

Esempio: monitor per la gestione di mailbox Utilizziamo il costrutto monitor per risolvere il problema della comunicazione tra processi. Il monitor incorpora il buffer dei messaggi (gestito in modo circolare), i processi produttori (o consumatori) inseriranno (o preleveranno) i messaggi tramite le funzioni definite nel monitor.

```
1     monitor buffer_circolare {
2         messaggio buffer[N];
3         int contatore, testa, coda;
4         conditon pieno, vuoto;
5
6         void send(messaggio m) {
7             if (contatore == N) pieno.wait;
8             buffer[coda] = m;
9             coda = (coda+1) % N;
10            ++contatore;
11            vuoto.signal;
12        }
13
14        messaggio recive() {
15            messaggio m;
16            if (contatore == 0) vuoto.wait;
17            m = buffer[testa];
18            testa = (testa+1) % N;
19            --contatore;
20            pieno.signal;
21            return m;
22        }
23
24        codice_iniz() {
25            contatore = testa = coda = 0;
26        }
27    }
```

Capitolo 5

CPU scheduling

5.1 Processi e process scheduling

Il processo viene definito come un programma in esecuzione, si tratta di un'entità attiva, il programma è invece un'entità passiva memorizzata su disco. Ogni processo possiede le seguenti informazioni:

- un ID;
- il program counter;
- una sezione di testo (codice programma da eseguire);
- uno Stack/Heap detta sezione dati;
- lo stato del processo;
- una copia dei registri della CPU;
- informazioni sull'uso della CPU.

Tutte queste informazioni vengono salvate nella *PCB* (*Process Control Block*), quando avviene un context switch vengono caricate le informazioni del PCB del processo in corso.

5.1.1 Stato di un processo

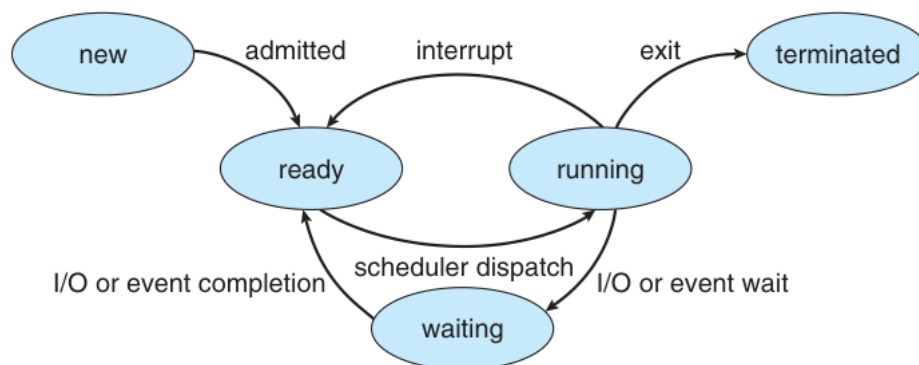


Figura 5.1: Stati di un processo

- **new:** il processo è stato creato;
- **running:** le istruzioni vengono eseguite;
- **waiting:** il processo è in attesa di qualche evento (es: I/O);
- **ready:** il processo sta aspettando che venga assegnato ad un processore;
- **terminated:** il processo ha concluso l'esecuzione.

Il passaggio di un processo da ready a running (o viceversa) dipende dall'*algoritmo di scheduling* in uso. Solitamente questi algoritmi effettuano decisioni in base alla priorità associata al processo. Il modulo che si occupa di assegnare e sottrarre il processo dal processore è il *dispatcher*.

5.1.2 Il dispatcher

Il modulo **dispatcher** passa il controllo della CPU al processo selezionato dallo scheduler a breve termine (cioè lo scheduler che decide quale processo eseguire tra quelli preenti nella coda pronti). Le operazioni di cui si occupa sono: *context-switching*; passaggio a user mode; salto alla posizione corretta del programma utente per riavvianne l'esecuzione.

Nel dispatcher ricopre un ruolo importante la *latenza di dispatch*: indica il tempo impiegato dal dispatcher per sospendere un processo (save state in PCB₀) e avviare una nuova esecuzione (restore state from PCB₁)

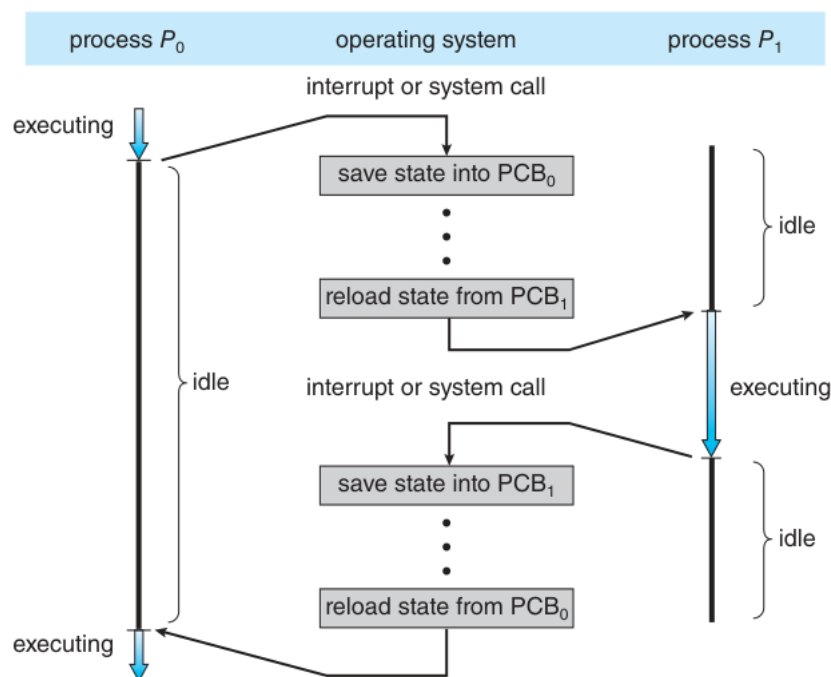


Figura 5.2: Stati di un processo

5.1.3 Scheduling CPU

Come abbiamo anticipato, lo scheduling consiste nell'individuare quale processo sarà il prossimo ad essere eseguito. Risulta conveniente ordinare i processi ready in base ad una priorità,

assegnata dallo scheduler di alto livello. L'assegnazione può essere dinamica, calcolata in base a criteri variabili nel tempo (es: risorse richieste o tempo passato dall'ultimo running). Ad ogni modo questi vengo memorizzati in una struttura dati, una buona implementazione si ottiene utilizzando un heap (in testa l'elemento con priorità massima).

Se il sistema operativo può togliere la CPU ad un processo in esecuzione (*preemption della CPU*), allora si dice dotato di *scheduling preemptive*, altrimenti viene definito *non-preemptive*.

Un algoritmo di scheduling deve tenere conto di alcuni aspetti fondamentali, tra i quali:

- utilizzo CPU: la CPU deve essere il più attiva possibile;
- throughput (produttività): numero dei processi che completano la loro esecuzione nell'unità di tempo;
- tempo di turnaround (tempo di completamento): tempo impiegato per l'esecuzione di un determinato processo;
- tempo di attesa: tempo passato dal processo in attesa nella ready queue;
- tempo di risposta: tempo che intercorre tra la sottoscrizione di una richiesta e le prima risposta prodotta;

5.1.4 Comunicazione tra processi

La comunicazione tra processi è necessaria per diversi motivi: per condividere informazioni; per migliorare la velocità computazionale (task in parallelo tra diversi cores); per la modularità del codice. Tutto ciò richiede un meccanismo di comunicazione tra processi.

Esistono due meccanismi di comunicazione *IPC (Inter-Process Communication)*:

- **scambio di messaggi:** i messaggi vengono inviati tramite system call. Utile per trasmettere piccole quantità di dati (es: semafori);
- **memoria condivisa:** richiede l'intervento del kernel solo per l'allocazione della memoria. Fornisce massima efficienza nella comunicazione.

Entrambi presentano vantaggi e svantaggi. I vantaggi dello scambio di messaggi sono: semplice da utilizzare; nessun potenziale conflitto perché tutto è gestito dal kernel; utile per scambio di piccole quantità di dati. Lo svantaggio è che non sono adatti a frequenti scambi di dati, le continue chiamate a sistema provocano dei rallentamenti. Il vantaggio della memoria condivisa è la velocità, non deve utilizzare troppe system call. Lo svantaggio è che è più difficile da gestire.

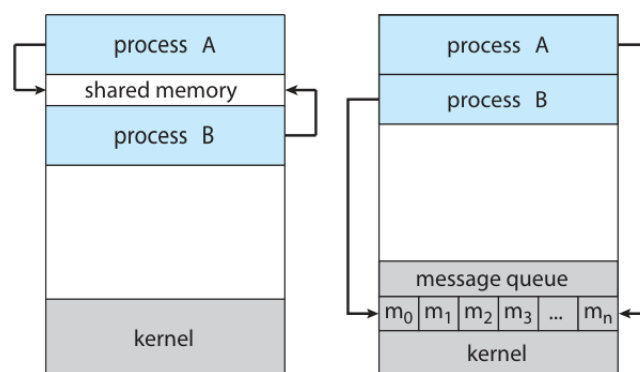


Figura 5.3: Shared memory e scambio di messaggi

5.2 Algoritmi di scheduling

5.2.1 First Come First Served (FCFS)

Si tratta dell'algoritmo di scheduling più semplice: la CPU viene assegnata al processo che la richiede per primo. L'implementazione della politica FCFS viene gestito con una coda FIFO. Quando un processo entra nella coda dei processi pronti, si collega il suo PCB all'ultimo elemento della coda. Quando la CPU è libera, viene assegnata al processo che si trova in testa alla coda e questo viene rimosso dalla coda. Nota che si tratta di uno scheduling non-preemptive.

[illegible]

Per descrivere come si realizza lo scheduling, si realizza il **diagramma di Gantt**: un istogramma che illustra la pianificazione e include i tempi di inizio e fine di ogni processo. Nota che il tempo in cui in processo si trova nello stato running viene detto **CPU-burst**.

Supponiamo che i tre processi arrivino nell'ordine P_1, P_2, P_3 . I tempi di attesa sono: $P_1 \rightarrow 0$, $P_2 \rightarrow 24$, $P_3 \rightarrow 27$. Il tempo medio di attesa è: $T_a = (0 + 24 + 27)/3 = 17ms$.

Se l'ordine fosse P_2, P_3, P_1 , il tempo di attesa medio sarebbe $T_a = (0 + 3 + 6)/3 = 3ms$. Sarebbe quindi molto inferiore: in questo caso non si verifica l'*effetto convoglio*, per il quale i processi di breve durata devono attendere che un processo lungo liberi la CPU.



5.2.2 Shortest Job First (SJF)

Si tratta di un algoritmo che associa ciascun processo al successivo tempo di burst. Quando la CPU è libera, viene assegnata al processo che ha il successivo tempo di burst più piccolo. Se il successivo CPU-burst di due processi è identico, allora viene utilizzata la politica FCFS.

<u>Process</u>	<u>Burst Time</u>
P_1	6
P_2	8
P_3	7
P_4	3

P_4
 P_1
 P_3
 P_2

0 3 9 16 24

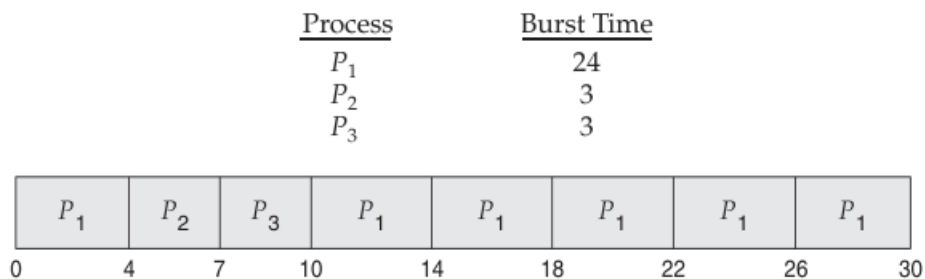
Nell'esempio, i tempi di attesa sono: $P_1 \rightarrow 3$, $P_2 \rightarrow 16$, $P_3 \rightarrow 9$, $P_4 \rightarrow 0$. Il tempo medio è $T_a = (3 + 16 + 9 + 0)/4 = 7ms$.

Il nome corretto per questo algoritmo dovrebbe essere *shortest-next-CPU-burst* in quanto, in realtà, non si basa sulla durata totale del processo.

5.2.3 Round Robin (RR)

Questa politica è simile alla politica FCFS, ma in aggiunta ha la preemption: a ciascun processo viene allocata una piccola quantità di CPU, un **quanto di tempo**; dopo che il quanto è scaduto, se il processo non è terminato, viene accodato alla ready queue (prelazione) e la CPU passa al processo successivo. Questa politica risolve il problema della starvation.

La scelta corretta del quanto di tempo è di fondamentale importanza: se il quanto è troppo grande, allora stiamo di fatto implementando una politica FCFS; se il quanto è troppo piccolo, allora sono richiesti molti contex switch, e di fatto si rallenta il sistema.



Poniamo che il quanto di tempo $q = 4ms$. Il tempo di attesa (somma tempi in cui il processo sta nella coda pronti): $P_1 \rightarrow 6$, $P_2 \rightarrow 4$, $P_3 \rightarrow 7$. Il tempo medio di attesa è $T_a = (6 + 4 + 7)/3 = 5.5ms$.

5.2.4 Shortest Process Next (SPN) e Shortest Remaning Time (SRT)

La politica SPN prevede che la CPU venga assegnata al processo con tempo di esecuzione minore, a parità di valore di utilizza la politica FCFS. Se realizzata non-preemptive favorisce i processi più brevi.

La politica SRT assegna la CPU al processo che ha il minor tempo di esecuzione rimanente. Il problema fondamentale è determinare la durata dei processi. Per effettuarla ci sono degli algoritmi predittivi che si basano sullo storico di processi simili. Si tratta di un algoritmo con prelazione.

5.2.5 Highest Response Ratio Next (HRRN)

Definiamo il fattore di risposta del processo P_i :

$$F_{r_i} = \frac{a_i + e_i}{e_i}$$

dove a_i indica il tempo di attesa del processo nello stato ready, e_i indica il tempo di esecuzione stimato. Lo scheduler attiva il processo che ha il fattore di risposta con il valore più alto. Con questa politica riusciamo a gestire agevolmente sia i processi brevi (basso valore di esecuzione) sia i processi lunghi che sono in coda da tempo (alto valore di attesa).

5.2.6 Code multilivello

Si tratta sostanzialmente di dividere i processi in diverse code a seconda del tipo di processo. Solitamente si distinguono i processi in **foreground** e **background**. Questi due tipi di processi richiedono differenti tempi di risposta, si necessita quindi di due code con diversi algoritmi di scheduling, per esempio: foreground gestita con RR; background gestita con FCFS. In aggiunta a ciò, necessitiamo di un algoritmo di scheduling per gestire le due code:

- *scheduling a priorità fissa*: per esempio si servono prima tutti i processi foreground, poi tutti quelli background (in questo modo si rischia la starvation);
- *time slice*: ciascuna coda occupa un certo intervallo di tempo la CPU che suddivide fra i propri processi. Per esempio possiamo assegnare 80% alla coda foreground e il restante alla coda background.

Generalmente, per assegnare i processi ad una determinata coda, ci si basa su una qualche caratteristica invariante del processo. Un processo può spostarsi fra le varie code (chiamate **code multiple con feedback**), se lo scheduler lo permette. Analizziamo un esempio: poniamo di avere tre code, Q_0 gestita con RR con quanto di tempo 8ms, Q_1 gestita con RR con quanto di tempo di 16ms e infine Q_2 gestita con FCFS. Lo scheduling potrebbe avvenire in questo modo: un nuovo job viene immesso in Q_0 , se non termina finisce in Q_1 e se non termina finisce in Q_2 . Quindi il job verrà servito all'interno dei cicli di CPU lasciati liberi dai processi delle code Q_0 e Q_1 .

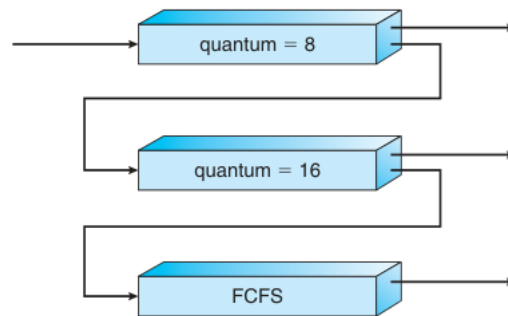
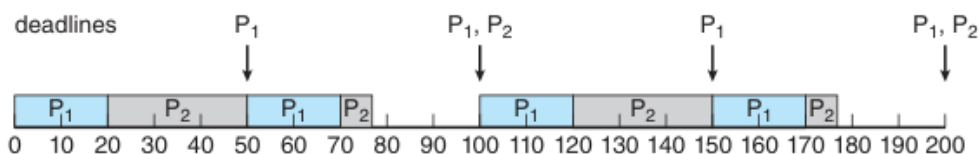


Figura 5.4: Coda multilivello con feedback

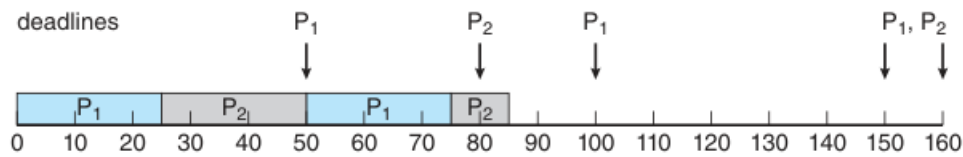
5.3 Scheduling sistemi Real-Time

5.3.1 Scheduling con priorità proporzionale alla frequenza

I processi sono associati ad un certo valore di priorità che è inversamente proporzionale al periodo. Il periodo del processo è fisso e noto a priori dallo scheduler. Per esempio: siano P_1 e P_2 due processi con periodi rispettivamente 50 e 100 e con tempi di CPU-burst di 20 e 35. Quindi il processo P_1 ha priorità maggiore di P_2 .



Analizziamo un altro esempio: P_1 con periodo 50 e CPU-burst di 25; P_2 con periodo 80 e CPU-burst di 35. P_1 ha sempre priorità maggiore.



In questo caso si nota che la scadenza relativa a P_2 non può essere rispettata.

Nel caso di eventi periodici, se esistono m eventi e se l'evento i arriva con un periodo P_i e richiede C_i secondi di tempo di CPU, il carico può essere gestito solo se:

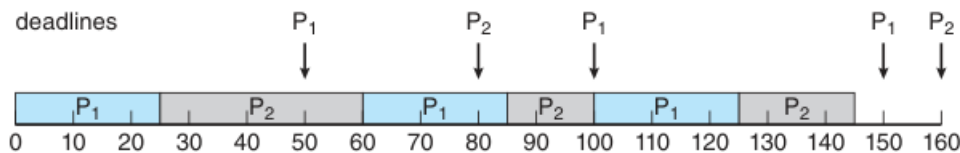
$$\sum_{i=1}^m \frac{C_i}{P_i} \leq 1 \quad (5.1)$$

Un sistema real-time che rispetta questo vincolo è detto **schedulabile**.

5.3.2 Scheduling Earliest-Deadline-First (EDF)

Schedula i processi assegnando le priorità dinamicamente a seconda delle scadenze. Quando un processo diventa eseguibile, deve annunciare la sua prossima scadenza allo scheduler, la priorità viene calcolata di conseguenza in base alla scadenza; più la scadenza è vicina e più la priorità è alta. Dopo che viene fissata la priorità di un nuovo processo, viene aggiornata la priorità dei processi già presenti nel sistema per riflettere la scadenza del nuovo processo.

Per esempio: siano P_1 e P_2 due processi con periodi rispettivamente 50 e 80 e con tempi di CPU-burst di 25 e 35.



Anche per questo algoritmo vale la 5.1.

5.4 Scheduling multiprocessore

Nella sezione precedente abbiamo analizzato gli algoritmi di scheduling considerando CPU single core. Le cose si complicano quando le CPU sono multi core o sono presenti più processori. Esistono diversi approcci per risolvere il problema:

- **asymmetric multiprocessing:** lo scheduling, l'elaborazione dell'I/O e le altre attività del sistema sono affidate ad un solo processore chiamato *master server*. In questo modo si riduce la necessità di condividere dati ma si rischiano colli di bottiglia;
- **symmetric multiprocessing (SMP):** si tratta dell'approccio standard utilizzato da Windows e Linux. Sostanzialmente i processi pronti vanno a formare una coda comune oppure è presente un'apposita coda per ogni processore. Nel secondo caso, ciascun processore ha un proprio scheduler che esamina la coda opportuna per prelevare il prossimo processo da eseguire. Bisogna stare attenti ai possibili sbilanciamenti di carico di

lavoro. Nel primo caso, la programmazione corretta degli scheduler è essenziale: l'accesso concorrente ad una struttura dati comune può portare alla scelta contemporanea di uno stesso processo da eseguire, quindi si possono creare race condition (soluzione: accesso alla coda tramite lock).

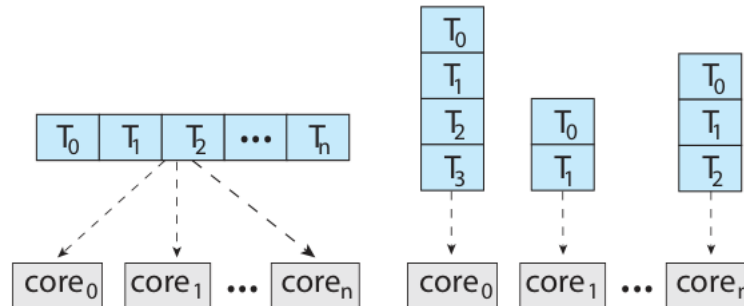


Figura 5.5: Common ready queue e per-core queues

Il *bilanciamento del carico* può essere realizzato tramite un processo dedicato che controlla periodicamente il carico, in tal caso si parla di **migrazione guidata**; alternatively un processore inattivo può sottrarre un processo in attesa ad un processore sovraccarico, in tal caso si parla di **migrazione spontanea**.

Un altro aspetto interessante è la *predilezione per un processore*: un processo viene mantenuto sempre sullo stesso processore, in questo modo viene riutilizzato il contenuto della cache per i CPU-burst successivi. Questo aspetto è influenzato dall'architettura della memoria, bisogna conoscere l'architettura per poter sfruttare la predilezione in maniera efficiente.

Capitolo 6

Gestione memoria principale

6.1 Background

La domanda che ci si pone è: dove vengono memorizzati i processi? Sappiamo che per poter essere eseguiti, i programmi devono essere copiati (almeno parzialmente) in memoria centrale (RAM), successivamente si "trasformano" in processi. La scelta di un metodo di gestione della memoria principale dipende dall'architettura del sistema di calcolo. Ci sono diversi modi per poter immagazzinare i processi in memoria, in questo capitolo analizziamo i problemi, le limitazioni e le soluzioni trovate.

Il primo problema è quello di verificare l'accesso agli indirizzi di memoria: un processo deve poter accedere (quindi leggere e/o modificare) soltanto i propri dati. La soluzione a tale problema prevede che ogni processo abbia un proprio spazio di memoria separato, i limiti di tale spazio vengono definiti da due indirizzi detti *base* e *limite*, tali valori vengono memorizzati in due registri. Quando un processo tenta un accesso in memoria si mette in atto il meccanismo di protezione: la CPU confronta l'indirizzo generato dal processo utente con i valori contenuti nei due registri, se l'accesso è lecito il processo continua la sua esecuzione, in caso non sia lecito il processo viene terminato con un messaggio di errore.

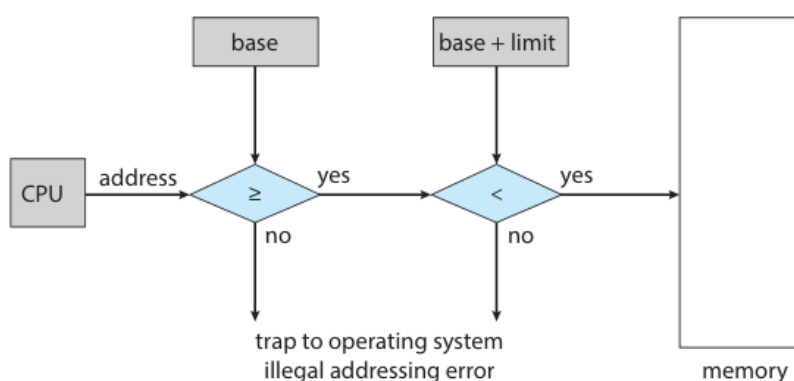


Figura 6.1: Hardware address protection

Prima di essere eseguiti, i programmi utente passano attraverso stadi diversi, con diverse rappresentazioni degli indirizzi:

1. nel programma sorgente gli indirizzi sono simbolici (nomi di variabile);
2. il compilatore associa gli indirizzi simbolici a indirizzi rilocabili, per esempio calcolati a partire dalla prima istruzione del programma;

3. il linker o il loader fa corrispondere gli indirizzi rilocabili a indirizzi assoluti (nella memoria fisica).

L'associazione (detta **binding**) di istruzioni e dati a indirizzi di memoria può avvenire durante una delle tre fasi:

1. **compilazione:** se la posizione in memoria del processo è nota a priori, può essere generato codice assoluto. Il problema è che se la locazione iniziale cambia (es: swapping di una parte del processo), è necessario ricompilare tutto il codice;
2. **caricamento:** se la posizione in memoria non è nota in fase di compilazione, è necessario generare codice *rilocabile*. Il compilatore genera indirizzi relativi che vengono convertiti in indirizzi assoluti dal loader. Il problema è che se l'indirizzo iniziale cambia, il codice deve essere ricaricato in memoria;
3. **esecuzione:** se il processo può essere spostato a run-time da un segmento di memoria all'altro, il binding viene rimandato fino al momento dell'esecuzione. Si parla di **codice dinamicamente rilocabile**. Questo approccio necessita un opportuno supporto hardware per il mapping degli indirizzi, ad esempio attraverso registri base e limite o *tabella delle pagine*.

6.2 Allocazione contigua

Si tratta di un metodo per immagazzinare processi in memoria: quando un processo deve essere inserito in RAM, gli viene assegnata un'area di memoria contigua. Questo metodo presenta il problema della **frammentazione**. La frammentazione può essere:

- **interna:** la memoria allocata può essere leggermente maggiore rispetto a quella effettivamente richiesta dal processo, la differenza di dimensioni indica lo spazio non utilizzato;
- **esterna:** all'interno della memoria principale, è presente spazio a soddisfare una richiesta ma non è contiguo. Per ridurre questa frammentazione si *compatta* la memoria.

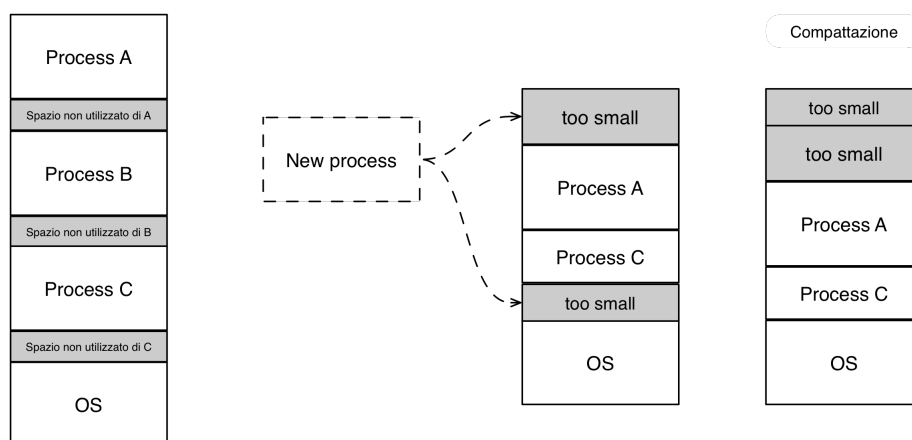


Figura 6.2: Frammentazione e compattazione

6.3 Paginazione

Un metodo alternativo all'allocazione contigua è la **paginazione**. Questo metodo consente la non contiguità dello spazio degli indirizzi fisici, in questo modo i processi possono essere allocati ovunque nella memoria principale. Sostanzialmente la memoria fisica viene divisa in blocchi di dimensione fissa, chiamanti **pagine fisiche** o **frame**; analogamente si divide la memoria logica (i processi) in blocchi della stessa dimensione chiamati **pagine**.

Gli indirizzi logici sono contigui, gli indirizzi fisici no. Per poter risalire ad un indirizzo fisico partendo da uno logico si sfrutta la **page table**, in questo modo ogni pagina logica viene mappata con la corrispondente pagina fisica. La CPU genera un indirizzo che viene suddiviso in due parti:

- *numero di pagina*
- *offset della variabile all'interno della pagina.*

In questo modo si riduce notevolmente il problema della frammentazione esterna, rimane comunque la frammentazione interna.

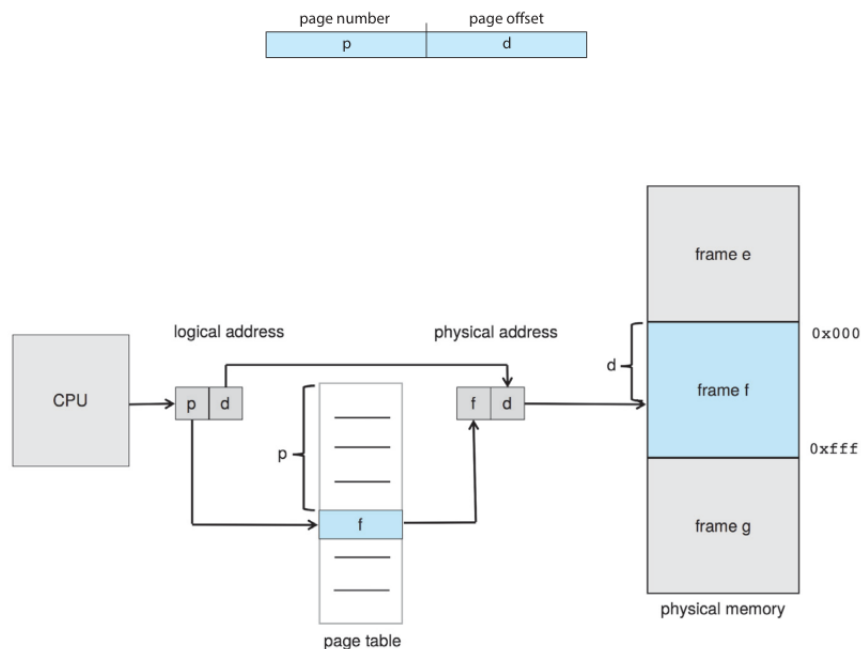


Figura 6.3: Paging hardware

Ogni processo ha la propria page table che risiede in memoria centrale, il problema di questo metodo è che ogni accesso ai dati o istruzioni richiede due accessi alla memoria: uno per la tabella e uno per le istruzioni/dati. Per risolvere tale problema si sfrutta una **translation look-aside buffer (TLB)** una memoria cache molto veloce. Se il numero di pagina è presente nel TLB si ha un hit e dunque si reperisce il numero di frame, in caso di miss si cerca il numero di frame nella page table caricata in memoria e successivamente l'informazione viene caricata anche nel TLB.

Per garantire la protezione nel TLB, viene aggiunta la colonna *Address Space Identifier (ASID)* che indica il processo proprietario degli indirizzi (e quindi chi può accedere a cosa).

Nota che la page table può provocare un *page fault*, in tal caso sarà necessario caricare la pagina dalla memoria secondaria nella memoria primaria. Il page fault si verifica (per esempio) ogni volta che un programma viene lanciato per la prima volta: la CPU deve accedere alla

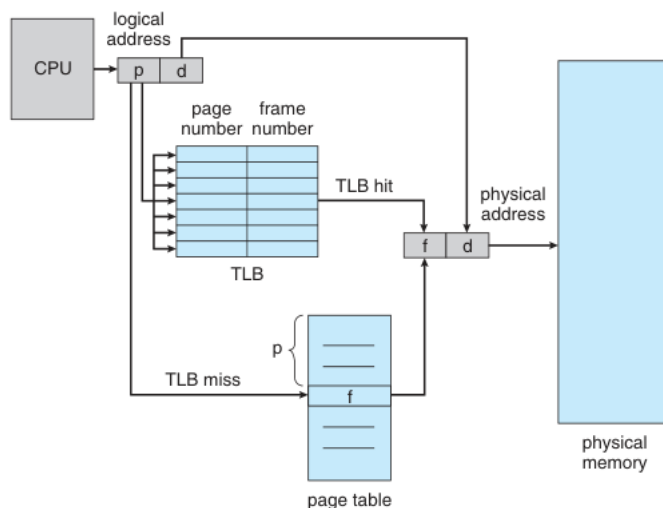


Figura 6.4: Paging hardware

memoria secondaria (dove abbiamo salvato il file eseguibile) e caricare la page table nella memoria principale.

Il problema in generale con il metodo delle page table è che ogni tabella può contenere milioni di elementi e ogni processo ha la propria tabella, quindi l'occupazione di memoria può essere elevata. Per risolvere tale problema esistono diverse strategie, analizziamole.

6.3.1 Tabella delle pagine invertita

Un elemento della tabella per ogni frame; gli elementi della tabella contengono l'indirizzo virtuale della pagina memorizzata nel dato frame e contengono le informazioni sul processo che possiede tale pagina; gli indirizzi sono costituiti dalla tripletta <#processo, #pagina, offset>. Con questa soluzione lo spazio richiesto è ridotto, esiste infatti un'unica tabella per tutti i processi. Come contro ha l'incremento del tempo necessario per ricercare nella tabella: bisogna scorrerla tutta.

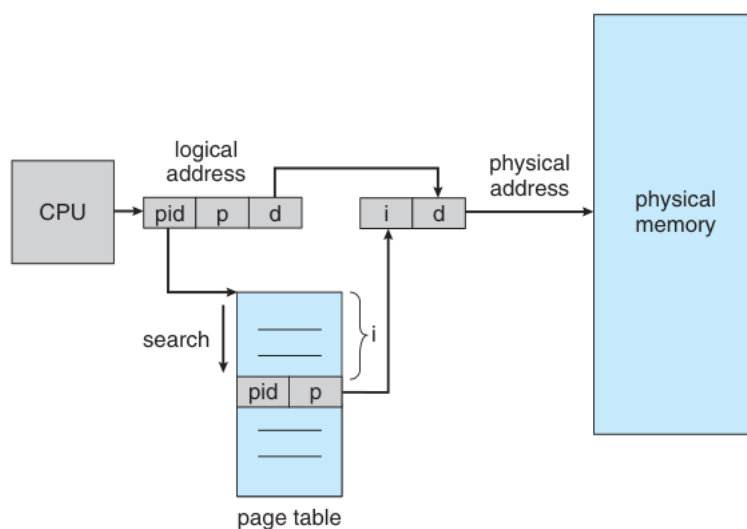


Figura 6.5: Page table invertita

6.3.2 Paginazione gerarchica

La tabella delle pagine viene essa stessa paginata. Vediamo un esempio di paginazione a due livelli per calcolatori a 32 bit.

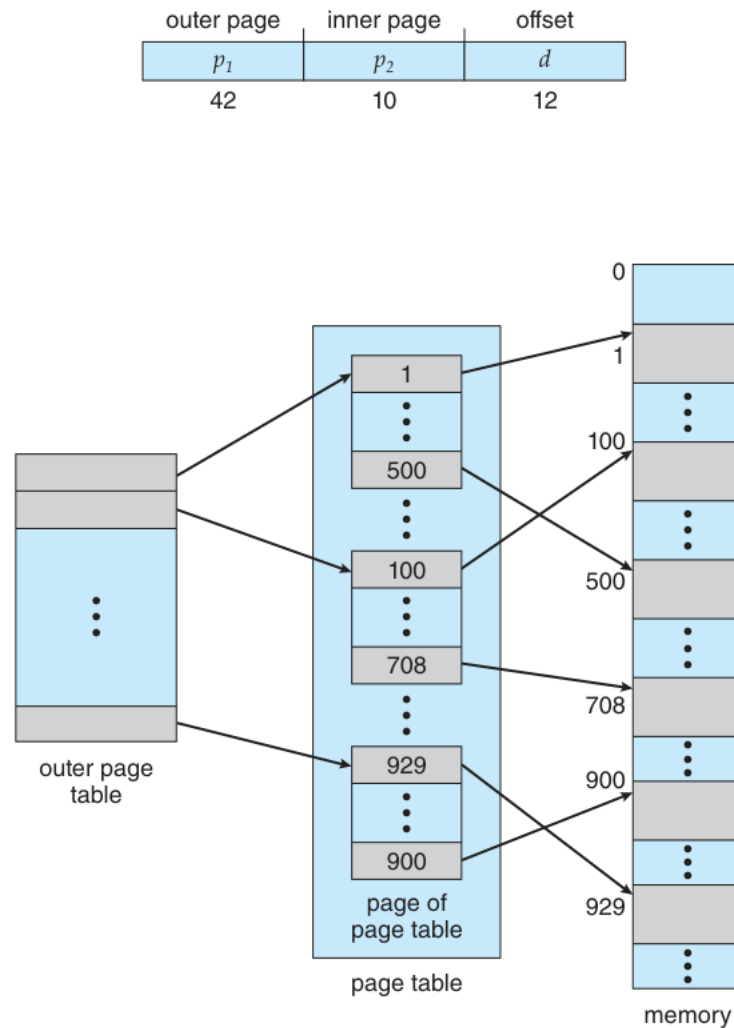


Figura 6.6: Page table invertita

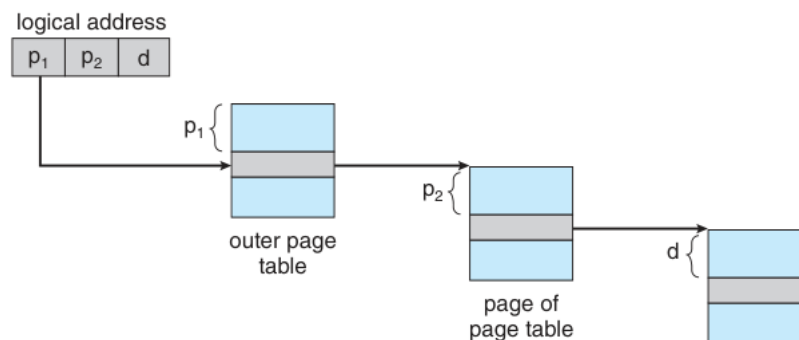


Figura 6.7: Traduzione dell'indirizzo

Siccome ciascun livello viene memorizzato come una tabella separata, la traduzione di un indirizzo può richiedere fino a tre accessi alla memoria. Tuttavia la presenza di una cache consente di mantenere delle prestazioni ragionevoli.

Per i sistemi a 64 bit questa soluzione non è praticabile.

6.3.3 Tabella delle pagine hash

Si tratta della strategia comune per spazi di indirizzi a più di 32 bit. Ciascun elemento della tabella è composto da tre campi: numero della pagina virtuale; indirizzo del frame corrispondente; puntatore al successivo elemento nella lista. La funzione hash prende in input il numero della pagina virtuale e ritorna come output l'indice della tabella hash associato. Il numero di pagina virtuale dell'indirizzo deve essere confrontato con tutti gli elementi della catena; se il valore cercato è presente, si estrae il corrispondente numero di frame e dunque si risale all'indirizzo fisico.

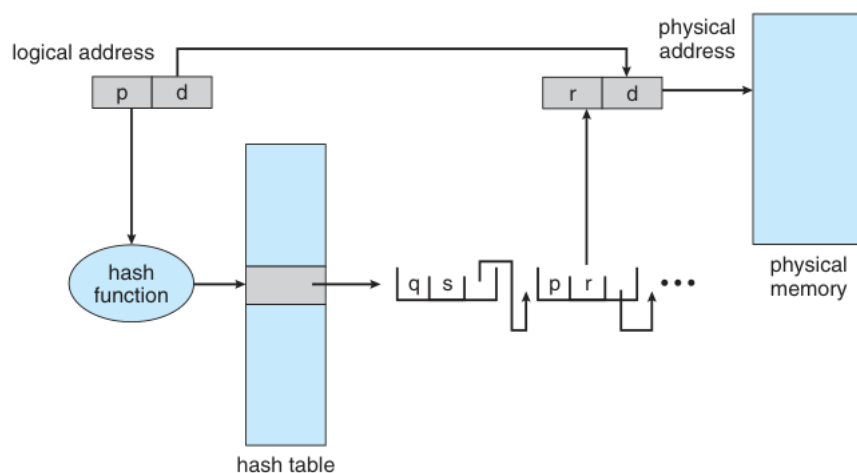


Figura 6.8: Traduzione dell'indirizzo

6.4 Segmentazione

Un programma è una collezione di segmenti; i segmenti sono unità logiche che si differenziano in base al contenuto e alla dimensione. Alcuni segmenti sono: programma main; procedure e funzioni; variabili locali e globali; strutture dati (tabella dei simboli, matrici, stack, etc). Gli elementi all'interno di un segmento vengono identificati mediante il loro offset dall'inizio del segmento. Gli indirizzi sono composti da due sezioni: `<#segmento, offset>`.

Abbiamo visto che la paginazione non soffriva del problema della frammentazione esterna ma della frammentazione interna. Con la segmentazione vale il viceversa.

Per tradurre gli indirizzi logici in indirizzi fisici, si sfrutta la **tabella dei segmenti** (una per processo). Le entrate della tabella contengono:

- *base*: indirizzo fisico iniziale della memoria contenente il segmento;
- *limite*: specifica la lunghezza del segmento.

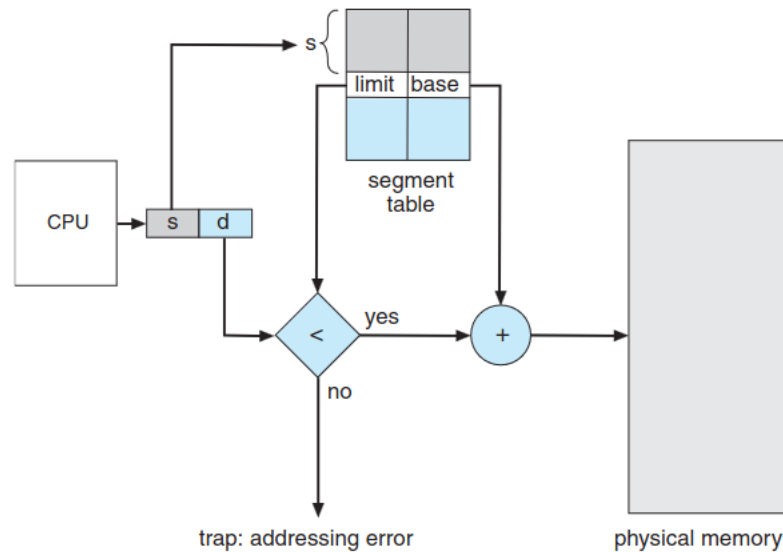


Figura 6.9: Segmentazione hardware

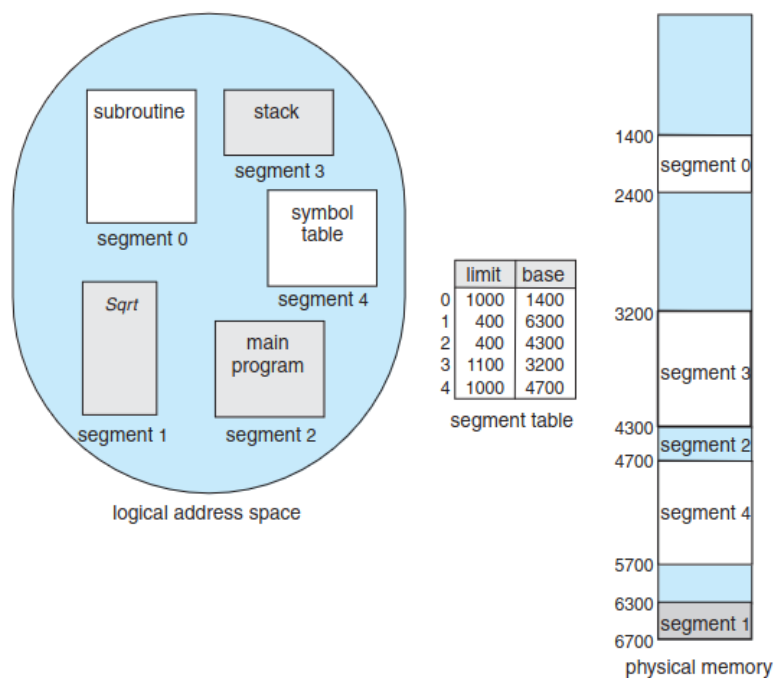


Figura 6.10: Esempio di segmentazione

6.5 Memoria virtuale

La memoria virtuale funge da estensione per la memoria principale. Molto raramente capita che tutto il codice relativo ad un dato programma sia necessario durante l'esecuzione, per questo motivo possiamo caricare in memoria secondaria alcune pagine che successivamente verranno caricate in memoria principale se e quando saranno necessarie. Il vantaggio di questo meccanismo è che i programmi non sono più vincolati alla quantità di memoria fisica

disponibile, inoltre, poiché ogni programma occupa meno memoria fisica, aumenta il grado di multiprogrammazione e in generale la produttività del sistema.

Con *spazio degli indirizzi virtuali* ci si riferisce alla collocazione dei processi in memoria dal punto di vista logico. Un processo inizia in corrispondenza dell'indirizzo logico 0 e si estende alla memoria contigua fino all'indirizzo Max, viceversa, la memoria fisica allocata al processo è composta da pagine sparse. Il mapping di indirizzi logici a fisici viene effettuato dall'*MMU* (*Mapping and Management Unit*).

Come abbiamo detto, non è necessario che i processi vengano spostati per intero in RAM durante la fase di loading, si può decidere di caricare una particolare pagina solo quando è necessaria. Il modulo del sistema operativo che si occupa della paginazione su richiesta è il **pager**. Quando un processo sta per essere caricato in memoria, il pager ipotizza quali pagine saranno usate e carica in memoria solo quelle che ritiene necessarie. Se la pagina richiesta dal processo è già presente in memoria, allora l'esecuzione procede come di consueto; se la pagina non è presente (**page fault**), allora bisogna localizzarla nella memoria secondaria e trasferirla in quella principale. Per risalire allo stato di una pagina, si sfrutta un bit di validità presente nella page table: se il bit vale *i* allora la pagina non è valida o non è presente in memoria; se il bit vale *v* allora la pagina è in memoria.

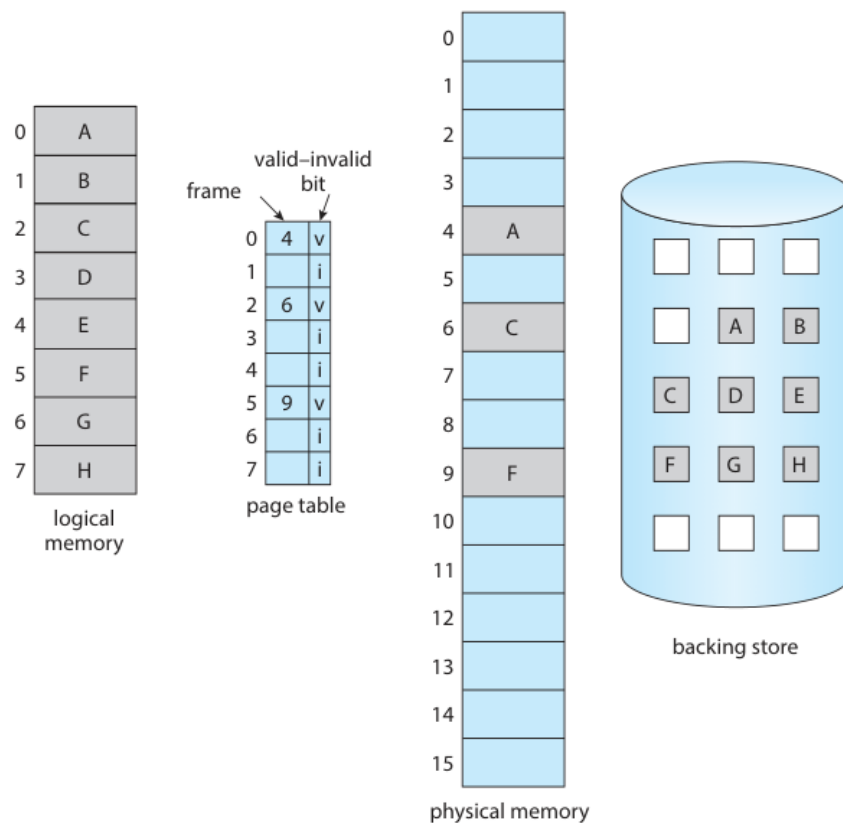


Figura 6.11: Page table e memoria virtuale

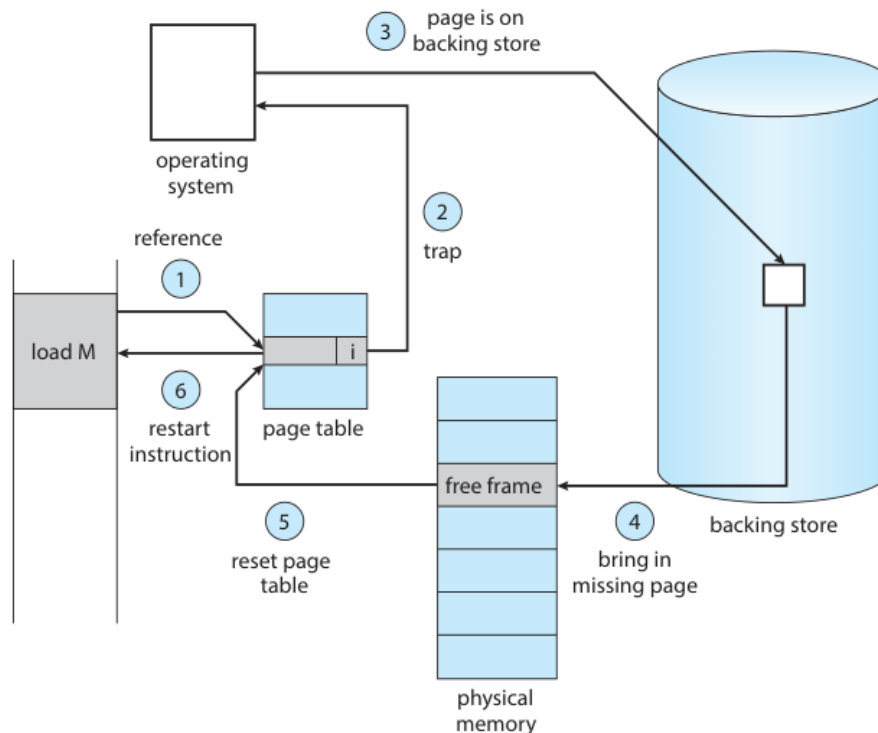


Figura 6.12: Gestione di un page fault

Un strategia adottata per la gestione dei page fault è la **demand paging**: una pagina viene caricata in memoria principale solo quando il processo la chiede. In questo modo verranno provocati tanti page fault all'inizio dell'esecuzione del processo, finché non verranno caricate in memoria tutte le pagine "necessarie".

6.5.1 Sostituzione delle pagine

Se durante l'esecuzione di un processo utente si verifica un page fault, una nuova pagina deve essere caricata in memoria. Se non vi sono frame liberi, quale sostituiamo? La scelta della pagina da sostituire viene fatta con un algoritmo di selezione: le prestazioni di tale algoritmo dipendono dal numero di page fault che produce. Se la pagina che deve essere sostituita è stata modificata, allora deve essere aggiornata su disco, questo processo ovviamente richiede tempo. Per ridurre il sovraccarico dei trasferimenti di pagine è presente un **dirty bit**: se è settato significa che la pagina è stata modificata e di conseguenza deve essere aggiornata su disco prima di essere sostituita.

Come abbiamo detto, la pagina che deve essere sostituita viene scelta da un algoritmo. Questi algoritmi vengono valutati eseguendoli su una particolare stringa di riferimenti a memoria chiamata *reference string* (stringa costituita da soli numeri di pagina e non da indirizzi) e vengono contati i page fault. Analizziamo alcuni algoritmi di sostituzione.

FIFO Viene implementato mediante una coda FIFO relativa a tutte le pagine del processo residenti in memoria. Il numero di frame assegnati per processo è fissato e la prossima pagina che verrà sostituita è quella che si trova in testa alla coda.

L'esempio riportato nella Figura 6.13 ci sono tre frame, quindi tre pagine per ciascun processo possono trovarsi contemporaneamente in memoria. Utilizzando l'algoritmo FIFO sulla reference string riportata, risulta che il numero di page fault è 15. Si potrebbe essere

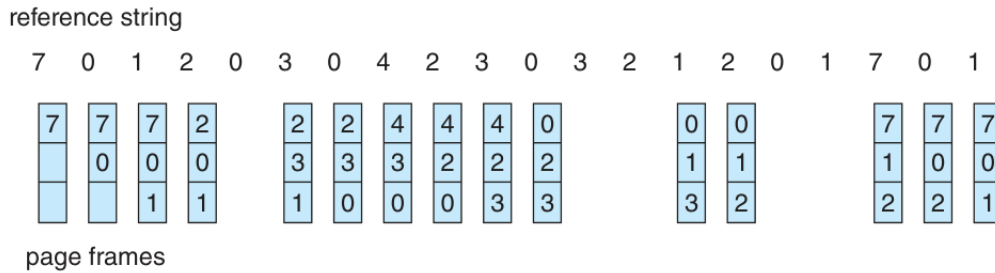


Figura 6.13: Algoritmo di sostituzione FIFO

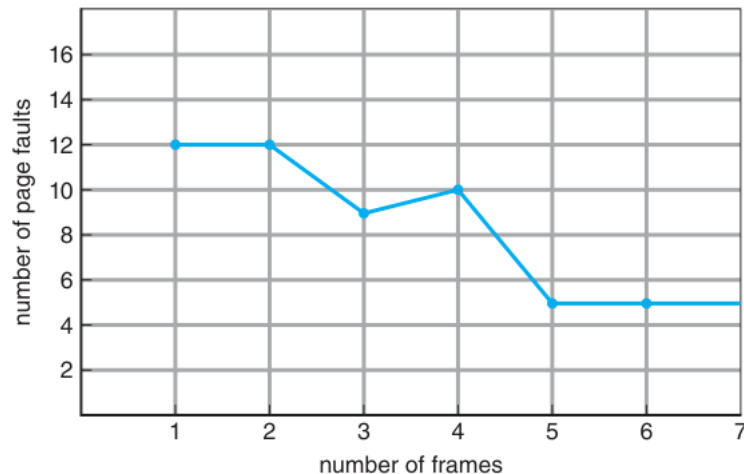


Figura 6.14: Grafico page fault in funzione del numero di frame

indotti a pensare che, aumentando il numero di frame per processo, il numero di page fault diminuirebbe di conseguenza. In realtà si osserva (Figura 6.14) che la relazione è si decrescente, ma non monotona. Tale anomalia (nota come *anomalia di Bledy*) si verifica quando il processo richiede consecutivamente un insieme di pagine in maniera ciclica, e tale insieme ha una dimensione uguale o maggiore del numero di frame fisici liberi.

OPT Si tratta dell'algoritmo ottimale (optimal). L'algoritmo ottimale si basa sul futuro: sostituisce la pagina che non verrà usata per il periodo di tempo più lungo. Si tratta di un algoritmo non realizzabile, è utile in quanto permette di calcolare il minimo numero di page fault ottenibile da una data reference string. Nell'esempio di Figura 6.15 il numero di page fault è 9.

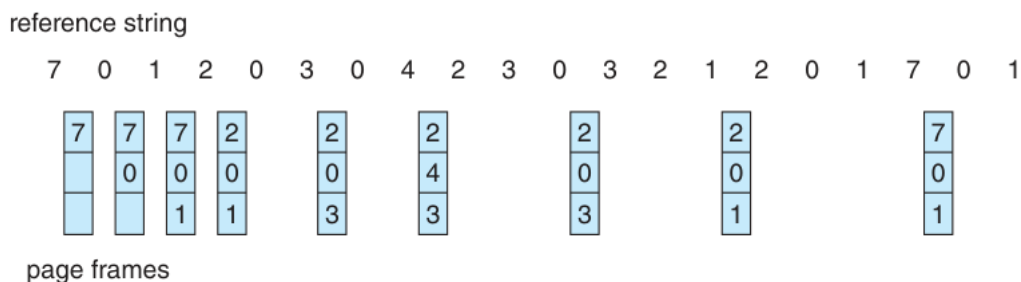


Figura 6.15: Algoritmo di sostituzione OPT

LRU Last Recently Used. Usa la conoscenza passata (acquisita) per prendere le decisioni: viene rimpiazzata la pagina che non è stata utilizzata per più tempo. Dunque ogni pagina viene associato il tempo di ultimo accesso. Nell'esempio di Figura 6.16 il numero di page fault è 12.

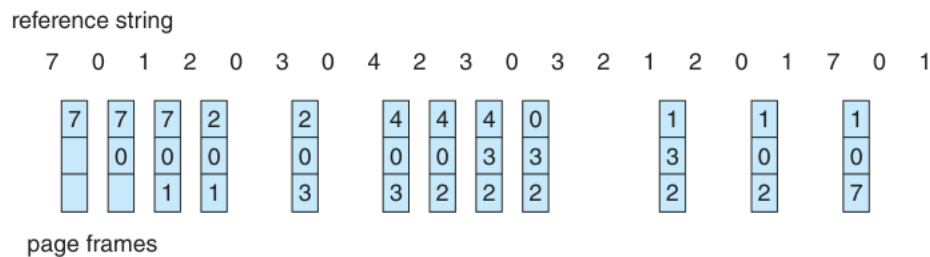


Figura 6.16: Algoritmo di sostituzione LRU

L'algoritmo potrebbe essere implementato sfruttando un contatore associato ad ogni pagina: ogni volta che viene effettuato un riferimento a tale pagina, il contatore viene incrementato. Il problema di tale approccio è che la ricerca della pagina da rimpiazzare richiede una ricerca lineare. L'implementazione più efficiente è quella realizzata mediante stack: si mantiene uno stack di numeri di pagina implementato mediante lista doppiamente concatenata. Ogni volta che una pagina viene referenziata, viene posta in cima allo stack, di conseguenza la pagina più referenziata si trova in cima e quella meno referenziata sempre al fondo. Questa operazione richiede di aggiornare al più 6 puntatori.

Entrambe le implementazioni risultano in ogni caso troppo lente senza la presenza di hardware dedicato. I sistemi tuttavia forniscono un aiuto hardware: il **bit di riferimento** in combinazione con i **registri a scorrimento**. A ciascuna pagina si associa un bit inizialmente inizializzato a 0; quando si fa riferimento a tale pagina si pone il bit a 1; viene rimpiazzata la pagina il cui riferimento di bit vale 0 (se esiste). Attraverso i registri di scorrimento (uno per pagina) si mantiene la cronologia delle variazioni dei bit di riferimento. Osservando in un dato istante i registri sapremo quando la pagina è stata utilizzata; quando il registro è pieno, viene semplicemente shiftato a destra (i record più vecchi vengono persi). Per esempio: una pagina con registro di scorrimento 11000100 è acceduta più recentemente di una pagina con registro di scorrimento 01100101 (si nota dal bit più significativo).

L'algoritmo non soffre dell'anomalia di Bledy: in un frame di dimensione n , le n pagine allocate sono le più utilizzate; se si aumenta il numero di frame a $n+1$ il discorso non cambia. In generale, tutti gli algoritmi "a pila" non soffrono dell'anomalia.

Seconda Chance Di base si tratta di un algoritmo FIFO. Necessita di un bit di riferimento. Quando la pagina riceve una seconda chance, il bit di riferimento viene azzerato ed il tempo di arrivo viene aggiornato al tempo attuale. Se la pagina da rimpiazzare ha il bit di riferimento a 1, allora: si pone il bit di riferimento a 0; si lascia la pagina in memoria; si rimpiazza la pagina successiva, in base alle stesse regole.

Seconda Chance migliorato Si tratta sempre di un algoritmo FIFO. Si considera la coppia (bit riferimento, bit modifica). Si ottengono quattro casi:

1. (0,0): non recentemente usata ne modificata. Rappresenta la migliore scelta per la sostituzione;
2. (0,1): non recentemente usata, ma modificata. Deve essere salvata su memoria di massa;

3. (1,0): recentemente usata, ma non modificata. Probabilmente verrà acceduta di nuovo a breve;
4. (1,1): recentemente usata e modificata. Rappresenta la migliore scelta per la sostituzione; verrà probabilmente acceduta in breve termine e deve essere salvata su memoria di massa prima di essere sostituita. Rappresenta la scelta peggiore.

Durante la ricerca (sequenziale) della pagina da sostituire, viene azzerato il bit di riferimento della pagina se questo è settato. Viene sostituita la prima pagina che si trova nella classe minima non trovata. Come si può intuire, la scansione può essere eseguita più volte prima di determinare la pagina da sostituire.

6.5.2 Allocazione dei frame

Ciascun processo utente richiede un numero minimo di pagine fisiche. Il numero minimo di frame necessari per ogni processo è definito dall'architettura, il massimo dipende dalla disponibilità di memoria. Elenchiamo i principali metodi di allocazione:

- **allocazione uniforme:** se si hanno n processi e m frame liberi, si assegna a ciascun processo $\frac{m}{n}$ frame.
- **allocazione proporzionale:** si allocano frame sulla base della dimensione del processo.
- **allocazione con priorità:** si impiega uno schema di allocazione proporzionale basato sulla priorità piuttosto che sulla dimensione. Se un processo genera un page fault, si seleziona per la sostituzione uno dei suoi frame (sostituzione locale), oppure si seleziona un frame di un processo con priorità minore (sostituzione globale, la più diffusa nei SO moderni). La sostituzione globale porta con sé un potenziale problema: processi a minore priorità provocano molti page fault, ne risulta che, un processo, è costantemente occupato a spostare pagine dal disco alla memoria e viceversa. Questo fenomeno prende il nome di **trashing**.
- **working set:** si tratta dell'insieme di pagine attive di certo processo, cioè delle pagine che sono contemporaneamente in uso attivo e sono spesso utilizzate. Per determinare tale insieme bisogna definire Δ : quando indietro vogliamo guardare rispetto allo storico di pagine accedute dal processo (partendo dall'ultimo ovviamente). L'insieme varia nel tempo; se idealmente fosse infinito, comprenderebbe tutto lo storico delle pagine accedute dal processo. Per esempio se lo storico fosse 3, 4, 2, 6, 1, 5, 7, 7, 7, 5, 1 e il $\Delta = 5$, allora il working set comprenderebbe le pagine {1, 5, 7}. Se la somma dei working set di tutti i processi (cioè il totale di pagine richieste) è maggiore del numero totale di frame, allora si ha trashing. Per risolvere il problema occorre sospendere un processo o sottoporlo a swapping. Esiste una correlazione tra la frequenza di page fault e il cambio di località del working set. Essendo che il working set varia nel tempo, quando si verifica un cambio di località del processo, si nota un picco nel grafico della frequenza di page fault.

6.5.3 Allocazione di memoria al Kernel

L'allocazione dei processi Kernel, attinge ad una riserva di memoria libera diversa rispetto a quella utilizzata per soddisfare i processi utente. Il Kernel richiede memoria per strutture dati dalle dimensioni variabili, viene posta maggiore attenzione agli sprechi di memoria e inoltre, in molti SO, il codice dati del Kernel non sono soggetti a paginazione. Parti della memoria del Kernel devono essere contigue perché alcuni dispositivi accedono direttamente alla memoria fisica senza l'interfaccia della memoria virtuale.

Vediamo due tecniche per allocare la memoria relativa al Kernel: **allocatore potenza di due; allocatore a lastra**. Il primo utilizza un segmento di dimensione fissa per l'allocazione della memoria, composto da pagine fisicamente contigue. Nel caso specifico dell'allocatore a potenza di due, i blocchi hanno dimensione pari a potenze di due, oppure, se non è presente abbastanza spazio, alla più piccola potenza del due che la contiene. Se viene richiesta meno memoria di quella che costituisce il segmento corrente, questo viene diviso in due segmenti gemelli di identica dimensione.

Per esempio: supponiamo di avere a disposizione un segmento da 256Kb e che il Kernel richieda invece 21Kb. Inizialmente in segmento viene diviso in due parti A_L e A_R , ciascuno di 128Kb; un'ulteriore divisione produce B_L e B_R di 64Kb, infine si ottengono C_L e C_R da 32Kb ciascuno. C_L viene utilizzato per soddisfare la richiesta, gli altri spazi vengono riempiti parzialmente (o non) per altre richieste. Quando due blocchi adiacenti sono entrambi liberi, vengono fusi nuovamente. Questa tecnica è molto utilizzata anche se genera frammentazione interna.

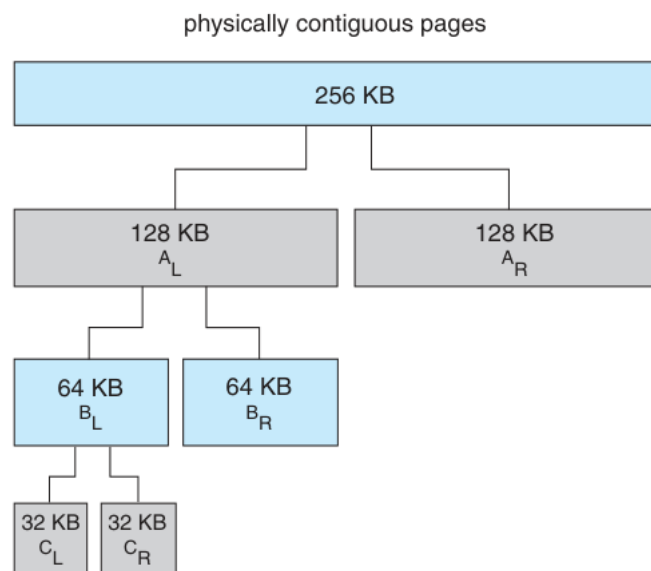


Figura 6.17: Allocatore potenza di 2

L'allocatore a lastra (slab) viene inventato per risolvere la frammentazione interna generata dalla soluzione precedente. Una slab è composta da uno o più frame fisicamente contigui, più slab vengono salvate in una cache. Vi è una sola cache per ciascuna categoria di strutture dati del kernel (una per i PCB, una ai descrittori di file, una ai semafori, etc), e ognuna è popolata da oggetti (istanze della relativa struttura dati). Una slab può trovarsi in tre stati: piena, se tutti gli oggetti sono contrassegnati come usati; vuota se tutti gli oggetti sono contrassegnati come liberi; parzialmente occupata. Quando una lastra diventa piena, un nuovo oggetto viene allocato in una slab completamente vuota, se non ne esistono, si procede all'allocazione di una nuova lastra (cache grow) e la si appende in fondo alla cache opportuna.

In che modo l'allocatore a lastra riduce la frammentazione interna? Semplicemente si può calcolare la dimensione di ciascuno slab in modo da ridurre al minimo la memoria interna sprecata; i byte "sprecati" entro lo slab possono essere sfruttati per contenere altre informazioni riguardanti quest'ultimo (es: lo stato libero/occupato degli oggetti).

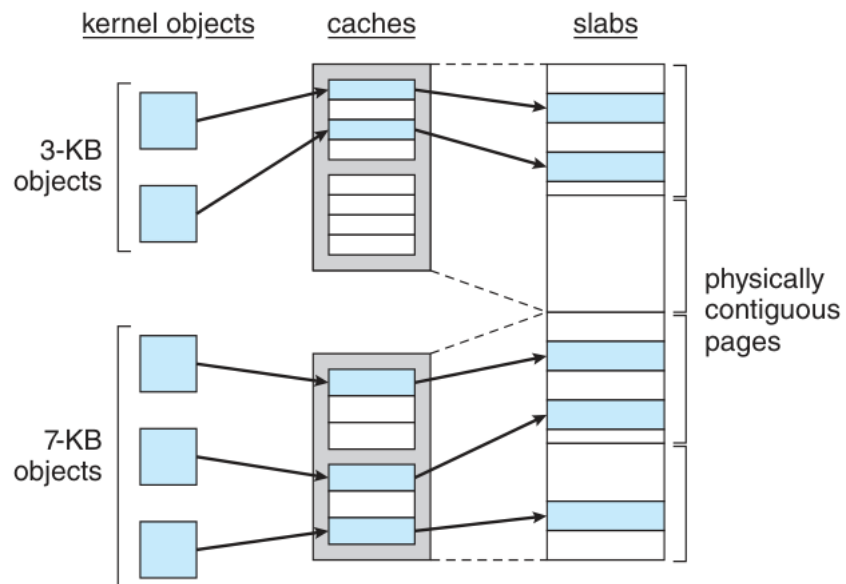


Figura 6.18: Allocatore a lastra

Capitolo 7

Gestione file

7.1 Introduzione

La struttura logica con cui il sistema operativo gestisce i file prende il nome di **file system**. Il file system è responsabile della gestione dei file in memoria di massa, si occupa della scrittura dei dati nei file, di organizzare i file in directory, di fornire all'utente un insieme di funzioni che permettano di operare sui file e deve garantire una gestione dei file indipendente dalle caratteristiche fisiche dei dispositivi che costituiscono la memoria di massa (astrazione utile sia all'utente sia al programma). Tutti i moderni sistemi operativi gestiscono l'archiviazione di file in modo gerarchico (*HFS - Hierarchical File System*), tuttavia tale organizzazione non ha alcun legame con la posizione fisica effettiva dei file in memoria di massa.

Un **file** consiste in uno spazio di indirizzi logici contigui; è un insieme di informazioni correlate registrate nella memoria secondaria a cui è stato associato un nome. I file vengono identificati dal loro *tipo*. Il tipo è un attributo che indica la struttura logica interna del file e permette al sistema operativo di interpretarne correttamente il contenuto. Esistono due tecniche principali per identificare il tipo di un file:

- meccanismo delle estensioni: il tipo viene indicato da un suffisso del nome (DOS);
- magic number: il tipo è rappresentato da un valore posto all'inizio del file (UNIX).

7.2 Operazioni sui file

Le operazioni possibili sono:

- creazione: reperire lo spazio all'interno del file system e creare un nuovo elemento nella directory in cui registrare le informazioni relative;
- scrittura: chiamata al sistema con il nome del file e il puntatore ai dati da scrivere come parametri, reperimento file, scrittura dei dati nella posizione indicata dal puntatore di scrittura e aggiornamento del puntatore;
- lettura: chiamata al sistema con nome del file e indirizzo di memoria, reperimento file nel file system, lettura dei dati nella posizione indicata dal puntatore di lettura e aggiornamento del puntatore;
- riposizionamento: reperimento file nel file system, aggiornamento del puntatore alla posizione corrente;
- cancellazione: reperimento file nel file system, si rilascia lo spazio allocato al file e si elimina il corrispondente elemento della directory.

Il sistema operativo mantiene in memoria centrale una tabella contenente informazioni su tutti i file aperti. Quando viene richiesta un'operazione su file, vengono ricercate le informazioni relative (tramite puntatore) nella tabella. Le chiamate di sistema per aprire e chiudere un file sono:

- `open(File)`: ricerca file su disco e copia il contenuto nella tabella dei file aperti in memoria centrale, riporta un puntatore all'elemento nella tabella;
- `close(File)`: copia il contenuto del file su disco e rimuove l'elemento dalla tabella.

Per controllare l'accesso ai file da parte dei processi, si utilizza il meccanismo del lock. Il lock può essere *shared* (più processi possono acquisirlo in contemporanea) oppure *exclusive* (un solo processo può acquisirlo). Alcuni SO (Windows) bloccano l'accesso ai processi che tentano di accedere ad un file che ha il lock acquisito; altri SO (UNIX) permettono l'accesso in qualsiasi caso.

7.2.1 Accesso ai file

L'accesso ai file può essere di tre tipi:

- sequenziale: il puntatore segue sequenzialmente il file dall'inizio alla fine;
- diretto: i dati vengono memorizzati in forma di record (dimensione fissa). In questo modo è possibile accedere direttamente ai dati tramite un offset;
- a indice: implica la costruzione di un file indice per l'accesso a file, che viene mantenuto in memoria centrale. L'indice contiene puntatori a blocchi del file; per reperire un elemento occorre cercare nell'indice l'elemento corrispondente e utilizzare il puntatore in esso contenuto per accedere al dato. Se il file è troppo lungo, il file indice stesso può risultare troppo grande, di conseguenza si crea un file indice anche per il file indice.

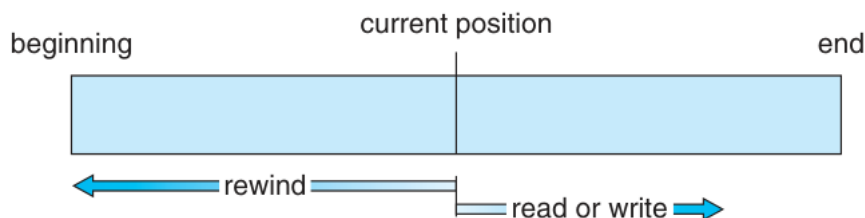


Figura 7.1: Accesso sequenziale

7.3 Struttura directory

Una directory deve essere implementata sfruttando una struttura che sia efficiente, cioè che permetta di reperire file rapidamente, deve permettere a file di utenti diversi di avere lo stesso nome, inoltre deve raggruppare i file sulla base di proprietà logiche (es: tutti i file binari).

La struttura di una directory è cambiata nel corso degli anni, la prima implementazione era la **directory monolivello**: esisteva un'unica directory per tutti gli utenti. Il problema era che tutti gli utenti dovevano avere file con nomi diversi, inoltre non esisteva nessun raggruppamento logico tra i file.

Successivamente si è passati alla **directory a due livelli**. Questa struttura ammetteva nomi uguali per file di utenti diversi e una ricerca efficiente, tuttavia il raggruppamento non

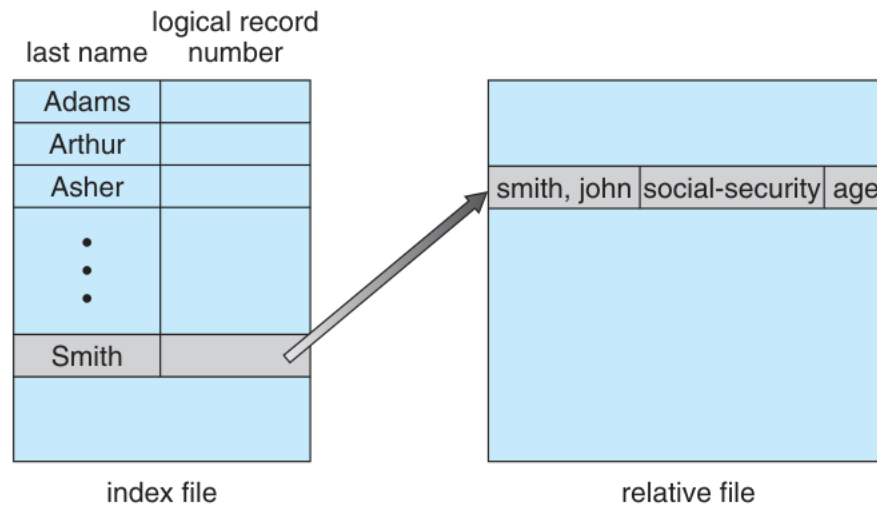


Figura 7.2: Accesso a indice

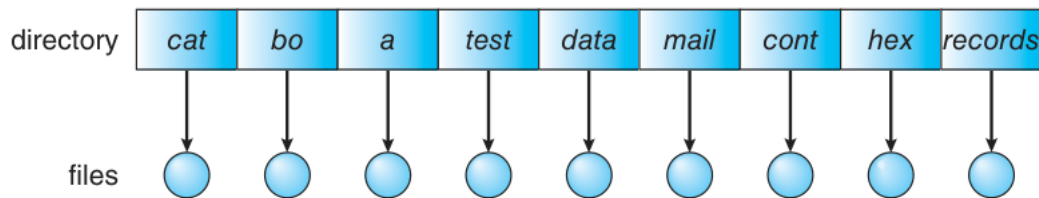


Figura 7.3: Directory monolivello

era ottimale (era solo in base al proprietario dei file), inoltre tutti gli utenti potevano accedere a qualsiasi file.

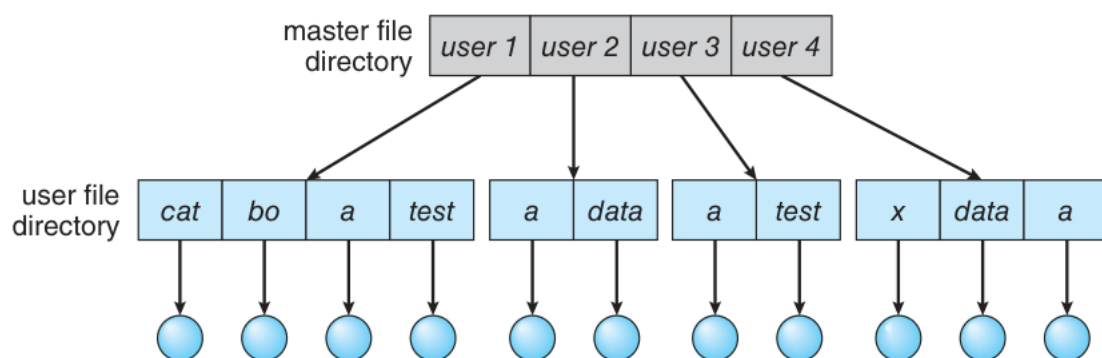


Figura 7.4: Directory a due livelli

Si è passati poi alla **directory con struttura ad albero**, questa struttura permetteva una ricerca efficiente, una buona capacità di raggruppamento e introduceva il concetto di *directory corrente*. Nasce quindi il concetto di percorso assoluto e relativo: la creazione o rimozione di un file viene effettuata nella cartella corrente, non serve più specificare il percorso assoluto (come accadeva nelle strutture precedenti).

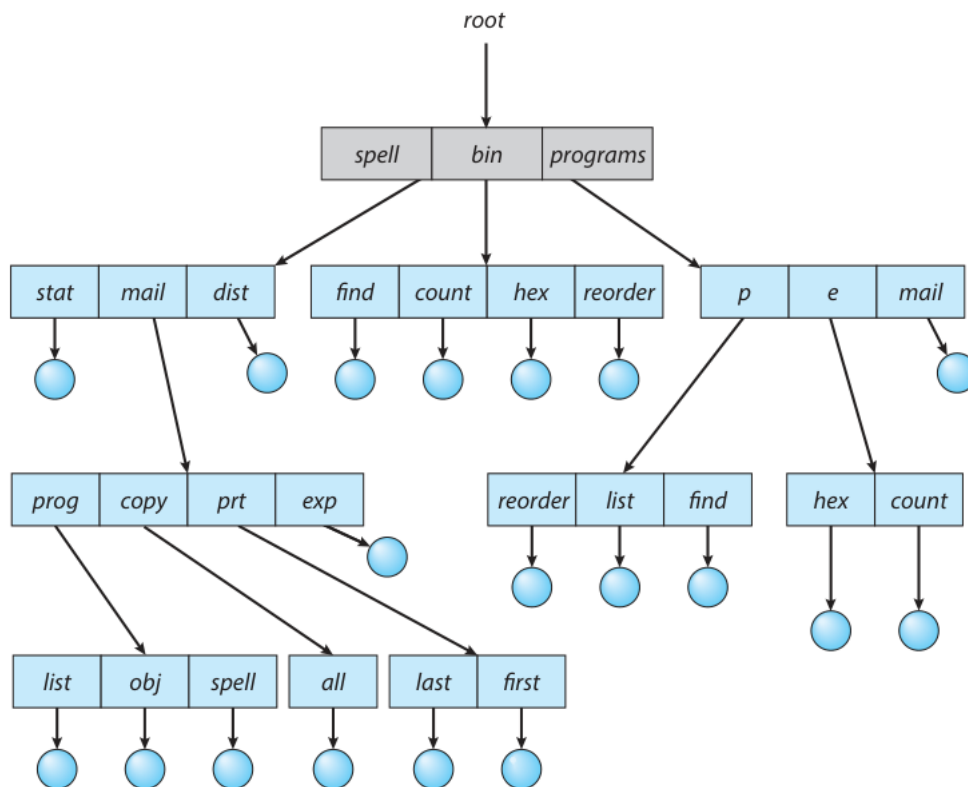


Figura 7.5: Directory ad albero

La struttura ad albero è stata migliorata, si è passati alla **directory a grafo aciclico**. Questa struttura ha introdotto il concetto di *link*: permettono a due o più nomi diversi di identificare lo stesso oggetto (file o directory). I link possono essere di due tipi:

- **soft link**: si tratta di un link simbolico, un puntatore che punta all'elemento nella directory, non al file fisico, non viene creato un elemento nella struttura della directory;
- **hard link**: si tratta di un link che punta allo stesso file fisico, quindi viene creato un nuovo elemento nella struttura della directory.

In questo tipo di struttura, per poter cancellare un file, bisogna adottare una strategia, per esempio: sfruttare puntatori all'indietro che permettano di reperire tutti i link al file cancellato e la loro eliminazione; conservare il file finché non esistono più link (contatore di riferimenti).

Mantenere il grafo aciclico garantisce la semplicità degli algoritmi di attraversamento del grafo (es: *DAG directed acyclic graph*), che possono essere utili, per esempio, in fase di backup. Ma come si può assicurare l'assenza di cicli? Un approccio è quello di permettere link a file ma non a sottodirectory. L'altro approccio è quello di verificare (con un algoritmo) la presenza di cicli, ogni volta che viene creato un nuovo link (molto dispendioso in quanto effettuato su memoria secondaria).

L'ultima variante che consideriamo è la **directory a grafo generale**. Questa struttura permette i cicli, quindi si possono creare link che puntano "all'indietro". Come abbiamo detto, la gestione di una struttura a grafo generale è più complicata e più dispendiosa (tuttavia viene utilizzata sui sistemi Linux).

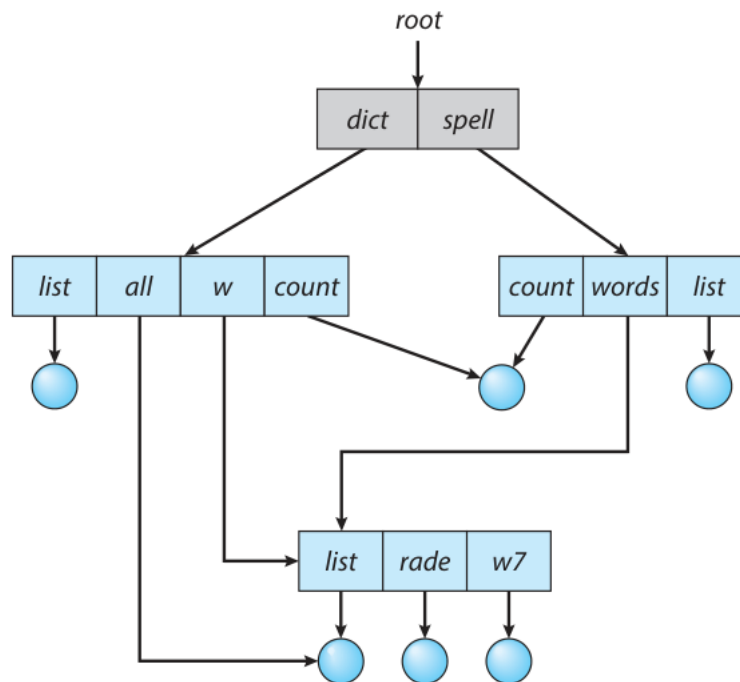


Figura 7.6: Directory ad albero aciclico

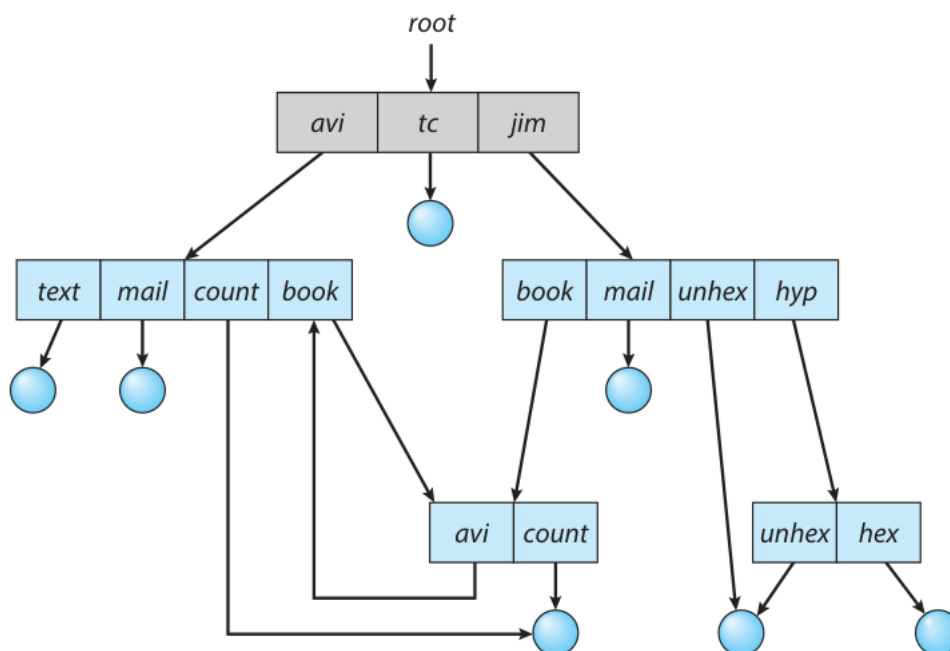


Figura 7.7: Directory ad albero ciclico

7.4 Operazione di montaggio

Per poter permettere ai processi di un sistema di accedere ad un dato file system, dobbiamo prima di tutto montarlo (*mount* in inglese). In generale un file system smontato (*unmounted*) può essere montato ad un mount point prescelto. Alcuni SO richiedono specifici file system, altri ne supportano diversi e sfondano le strutture del dispositivo per determinarne il tipo.

Per poter montare un file system, bisogna fornire al SO il nome del dispositivo da montare (sotto forma di volume e pathname) e la locazione che dovrà assumere nella struttura del file system (solitamente un directory vuota), questa locazione prende il nome di punto di mount. Una volta montato il file system, diventa parte integrante del grafo delle directory e programmi e utenti possono accedervi in maniera trasparente.

Nei sistemi MAC e Windows, il rilevamento di tutti i dispositivi e il montaggio dei relativi file system, viene effettuato in fase di boot. Nei sistemi UNIX-like, i file system montati in fase di boot devono essere specificati nel file `/etc/fstab`.

7.5 Struttura file system

Il file system è stratificato, è organizzato a livelli. Ogni livello si serve delle funzioni dei livelli inferiori per crearne di nuove, impiegate dai livelli superiori.

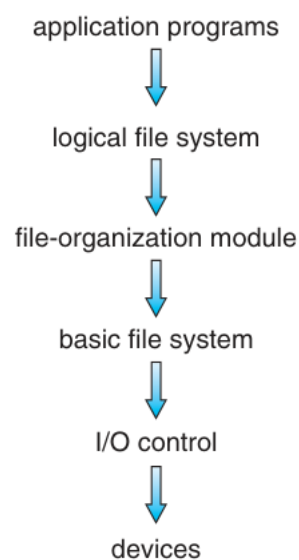


Figura 7.8: Layered file system

- I/O control: scrivono specifiche configurazioni di bit in specifiche locazioni della memoria del driver per indicare al dispositivo I/O che operazioni compiere e dove. Le configurazioni di bit sono il risultato della traduzione di comandi di alto livello (es: "leggi dal disco 1, cilindro 72, traccia 2, settore 10 nella locazione di memoria 1060");
- basic file system: invia comandi generici al driver del dispositivo per leggere/scrivere blocchi fisici su disco (es: "recupera il blocco 123");
- file-organization module: traduce gli indirizzi logici di blocco in indirizzi fisici e contiene il modulo per la gestione dello spazio libero;
- logical file system: gestisce i metadati (informazioni sul file) dei file, cioè tutte le strutture del file system, eccetto i dati veri e propri memorizzati nel file;
- application programs: forniscono all'utente un accesso semplificato al file.

In generale, le strutture a strati, sono utili per ridurre la complessità e la ridondanza; tuttavia aggiungono un overhead che può diminuire le prestazioni.

7.6 Metodi di allocazione dei file

L'obiettivo è quello di allocare i file su disco in modo tale da ridurre al minimo lo spreco di spazio e di mantenere rapido l'accesso. I metodi di allocazione dei file descrivono come i blocchi fisici del disco vengono allocati ai file.

Allocazione contigua I blocchi vengono allocati in maniera contigua su disco. Per reperire un file occorre solo la locazione iniziale (numero blocco iniziale) e la lunghezza (numero di blocchi): le prestazioni sono ottime. Il problema principale è che i file non possono crescere di dimensione, non possono superare la dimensione dei blocchi assegnati. Si potrebbero adottare delle soluzioni per risolvere tale problema ma in ogni caso risulterebbe un aumento della frammentazione esterna (quindi a spreco di spazio).

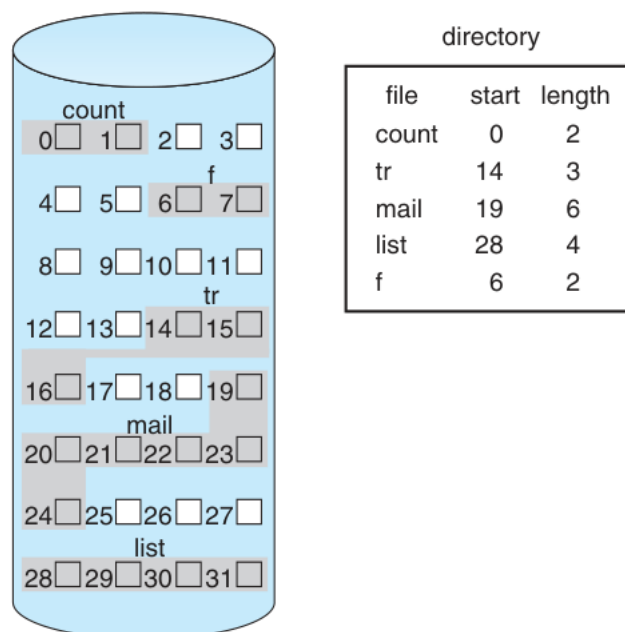


Figura 7.9: Allocazione contigua

Allocazione concatenata Ciascun file è una lista concatenata di blocchi: i blocchi possono essere sparsi ovunque nel disco. Ciascun blocco contiene un puntatore al blocco successivo; il file termina quando si incontra un blocco con puntatore vuoto. Quando un file aumenta di dimensione basta allocare un nuovo blocco e collegarlo all'ultimo, in questo modo si risolve il problema della frammentazione esterna. Uno dei problemi è che se uno dei blocchi si corrompe, l'intero resto del file è perduto.

File Allocation Table (FAT) I puntatori che abbiamo descritto nell'allocazione concatenata, vengono inseriti immagazzinati in celle di memoria contigua (una tabella). La FAT ha un elemento per ogni blocco del disco ed è indicizzata dal numero di blocco. Per accedere al contenuto del file: si parte dall'elemento di directory, questo contiene il numero del primo blocco del file; l'elemento della FAT indicizzato a quel blocco contiene a sua volta il numero del blocco successivo del file; l'ultimo blocco ha come elemento della tabella un valore speciale di fine file. Per velocizzare le prestazioni la tabella viene mantenuta in memoria.

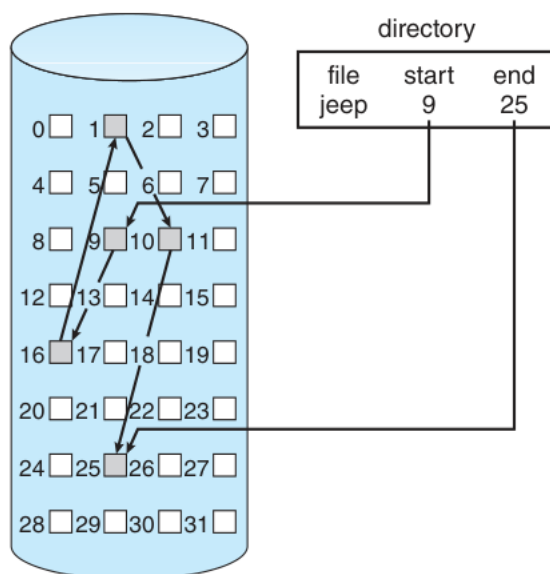


Figura 7.10: Allocazione concatenata

I file system FAT sono nominati FAT-X, dove X è il numero di bit utilizzati per esprimere l'indice di blocco, tale numero influenza la dimensione massima che può avere la partizione ($= 2^X$ blocchi).

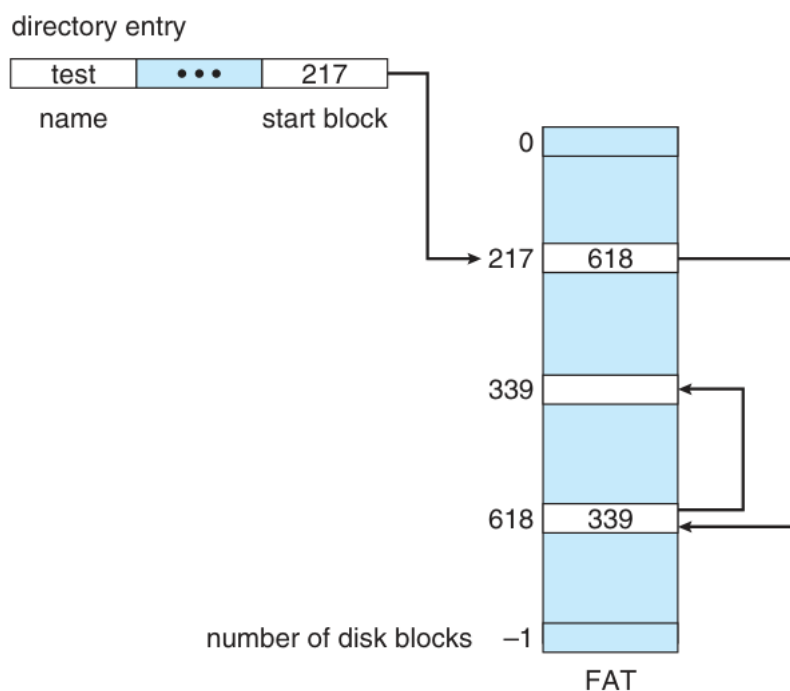


Figura 7.11: File-allocation table

Allocazione indicizzata Per ogni file, vengono collezionati tutti i puntatori in un unico blocco indice. Sussiste ancora il problema della dimensione massima del file: il blocco indice ha dimensione fissa. Per risolvere tale problema si può sfruttare uno schema concatenato:

si collegano blocchi della tabella indice; il primo blocco contiene l'insieme degli indirizzi dei primi $n - 1$ blocchi del file più un puntatore al blocco indice successivo.

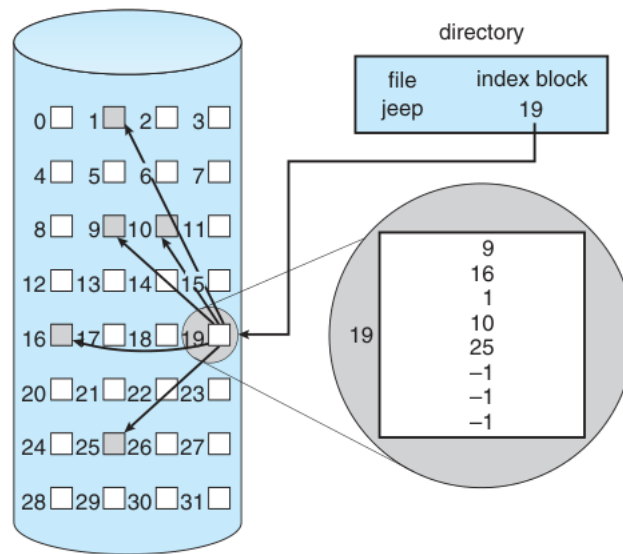


Figura 7.12: Allocazione indicizzata blocco contenente indici

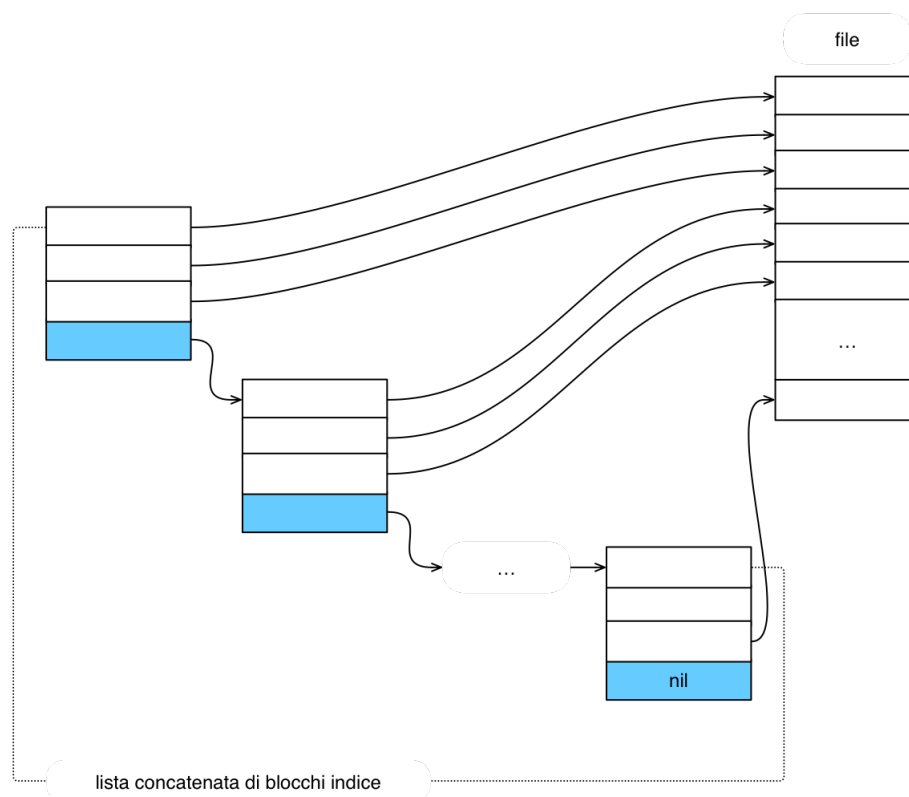


Figura 7.13: Allocazione indicizzata struttura concatenata

Indice multilivello Funziona in maniera simile alla paginazione multilivello: si creano più tabelle di indici alle quali accedere prima di arrivare ai blocchi dati. Il problema è l'alto numero di accessi in memoria.

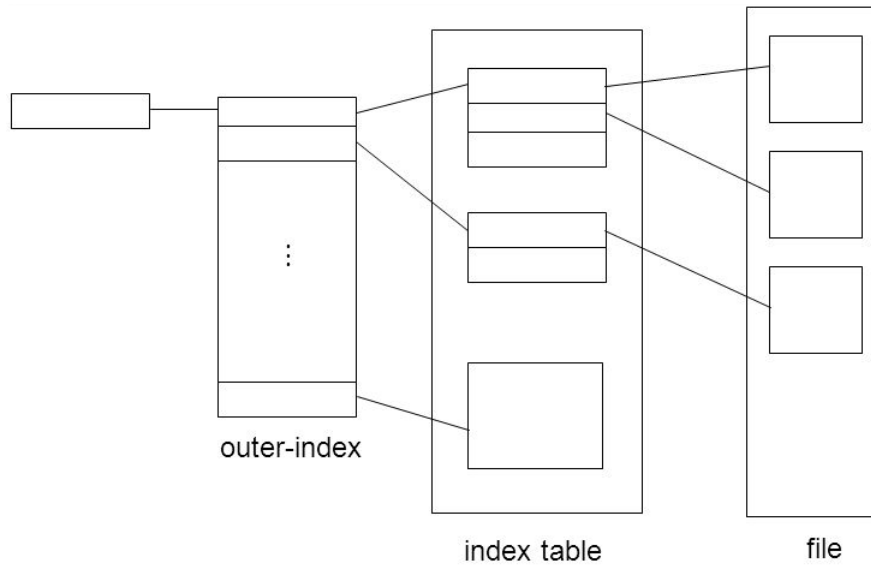


Figura 7.14: Allocazione indicizzata a indice multilivello

Schema combinato Soluzione che trae il meglio da quelle precedenti. L'inode del file mantiene le seguenti informazioni riguardanti i blocchi dati del file:

- i primi 10 indirizzi riferiscono blocchi di dati (indirizzamento diretto);
- l'undicesimo indirizzo punta a un blocco che punta a sua volta a dei blocchi di dati (primo livello di indirettezza);
- dal dodicesimo indirizzo si realizza il secondo livello di indirettezza;
- dal tredicesimo indirizzo si realizza il terzo livello di indirettezza.

Se, per esempio, poniamo: dimensione blocco = $512B$ e indirizzi di $32b = 4B$; quindi un blocco può contenere $512/4 = 128$ indirizzi. Si ottiene che:

- primi 10 blocchi di dati sono accessibili direttamente $\rightarrow$ file di dimensioni minori di $10 * 512B = 5KB$ sono accessibili direttamente;
- 128 blocchi di dati sono accessibili con indirettezza singola (mediante il puntatore 11) $\rightarrow 128 * 512B = 64KB$;
- $128 * 128$ blocchi di dati sono accessibili con indirettezza doppia (mediante il puntatore 12) $\rightarrow 128 * 128 * 512B = 8MB$;
- $128 * 128 * 128$ blocchi di dati sono accessibili con indirettezza tripla (mediante il puntatore 13) $\rightarrow 128 * 128 * 128 * 512B = 1GB$.

Sommando le dimensioni si ottiene a massima dimensione di un file consentita.

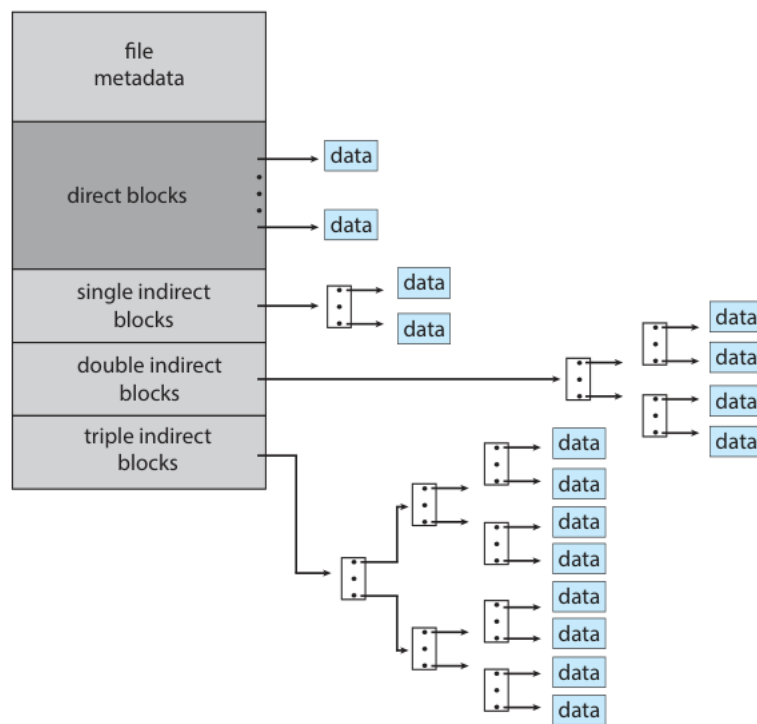


Figura 7.15: Schema combinato

Capitolo 8

Gestione memoria secondaria

8.1 Generalità sui dischi

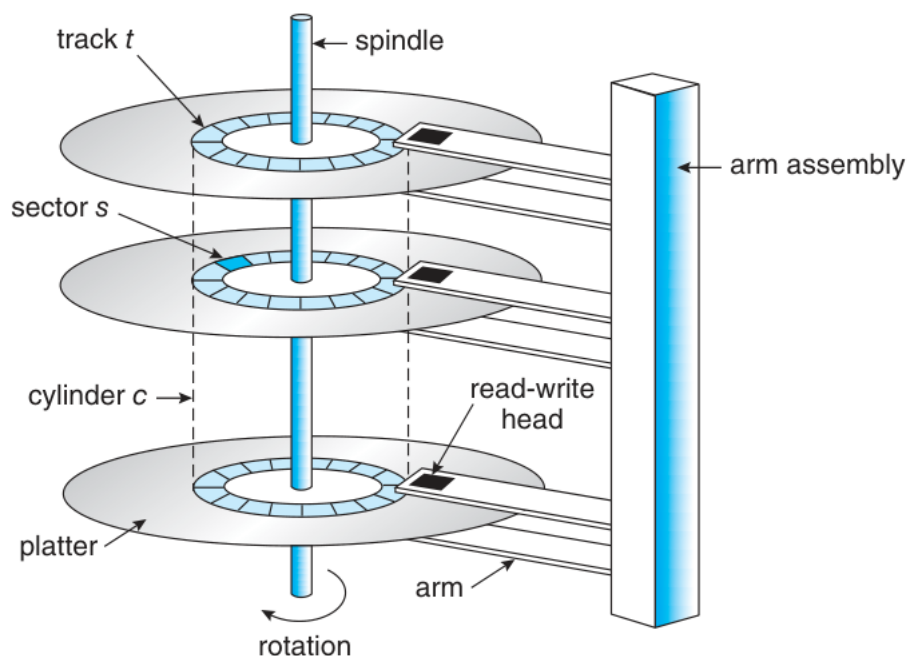


Figura 8.1: Meccanismo di movimento della testa negli HDD

In un HDD individuiamo le seguenti parti:

- piatto: dischi paralleli, identificato da un numero univoco e destinato alla memorizzazione dei dati;
- traccia: anello concentrico presente sul piatto, identificate da un numero univoco;
- cilindro: insieme di tracce poste alla stessa distanza dal centro e relative a tutti i dischi, corrisponde a tutte le tracce con lo stesso numero, ma giacenti su piatti diversi;
- settore: ogni traccia è suddivisa in settori circolari, in "spicchi" uguali identificati da un numero;
- blocco: insieme di settori posti nella stessa posizione in tutti i piatti;

- testina: su ogni piatto è presente una testina di lettura/scrittura. Se una testina è posizionata sopra una traccia, tutte le altre saranno posizionate sul cilindro a cui la traccia appartiene.

Il tempo di accesso può essere diviso in due componenti principali:

- tempo di ricerca (seek time): tempo impiegato per spostare la testina sul cilindro che contiene il settore desiderato;
- latenza di rotazione (rotational latency): tempo necessario perché il disco ruoti fino a portare il settore desiderato sotto la testina.

Quindi il tempo di accesso medio è dato dalla somma tra il seek time medio più la latenza media, mentre il tempo medio di I/O è dato dal tempo medio di accesso più la quantità di dati da trasferire diviso la velocità di trasferimento, più un eventuale overhead (i settori danneggiati vengono esclusi). Il mapping fra indirizzi logici e fisici si realizza in maniera diretta.

Per esempio: poniamo di voler trasferire un blocco da $4KB$ su un disco con una velocità di rotazione di $7200RPM$, tempo medio di ricerca pari a $5ms$, velocità di trasferimento di $1Gb/s$ e con un overhead dovuto al controllore di $0.1ms$. Si ottiene che

$$\text{Tempo di trasferimento} = \frac{4 \cdot 1024 \cdot 8}{10^9} = 0.031ms$$

$$\text{Latenza media} = \frac{1000/(7200/60)}{2}ms$$

$$\text{Tempo medio di I/O per un blocco da } 4KB = 5 + \frac{1000/(7200/60)}{2} + 0.1 + 0.031 = 9.301ms$$

Esistono due modalità in cui avviene la scrittura e la lettura:

- *Constant Linear Velocity*: tracce lontane dal centro sono più lunghe e contengono un maggior numero di settori. La velocità di rotazione aumenta spostandosi verso l'interno ($v = \omega r$), in questo modo la velocità lineare rimane costante e di conseguenza rimane costante la quantità di dati che passano sotto le testine nell'unità di tempo. Talvolta si ha un'unica traccia a spirale.
- *Constant Angular Velocity*: la velocità di rotazione viene mantenuta costante. La densità di bit decresce dalle tracce più interne alle tracce più esterne per mantenere costante la quantità di dati che passano sotto le testine nell'unità di tempo. La velocità lineare aumenta man mano che ci si allontana dal centro.

8.2 Lo scheduling del disco

Il sistema operativo è responsabile dell'uso efficiente del dell'hardware. Nel caso dei dischi, ciò significa garantire tempi di accesso contenuti e ampiezze di banda elevate. L'ampiezza di banda è data dal numero totale di byte trasferiti diviso il tempo trascorso fra la prima richiesta e il completamento dell'ultimo byte trasferito. Per quanto riguarda il tempo di accesso, abbiamo visto che è dato dalla somma tra tempo di ricerca e latenza di rotazione: per migliorare le prestazioni possiamo intervenire soltanto sul tempo di ricerca.

In generale, quando un processo (utente o di sistema) deve effettuare un'operazione di I/O relativa ad un'unità a disco, effettua una chiamata al SO in cui specifica: il tipo di operazione (immissione/emissione di dati); l'indirizzo su disco relativamente al quale effettuare il trasferimento; l'indirizzo nella memoria relativamente al quale effettuare il trasferimento; il numero di byte da trasferire. Tale richiesta può essere gestita immediatamente se l'unità

a disco rigido e il driver sono disponibili, altrimenti la richiesta viene aggiunta in coda alle richieste per quell'unità. Il SO ha il compito di scegliere quale delle richieste in attesa servire per prima, per la scelta si basa su un **algoritmo di scheduling del disco**. Come gli algoritmi di scheduling per i processi, anche questi vengono testati su una stringa di riferimenti. Analizziamo alcuni algoritmi testandoli sulla coda di richieste 98, 183, 37, 122, 14, 124, 65, 67 per i cilindri 0 – 199 e posizionando inizialmente la testina sul cilindro 53 (ricordiamo che il cilindro 0 è quello più esterno).

8.2.1 FCFS

Produce un movimento totale della testina pari a 640 cilindri. Può produrre grandi oscillazioni.

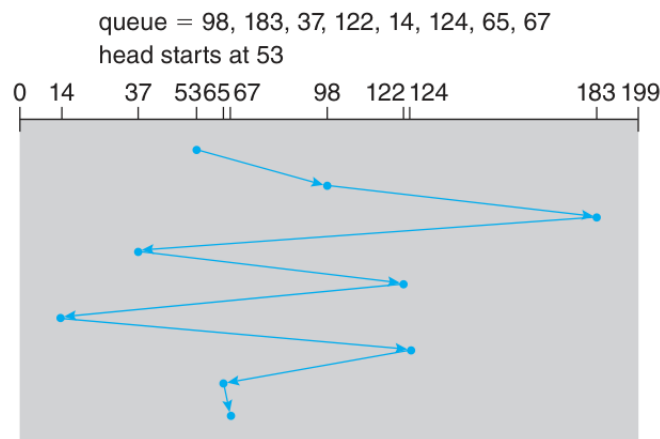


Figura 8.2: FCFS disk scheduling

8.2.2 Shortest seek time first (SSTF)

Seleziona la richiesta di accesso con seek time minimo in relazione alla posizione corrente della testina. Si tratta dell'analogo del SJF per i processi; può causare l'attesa indefinita di alcune richieste. Provoca un movimento di 236 cilindri.

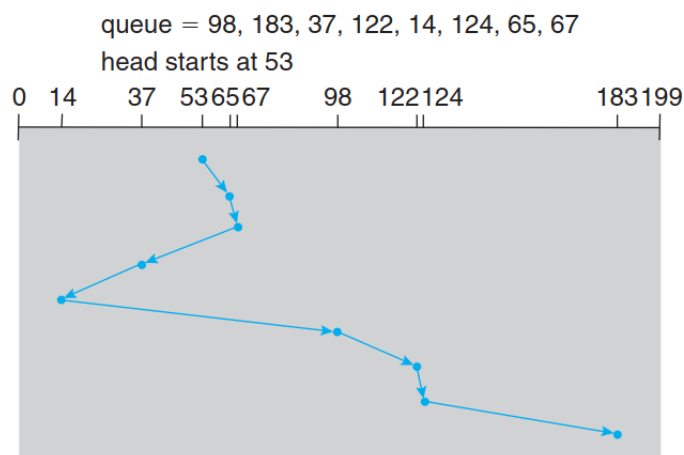


Figura 8.3: SSTF disk scheduling

8.2.3 Scan

Viene anche chiamato algoritmo dell'ascensore. Sostanzialmente, il braccio della testina, si muove da un estremo all'altro del disco, servendo sequenzialmente le richieste; giunto ad un estremo inverte l'ordine di direzione di marcia e, conseguentemente, l'ordine di servizio. Provoca un movimento di 236 cilindri.

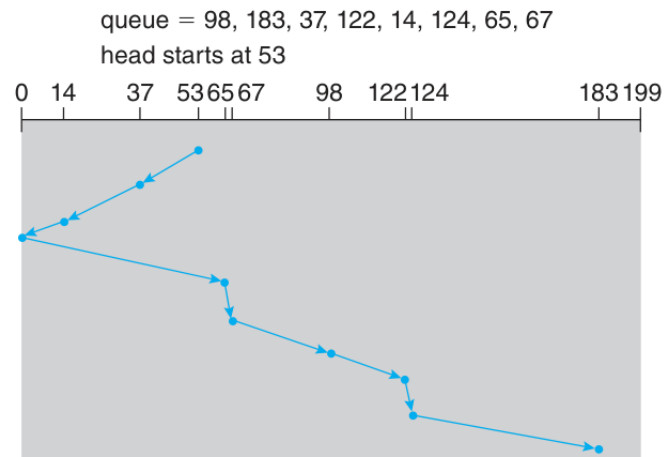


Figura 8.4: Scan disk scheduling

8.2.4 C-Scan

Garantisce un tempo di attesa più uniforme rispetto a Scan: la testina si muove sempre da un estremo all'altro del disco servendo sequenzialmente le richieste; quando raggiunge l'ultimo cilindro ritorna immediatamente all'inizio del disco senza servire richieste durante il viaggio di ritorno. Sostanzialmente considera i cilindri come organizzati secondo una lista circolare. Provoca un movimento di totale di 383 cilindri.

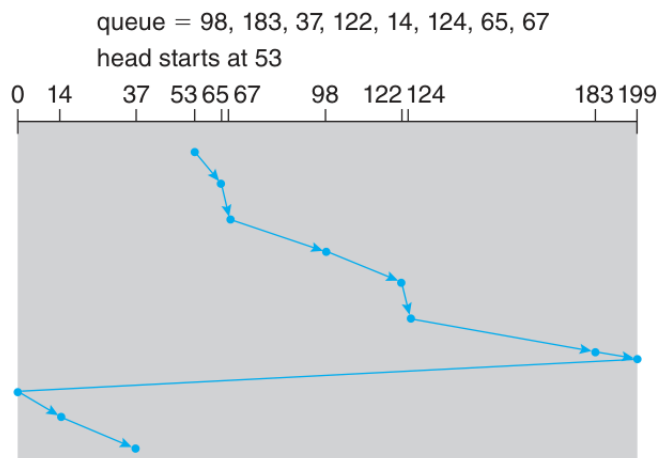


Figura 8.5: C-Scan disk scheduling

8.2.5 C-Look

Si tratta di una versione ottimizzata di C-Scan: il braccio serve l'ultima richiesta in una direzione e poi inverte il movimento senza arrivare al termine del disco. Produce un movimento di 353 cilindri.

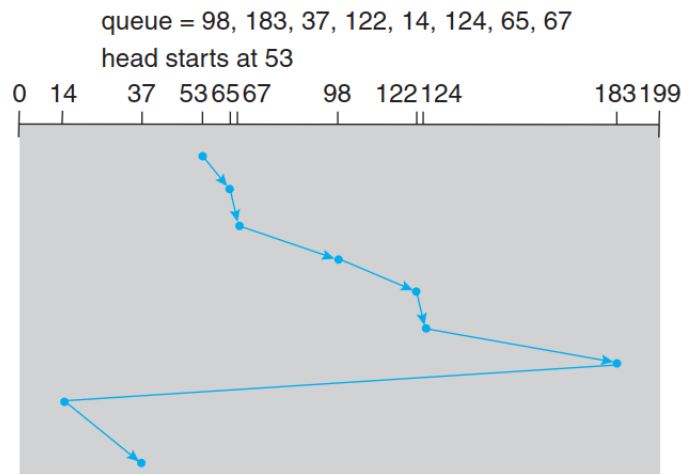


Figura 8.6: C-Look disk scheduling

8.3 La formattazione

La formattazione è divisa in due tipologie:

- formattazione di basso livello o *fisica*: il disco viene suddiviso in settori che possono essere letti e scritti dal controllore del disco. All'inizio di ogni settore (in testa) viene sistemato il suo numero, mentre alla fine (in coda) viene messo un codice di correzione (es: codice di Hamming). Tale formattazione viene effettuata dalle case produttrici;
- formattazione *logica*: viene effettuata dall'utente. Il disco viene partizionato in uno o più gruppi di cilindri, ognuno dei quali rappresenta un "disco logico". La formattazione implica la creazione di un file system, in questo modo il SO può interagire con il dispositivo.

Può accadere che i settori fisici risultino difettosi e quindi inutilizzabili. Per far fronte a tale inconveniente, le case produttrici, durante la formattazione fisica, mantengono un gruppo di settori di riserva non visibili al SO. Quando un settore risulta danneggiato, il driver del disco è istruito per sostituire, dal punto di vista logico, il settore con uno tra quelli di riserva.

8.3.1 Gestione degli errori

Le memorie elettroniche o magnetiche, memorizzano bit utilizzando meccanismi che possono occasionalmente generare errori, per esempio può accadere che un bit settato a 1 venga poi letto uguale a 0, o viceversa. Formalizziamo il concetto di errore in una codifica a n bit: definiamo C la codifica corretta; C' la codifica errata; definiamo **distanza di Hamming** tra le codifiche $H(C, C')$. La distanza di Hamming indica il numero di cifre binarie differenti a parità di posizione: se vale 0, allora significa che le codifiche sono uguali; se vale 1 significa che differiscono per un bit; se vale 2 differiscono per due bit; etc; Per scoprire gli errori singoli (cioè se $H(C, C') = 1$) viene aggiunto un **bit di parità**. Tale bit viene settato a 1 affinché il

numero di 1 presenti nella codifica sia pari. Se si verifica un errore, il numero di bit di valore uno in C' non sarà più pari. Con un singolo bit di parità riusciamo a scoprire un singolo errore nella codifica.

8.4 Il bootstrap

Il *bootstrap* indica quella situazione in cui un processo lancia un processo che a sua volta lancia altri processi. Durante la fase di avvio, il primo bootstrap che viene lanciato dalla macchina è a carico del BIOS (Basic Input Output System), il quale lancia il processo `init`, il quale si occuperà di lanciare tutti gli altri processi.

Il BIOS è un piccolo chip installato sulla ROM (Read Only Memory), la CPU sa dove si trova la ROM e riesce quindi a rintracciare il BIOS. Il compito principale del BIOS è quello di eseguire una serie di operazioni (dette *POST Power On Self Test*) che permettono alla macchina di riconoscere le periferiche. Una volta completato il POST, il controllo del sistema passa dal BIOS al bootloader.

Il bootloader risiede nel primo settore dell'hard disk, conosciuto anche come Master Boot Record (MBR): all'interno si trova le informazioni sul bootloader (es di bootloader: GRUB). Il bootloader, in Linux, è responsabile del caricamento dell'immagine del kernel e del disco RAM iniziale (che contiene alcuni file critici e i driver dei dispositivi necessari per avviare il sistema) in memoria. Il bootloader riconosce i SO installati sul nostro disco (riconosce le boot partition) e ci fa scegliere quale vogliamo avviare.

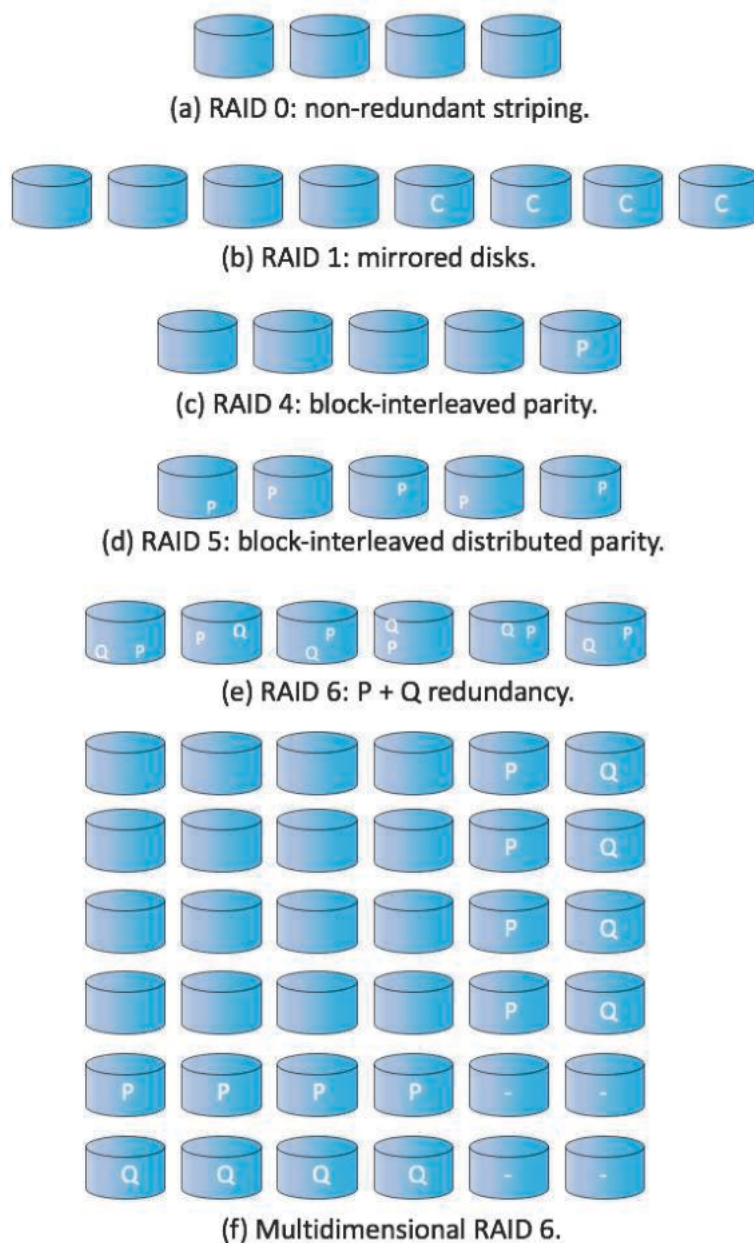
8.5 RAID

RAID sta per *Redundant Array of Independent Disk*, si tratta di una serie di tecniche che mirano a garantire l'affidabilità dei dati salvati su disco sfruttando la ridondanza. I RAID consistono di una serie di hard disk che comunicanti che vanno a formare un unico RAID Group. Esistono due macro tipologie di RAID:

- mirroring: si tratta di un sistema che copia a "specchio" su un hard disk parallelo al nostro disco dati. Si tratta di un sistema con granularità grossa dei dati. Più file possono essere letti contemporaneamente dai due dischi quindi la lettura è il doppio più veloce, mentre la scrittura deve essere effettuata su entrambi i dischi e di conseguenza la scrittura rimane costante, non migliora ne peggiora. In questo caso c'è rindondanza dei dati. Con questa configurazione, la memoria fisica utilizzabile è la metà di quella fisica totale (un disco è la copia dell'altro).
- striping: i dati vengono vengono divisi su dischi diversi, si parla di granularità fine. I dati possono essere divisi in blocchi o byte. Lettura e scrittura di un singolo file possono avvenire contemporaneamente. In questo caso non c'è rindondanza, quindi se si rompe un disco perdiamo dati.

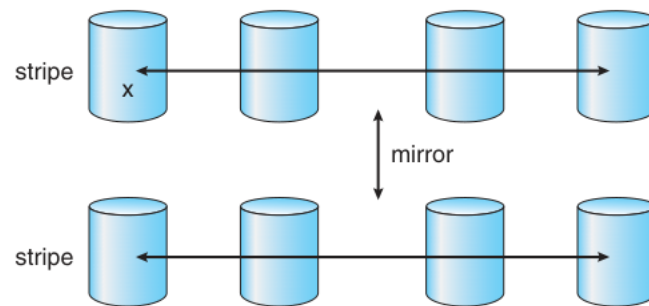
Il mirroring garantisce affidabilità ma è costoso, lo striping fornisce alti data-transfer rates ma è poco sicuro. Per raggiungere un compromesso sono nate delle configurazioni classificate in livelli, analizziamone alcune.

- RAID level 0: non è una vera architettura RAID, infatti non mantiene rindondanza e consente solo data striping, a livello di blocco o multipli di blocco. Presenta il minor costo tra le architetture, ha le migliori prestazioni in scrittura (non in lettura) e l'affidabilità è bassa.
- RAID level 1: i dati sono mantenuti in duplice copia su dischi distinti, tutti i dischi sono duplicati.

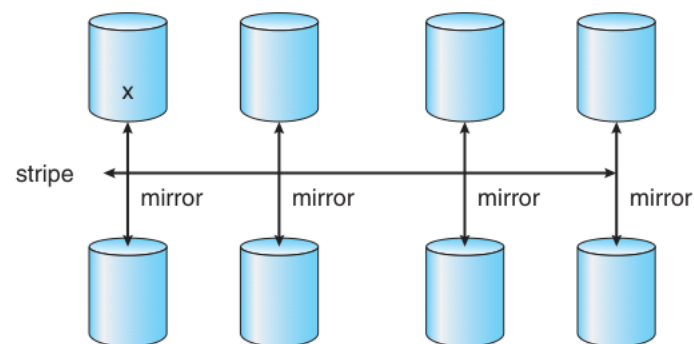


- RAID level 2: alcuni dischi vengono utilizzati per memorizzare i bit di parità, viene sfruttato lo striping a livello di bit. Un unico disco di parità può rivelare un singolo errore, ma per correggere un errore sono necessari più dischi di parità per identificare il disco con l'errore.
- RAID level 0+1: stripe of mirrors, i dati sono distribuiti su insiemi di dischi in duplice copia.
- RAID level 1+0: mirroring of stripes, i dati sono distribuiti su un array di dischi che viene completamente duplicato su uno o più array di dischi.
- RAID level 3: striping a livello di bit, unico movimento parallelo delle testine, viene utilizzato un unico disco di parità. Questo sistema del tutto simile al livello 2 ma nell'ultimo disco per i bit di parità.

- RAID level 4: striping a livello di blocco, testine indipendenti, usa un singolo disco di parità. Questo sistema riserva al blocco dell'ultimo disco la funzione di blocco di parità.
- RAID level 5: striping a livello di blocco e parità distribuita fra tutti i dischi; testine indipendenti.
- RAID level 6: striping a livello di blocco e parità distribuita su tutti i dischi; testine indipendenti. Tollerare fino a due guasti e utilizza codici di Reed-Solomon per le correzioni degli errori.



a) RAID 0 + 1 with a single disk failure.



b) RAID 1 + 0 with a single disk failure.

8.6 Gestione dei blocchi liberi

Per tenere traccia dello spazio libero in un disco, il sistema conserva una lista dello spazio libero. Ogni volta che viene creato un file, bisogna innanzitutto cercare nella lista dello spazio libero la quantità di spazio necessaria; successivamente bisogna aggiornare il contenuto della lista. Mentre quando si cancella un file, bisogna aggiungere alla lista dello spazio libero i blocchi precedentemente assegnati al file.

La lista dello spazio libero non è per forza di cose una lista, può essere realizzata mediante altre strutture. Vediamo alcune soluzioni.

Vettore di bit o bitmap Abbiamo un vettore di bit, ogni bit è riservato ad un blocco, quindi il vettore è lungo quanto il numero di blocchi del disco. Questa memorizzazione permette di individuare le zone di *extent*, cioè le zone del disco che hanno più blocchi consecutivi liberi. Se il vettore di bit è memorizzato in memoria centrale, si ottengono buone prestazioni. Se per esempio abbiamo un disco da $1TB = 2^{40}B$ e la dimensione di un blocco è di $4kB = 2^{12}B$, allora la dimensione del vettore è di $n = 2^{40}/2^{12} = 2^{28}$ blocchi $\Rightarrow 2^{28}bit = 32MB$.

001111001111110001100000011100000...

$$\text{bit}[i] = \begin{cases} 1 & \text{se blocco}[i] \text{ è libero} \\ 0 & \text{se blocco}[i] \text{ è occupato} \end{cases}$$

Lista concatenata Si collegano tutti i blocchi liberi mediante puntatori e si mantiene un puntatore alla testa della lista in memoria centrale. Ogni blocco contiene un puntatore al blocco successivo blocco libero. Questa soluzione non è facile da utilizzare per ottenere spazio contiguo.

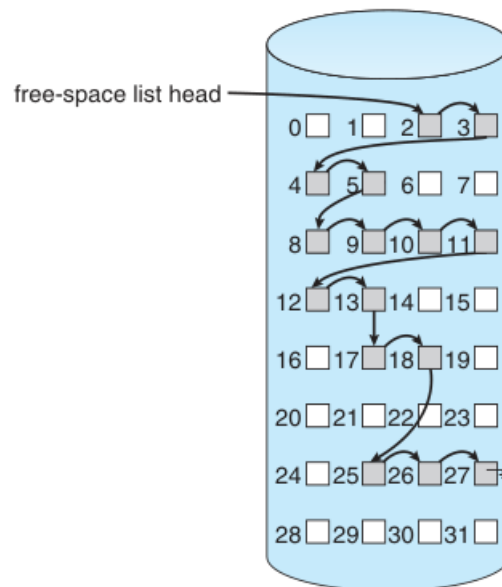


Figura 8.7: Linked free-space list nel disco

Indice dei blocchi liberi Si mantiene una lista contenente un indirizzo del disco ed un contatore, che indicano rispettivamente un blocco libero e da quanti altri blocchi liberi (contigui) è seguito.

Capitolo 9

Affidabilità dei dispositivi

9.1 Definizioni e formule

Definizione 9.1 (Affidabilità di un dispositivo). *Probabilità che un sistema funzioni correttamente nell'intervallo $(0, t)$.*

$$R(t) = \frac{n(t)}{n(t_0)}$$

dove $n(t)$ indica il numero di dispositivi funzionanti al tempo t .

Definizione 9.2 (Probabilità di guasto). *Probabilità che un sistema si guasti per la prima volta nell'intervallo di $(0, t)$.*

$$Q(t) = 1 - R(t) = 1 - \frac{n(t)}{n(t_0)}$$

Definizione 9.3 (Disponibilità). *La frazione di tempo in cui il dispositivo funziona correttamente*

$$A = \frac{MTTF}{MTTF + MTTR}$$

dove: $MTTF$ è il Mean Time To Failure; $MTTR$ è il Mean Time To Repair.

Capitolo 10

Introduzione a Go

Go è un linguaggio di programmazione sviluppato da Google. Rende semplice ed efficace la gestione della concorrenza.

Vediamo un primo esempio di programma:

```
1 package main
2 import "fmt"
3
4 func main() {
5     fmt.Print("Hello World")
6 }
```

10.1 Variabili

```
1 var x int // dichiarazione
2 var z int = 43 // definizione
3 y := 34 // il compilatore deduce il tipo
4 const Pi = 3.14 // costante
5 x, y, z := 23, 3, "bar"
```

10.1.1 Puntatori

I puntatori si utilizzano pressoché come in C++.

```
1 a := 4
2 var p *int = &a // puntatore ad a
3 ms := myStruct{foo: 34}
4 p := &ms // automaticamente dereferenziato
5 p := &myStruct{foo: 34} // diversamente da c++ e' possibile puntare ad un
6 indirizzo di memoria temporaneo
p := new(myStruct) // non e' possibile inizializzarlo
```

10.2 Slices

Gli slices sono sequenze di elementi dello stesso tipo, assomigliano ai `std::vector` di C++. Sono sostanzialmente array dinamici.

```

1  sl := []int { 1, 2, 3, 4, 5 } // definisco uno slice
2  sl := make([]int, 10) // slice con lunghezza e capacita' 10
3  sl := make([]int, 10, 100) // slice con lunghezza 10 e capacita' 100
4  fmt.Println("l'intero slice", sl) // => [1, 2, 3, 4, 5]
5  fmt.Println("l'intero slice", sl[:]) // => [1, 2, 3, 4, 5]
6  fmt.Println("una parte dello slice", sl[1:3]) // => [2, 3]
7  fmt.Println("lunghezza", len(sl))
8  fmt.Println("capacita'", cap(sl))
9  sl = append(sl, 5, 33) // aggiungo due elementi allo slice
10 sl = append(sl, a[:]...) // appendo a sl tutti gli elementi a
11 b := sl // sl e b puntano allo stesso array
12 for i := range sl { fmt.Print(sl[i], ", ") } // foreach
13 for i, x := range sl { fmt.Print("element =", x, "index =", i) } // foreach

```

Gli array "stile C" presentano delle differenze:

```

1  a := [3]int { 1, 2, 3 } // definisco array
2  a := [...]int { 1, 2, 3 } // altro modo di definire
3  var a [3] int // array inizializzato di default
4  b := a // a array => b e' una copia di a

```

10.3 Funzioni

```

1  func maxmin(x float64, y float64) (float64, float64) {
2      max := math.Max(x, y)
3      min := math.Min(x, y)
4      return max, min
5  }
6
7  func useless(a, b, c, d int) int {
8      return a + b + c + d
9  }
10
11 func passSlice(bar string, baz ...int) {
12     fmt.Println(bar)
13 }
14
15 func foo(a, b int) (int, error) { // pattern molto comune
16     if b == 0 { return 0, fmt.Errorf("errore") }
17     return a/b, nil
18 }
19
20 func main() {
21     max, min := maxmin(13.4, 44,5)
22     fmt.Println("max =", max, "min =", min)
23
24     passSlice("bar", 1, 2, 3, 4)
25
26     r, err := foo(1, 0)
27     if err := nil {
28         fmt.Println(err)
29         return // quit
30     }

```

```

31
32     f := func(s string) {
33         fmt.Println(s)
34     }
35     f("Hello")
36 }

```

In Go è possibile tornare un indirizzo di una variabile dichiarata all'interno di una chiamata a funzione. Tale variabile viene promossa dallo stack allo heap.

10.3.1 Metodi

```

1     type foo struct {
2         a int
3         s string
4     }
5
6     func (f foo) bar() { // metodo con passaggio per copia
7         fmt.Println(f.a, f.s) // => 12 hello
8     }
9
10    func (f *foo) baz() { // metodo con passaggio per puntatore
11        f.a = 5 // dereferenziato automaticamente
12        fmt.Println(f.a, f.s) // => 5 hello
13    }
14
15    func main() {
16        f := foo{12, "hello"}
17        f.bar()
18        f.baz()
19    }

```

10.4 Strutture

```

1     type Coordinate struct {
2         x int
3         y int
4     }
5
6     func changeX(c *Coordinate, x int) { c.x = x }
7
8     func main() {
9         c := Coordinate{2, 34}
10        d := Coordinate { // inizializzazione alternativa
11            x: 3,
12            y: 5,
13        }
14        changeX(&c, 5)
15        fmt.Println(c.x) // => 5
16        fmt.Println(c) // => {5 34}
17    }

```

In Go non esiste l'ereditarietà, ma esiste il rapporto "has a", per esempio:

```

1  type Shape struct {
2      name string
3      area float32
4  }
5
6  type Circle struct {
7      Shape // has a Shape
8      r float32
9  }
10
11 func main() {
12     c := Circle {
13         r: 4,
14         Shape: Shape{name:"circle", area: 3.14 * 4 * 4},
15     }
16     fmt.Println("area of", c.type, " =", c.area)
17 }

```

10.5 Defer e Panic

La keyword `defer` posticipa l'esecuzione di un'istruzione alla fine del codice. Utili per esempio per chiudere risorse acquisite.

```

1  func main() {
2      defer fmt.Println(1)
3      defer fmt.Println(2)
4      fmt.Println(3)
5      // => 3 => 2 => 1
6      // LIFO order
7
8      a := "start"
9      defer fmt.Println(a)
10     a := "end"
11     // => start
12     // viene stampato 'start' perche' viene salvato il valore di 'a'
13     // nell'istante in cui uso 'defer'
14 }

```

In Go non esistono le eccezioni, solitamente si utilizzano gli *error* che possono essere gestiti dal programmatore. Lo strumento utilizzato per gli la gestione degli errori runtime che terminano il programma, sono i **panic**. I panic sono molto simili alle eccezioni.

```

1  // esempio 1
2  func main() {
3      fmt.Print("start")
4      defer fmt.Println("after panic") // viene eseguita
5      panic("error")
6      fmt.Print("end") // non viene eseguita
7      // => start after panic panic: error
8  }
9
10 // esempio 2: recover from panic

```

```

11 func main() {
12     a, b := 1, 0
13     defer func() {
14         if err := recover(); err != nil {
15             fmt.Print("Gestione errore")
16             panic("errore", err) // rilancio il panic
17         }()
18     }()
19     c := a / b // divide by zero
20     fmt.Print(c) // non eseguito

```

10.6 Goroutine

Eseguiamo la concorrenza attraverso le **goroutine**, sono di fatto dei thread. Per avviare una goroutine da un altro thread, usiamo la parola chiave `go`.

```

1 func writeTwoTimes(s string) {
2     for i:=0; i<2: i++ {
3         time.Sleep(100 * time.Millisecond)
4         fmt.Print(s)
5     }
6 }
7
8 func main() {
9     go writeTwoTimes("A ")
10    writeTwoTimes("B ")
11    // => B A A B oppure => A B B
12 }

```

10.7 Channels

I **channel** sono una struttura che permette a due goroutine di inviarsi dati. Un thread si blocca finché l'istruzione di inviare o ricevere un elemento non si è completata. Se si vuole ricevere un elemento da un channel vuoto, il thread rimane in attesa finché non viene inserito un elemento nel channel.

Possiamo chiudere un channel con `close`, se si tenta di ricevere un elemento da un channel chiuso, si riceve l'elemento zero della classe. Se il channel è stato chiuso e si tenta di leggere o scrivere un elemento, viene restituito un errore dal compilatore.

Se si chiedono due output, il secondo è un flag che indica se il channel è aperto.

```

1 func main() {
2     ic := make(chan int) // channel (unbuffered) di int
3     go func() { // anonymous function invocata immediatamente
4         ic <- 3
5         ic <- 4
6         close(ic)
7     }()
8     time.Sleep(time.Second)
9     n1 := <- ic // ricevo il primo numero
10    fmt.Println(n1, " ") // => 3
11    n2 := <- ic // ricevo il secondo numero
12    fmt.Println(n2, " ") // => 4

```

```

13     n3, ok := <- ic // ricevo il terno numero (non presente) e il flag
14     fmt.Println(n3, " ") // => 0
15     fmt.Println(ok, " ") // => false
16 }

```

I channel possono essere di due tipi:

- **unbuffered:** in ogni channel c'è spazio per un solo elemento, l'inserimento si blocca solo se il thread trova il buffer pieno. Per esempio: `c := make(chan int)`;
- **buffered:** la capienza del buffer va settata quando si crea il buffer. Per esempio: `c := make(chan int, 3)` per un channel buffer di dimensione 3. In questo caso il l'inserimento si blocca solo quando il channel contiene già tre elementi, cioè quando è pieno.

I buffered channel sono utili per esempio quando il reciver è più lento del writer: il writer può continuare a scrivere senza interrompersi; il reciver può continuare a ricevere dati alla sua velocità.

In Go, solitamente, si creano goroutine che fungono da *writer* e altre che fungono da *reciver*, si tende a non mischiare i due comportamenti. Possiamo dichiarare questo intento esplicitamente:

```

1     func writer(ch chan<- int) {
2         ch <- 56 // scrivo
3         i := <- ch // errore: non posso leggere
4         wg.Done()
5     }
6
7     func reciver(ch <-chan int) {
8         i := <- ch
9         ch <- 4 // errore: non posso scrivere
10        wg.Done()
11    }

```

Questo comportamento viene gestito runtime dal compilatore. Sostanzialmente, il channel passato, viene castato da bidirezionale a unidirezionale.

Possiamo iterare un channel. Il ciclo rimane in attesa di ricevere dati, termina quando il channel viene chiuso.

10.8 Select

La keyword `select` permette di aspettare il primo a finire tra un gruppo di thread. Nel caso in cui il channel da cui si vuole ricevere un input sia vuoto, l'opzione di default permette di proseguire senza fermarsi.

```

1     func returnNumber(n int, c chan in) {
2         time.Sleep(time.Duration(rand.Intn(5)) * time.Millisecond)
3         c <- n
4     }
5
6     func main() {
7         rand.Seed(time.Now().UnixNano())
8         c1 := make(chan int)
9         c2 := make(chan int)
10        go returnNumber(1, c1)

```



```

11     go returnNumber(2, c2)
12     time.Sleep(time.Duration(rand.Intn(5)) * time.Millisecond)
13     select {
14         case x := <- c1:
15             fmt.Println(x)
16         case x := <- c2:
17             fmt.Println(x)
18         case <- time.After(time.Duration(rand.Intn(5)) * time.Millisecond):
19             // se nessuno finisce in tempo (caso default)
20             fmt.Println("nessun input")
21             // default: // alternativa
22             // fmt.Println("nessun input")
23     }
24     // possibili output:
25     // => 1
26     // => 2
27     // => nessun input

```

10.9 Data races

Con **data race** si intende la potenziale modifica contemporanea di un oggetto da parte di due (o più) thread. Per evitare ciò si sfrutta un buffered channel. In questo modo, ogni thread che vuole modificare o leggere tale oggetto, deve acquisirlo e infine deve reinserirlo nel channel.

```

1     func increaseNum(c chan int) {
2         n := <- c // estraggo
3         n++
4         c <- n // reinserisco
5     }
6
7     func main() {
8         c := make(chan int, 1)
9         c <- 0
10        // ...
11        go increaseNum(c)
12        go increaseNum(c)
13        time.Sleep(time.Millisecond)
14        fmt.Println(<- c) // => 2
15    }

```

Un parametro utile da passare al compilatore è `-race`. In questo modo il compilatore segnala se trova se ci sono possibili accessi multipli alla stessa locazione di memoria nello stesso istante.

10.10 WaitGroup

I **wg** vengono utilizzati per aspettare che finisca l'esecuzione di un gruppo di thread prima di passare oltre. Utilizziamo

- `add(n)`: per aggiungere all'oggetto un numero *n* di thread da aspettare;

- Done(): comunica che il thread in questione è arrivato al punto di sincronizzazione. Decrementa di uno il contatore dei thread da aspettare;
- Wait(): mette in pausa tutti i thread in questione finché non raggiungono il punto di sincronizzazione.

Nota: l'oggetto `wg` deve essere sempre passato come puntatore.

```
1 func incNum(c chan int, wg *sync.WaitGroup) {
2     n := <- c
3     n++
4     c <- n
5     wg.Done()
6 }
7
8 func main() {
9     c := make(chan int, 1)
10    c <- 0
11    // ...
12    var wg sync.WaitGroup
13    wg.Add(2)
14    go incNum(c, &wg)
15    go incNum(c, &wg)
16
17    wg.Wait()
18    fmt.Println(<- c) // => 2
19 }
```

```
1 // sincronizzare go routine
2 func someThread(i int, wg *sync.WaitGroup) {
3     fmt.Println("entro", i)
4     wg.Done()
5     wg.Wait()
6     fmt.Println("esco", i)
7 }
8
9 func main() {
10    var wg sync.WaitGroup
11    wg.Add(3)
12    go someThread(1, &wg)
13    go someThread(2, &wg)
14
15    fmt.Println("entro", 0)
16    wg.Done()
17    wg.Wait()
18    fmt.Println("esco", 0)
19 }
20
21 // => entro 0 entro 2 entro 1 esco 1 esco 2 esco 0
```